

UNIVERSITÀ DEGLI STUDI DI ROMA TOR VERGATA

MACROAREA DI INGEGNERIA



TOR VERGATA
UNIVERSITÀ DEGLI STUDI DI ROMA

CORSO DI LAUREA MAGISTRALE IN

Ingegneria dell'Automazione

TESI DI LAUREA IN

Analisi e Sintesi di Sistemi Non Lineari

TITOLO

*Progettazione e Realizzazione di un
Veicolo da Trasporto Teleoperato in CANOpen*

Relatore:

Chiar.mo Prof.
Daniele Carnevale

Laureando:

Andrea Efficace
Matricola: 0300125

Correlatore:

Roberto Masocco, Ph.D.
Ing. Moreno Mattia

Anno Accademico 2024/2025

Indice

1	Introduzione	5
1.1	Daikin Applied Europe	5
1.1.1	L'automazione e il reparto di Industrial Innovation	5
1.2	Obiettivo di tesi	6
1.3	Struttura della tesi	6
2	Robot mobili e autonomi	7
2.1	Classificazione dei robot mobili	7
2.2	Sistemi di navigazione e localizzazione	7
2.3	Algoritmi di navigazione	8
2.3.1	Pianificazione del percorso (Path Planning)	8
2.3.2	Evitamento degli ostacoli (Obstacle Avoidance)	8
2.3.3	SLAM (Simultaneous Localization and Mapping)	9
2.4	Protocolli di comunicazione industriali	10
2.4.1	CAN Bus	10
2.4.2	CANopen	11
2.4.3	Altri protocolli	12
3	Requisiti e specifiche	13
3.1	Selezione del payload	13
3.2	Requisiti del veicolo	15
3.3	Specifiche tecniche	15
3.4	Sistemi di bordo e sensoristica	17
3.5	Sistema di Movimentazione	17
3.6	Selezione componenti	19
3.6.1	Motori	19
3.6.2	Verifica dei requisiti di trazione	19
3.6.3	Batteria	21
3.6.4	Verifica dei requisiti energetici	22
3.6.5	Caricabatterie	24
3.6.6	Driver per motori elettrici	25
3.6.7	Computer di bordo	27
3.6.8	Adattatore USB-CAN	28
3.6.9	Convertitori DC-DC	31
3.7	Sensoristica	32
3.7.1	LiDAR	32
3.7.2	Encoder	35
3.7.3	Sonda di temperatura	36
3.8	Telaio e struttura meccanica	38
3.8.1	Analisi agli elementi finiti	38
3.9	Analisi dei costi	41

4	Modello cinematico del robot	45
4.1	Cinematica diretta	47
4.1.1	Modello a biciclo sterzante	48
4.1.2	Modello a doppia ruota sterzante	49
4.2	Cinematica inversa	54
4.2.1	Modello a Biciclo Sterzante	56
4.2.2	Modello a Doppia Ruota Sterzante	58
4.3	Analisi dei modelli e Simulazioni	58
4.3.1	Setup e Considerazioni Preliminari	58
4.3.2	Simulazioni in Catena Aperta	61
4.4	Controllo in Feedback	66
4.4.1	Simulazioni in Feedback	68
5	Architettura software e somunicazione	73
5.1	Problema di integrazione hardware-software	73
5.2	Architettura a livelli	74
5.3	Protocollo Waveshare USB-CAN-A	75
5.3.1	Caratteristiche del protocollo	76
5.3.2	Tipologie di frame	76
5.4	Libreria waveshare_cpp	78
5.4.1	Architettura della libreria	78
5.4.2	Costruzione e validazione dei frame	78
5.4.3	Gestione della porta seriale	78
5.4.4	Conversione tra protocolli	79
5.4.5	Testing e validazione	79
5.5	Bridge USB-SocketCAN	80
5.5.1	Architettura del bridge	80
5.5.2	Configurazione del bridge	80
5.5.3	Setup interfaccia virtuale SocketCAN	81
5.5.4	Funzionamento del bridge	81
5.6	Integrazione con lo stack ROS 2	82
5.6.1	Stack completo di comunicazione	82
5.6.2	Vantaggi dell'architettura	83
6	Conclusioni e sviluppi futuri	85
6.1	Risultati raggiunti	85
6.1.1	Analisi e progettazione del sistema	85
6.1.2	Modellazione cinematica e controllo	85
6.1.3	Architettura software e comunicazione	86
6.1.4	Stato attuale del progetto	86
6.2	Sviluppi futuri	87
6.2.1	Implementazione e integrazione	87
6.2.2	Validazione sperimentale	87
6.2.3	Estensioni e miglioramenti	88
6.3	Considerazioni finali	88

Capitolo 1

Introduzione

1.1 Daikin Applied Europe

Daikin è un'azienda multinazionale giapponese specializzata nella produzione di sistemi di climatizzazione e soluzioni per il riscaldamento. Fondata nel 1924, Daikin è diventata uno dei principali attori nel settore HVAC (Heating, Ventilation, and Air Conditioning) a livello globale. La sede *Applied* di Roma, in particolare, si occupa della progettazione e produzione di unità di trattamento aria (UTA) e sistemi di climatizzazione per applicazioni commerciali e industriali, prevalentemente rivolto al mercato europeo e con produzione di tipo *make-to-order*¹.

L'andamento crescente del mercato della climatizzazione, unito alla crescente domanda di soluzioni efficienti e personalizzate (es. datacenter, ospedali, centri commerciali), ha portato DAE a investire sempre più risorse nella ricerca e sviluppo di nuove tecnologie per migliorare non solo i prodotti offerti, ma anche i processi che ne permettono la costruzione.

1.1.1 L'automazione e il reparto di Industrial Innovation

All'interno dell'azienda, il reparto di Industrial Innovation si occupa di implementare soluzioni innovative per ottimizzare i processi produttivi e migliorare l'efficienza operativa a diversi livelli. Alcuni esempi di progetti sviluppati includono:

- l'implementazione di sistemi di monitoraggio in tempo reale della produzione mediante Manufacturing Execution System (MES);
- sistemi di collaudo automatico di sottoassiemi e prodotti finiti mediante test-bench dedicati;
- l'adozione di tecnologie di visione artificiale per il controllo qualità e l'ispezione dei componenti;
- l'integrazione di robot mobili per la movimentazione di materiali all'interno dello stabilimento.

L'adozione di queste tecnologie, in forma integrata e sinergica, ha l'obiettivo di trainare l'azienda verso un modello di industria più informatizzato ed efficiente, in linea con i principi dell'Industria 5.0.

¹Tipologia di produzione per la quale il prodotto è realizzato su specifica del cliente.

1.2 Obiettivo di tesi

L'obiettivo della tesi è stato quello di progettare e realizzare un veicolo automatico, in grado di muoversi all'interno del complesso produttivo dello stabilimento di Ariccia (RM). L'intento è quello di valutare le difficoltà e le potenzialità legate all'implementazione di un sistema di trasporto automatizzato in un ambiente industriale reale, nel quale la produzione make-to-order comporta la difficoltà di dover gestire percorsi e destinazioni variabili in funzione:

- delle esigenze produttive giornaliere;
- della disposizione degli spazi, che può variare in funzione delle necessità logistiche;
- della presenza di ostacoli dinamici, come operatori e carrelli elevatori.

Quello a cui si è voluto mirare è pertanto la transizione da una tipologia di movimentazione manuale e che richiede l'intervento umano (tramite carrelli o muletti elettrici) e un costo elevato in termini di tempo e risorse, a una soluzione automatizzata che permetta di ridurre i costi operativi e aumentare l'efficienza logistica all'interno dello stabilimento in futuro, introducendo ottimizzazioni sullo stoccaggio e la distribuzione dei materiali.

Ad esempio, collegando il veicolo a un sistema di gestione del magazzino (WMS) e a un sistema di pianificazione della produzione (ERP), sarebbe possibile automatizzare l'intero processo di movimentazione dei materiali, dalla richiesta alla consegna, riducendo al minimo l'intervento umano e gli errori associati alla ricerca e al trasporto dei componenti.

Nota: Il progetto è stato realizzato mantenendo i costi di realizzazione del veicolo il più bassi possibile, in modo da rendere il prototipo un dimostratore tecnologico economico per le potenzialità della tecnologia e valutarne l'effettiva applicabilità in un contesto industriale reale.

1.3 Struttura della tesi

Nei capitoli successivi verranno approfonditi i seguenti argomenti.

- Capitolo 2: Panoramica sulle tecnologie di veicoli autonomi, con particolare attenzione ai sistemi di navigazione e localizzazione e ai protocolli di comunicazione industriali.
- Capitolo 3: Analisi dei requisiti e specifiche del veicolo autonomo progettato per l'ambiente industriale considerato. Selezione componenti hardware. Analisi dei costi.
- Capitolo 4: Analisi del modello cinematico e proposta di algoritmi di controllo per la navigazione.
- Capitolo 5: Architettura software implementata per la gestione del veicolo e l'integrazione dei sensori.
- Capitolo 6: Possibili sviluppi futuri del progetto.

Capitolo 2

Robot mobili e autonomi

In questo capitolo verranno analizzate le principali tecnologie e metodologie utilizzate nella realizzazione di robot mobili autonomi, con particolare attenzione ai sistemi di navigazione e localizzazione.

2.1 Classificazione dei robot mobili

I robot mobili autonomi (AMR - Autonomous Mobile Robots) sono veicoli in grado di muoversi e operare in ambienti complessi senza la necessità di un controllo umano diretto. Questi robot sono dotati di sensori, attuatori e algoritmi di controllo e localizzazione che consentono loro di percepire l'ambiente circostante, prendere decisioni e navigare in modo sicuro per se stessi e gli elementi presenti nell'ambiente (es operatori, ostacoli, ecc.). Alcuni esempi di robot mobili autonomi sono i seguenti.

- **Robot a ruote:** utilizzano ruote per la locomozione e sono adatti per superfici piane e lisce. Esempi includono carrelli elevatori autonomi e robot di consegna.
- **Robot a cingoli:** utilizzano cingoli per la locomozione e sono adatti per terreni irregolari e accidentati. Sono utili per lavori di esplorazione e soccorso in ambienti difficili.
- **Robot a gambe:** utilizzano sistemi che ricordano arti umani o animali e sono in grado di affrontare terreni complessi e ostacoli. Di particolare interesse negli ultimi anni sono i robot quadrupedi, come quelli sviluppati da Boston Dynamics.
- **Robot volanti:** utilizzano eliche o ali per la propulsione e sono in grado di volare. Esempi includono droni e veicoli aerei senza pilota (UAV).

Una categoria analoga ai robot mobili autonomi sono i veicoli a guida autonoma (AGV - Automated Guided Vehicles), che sono progettati per operare in ambienti industriali e di magazzino, seguendo percorsi predefiniti e ben strutturati che consentono di evitare logiche di navigazione complesse. Un esempio comune sono carrelli detti *seguì-linea*, che seguono linee tracciate sul pavimento o nastri magnetici per spostarsi all'interno di un magazzino o di una fabbrica.

2.2 Sistemi di navigazione e localizzazione

I robot mobili autonomi utilizzano una combinazione di sensori, algoritmi di localizzazione e tecniche di mappatura per navigare in ambienti complessi. Alcuni dei principali sistemi e tecnologie utilizzati sono a seguire.

- **Sensori di prossimità:** utilizzati per rilevare ostacoli e misurare distanze. Esempi comuni includono sensori a ultrasuoni, sensori infrarossi e LIDAR (Light Detection and Ranging).
- **Sistemi di visione:** Telecamere, sensori di profondità e marker (es. QR-code) vengono utilizzati per acquisire informazioni visive sull'ambiente circostante, consentendo al robot di riconoscere oggetti e ostacoli.
- **Sistemi di localizzazione:** tecnologie come GPS, SLAM (Simultaneous Localization and Mapping) e sistemi di localizzazione basati su visione vengono utilizzati per determinare la posizione del robot nell'ambiente.
- **Mappe ambientali:** i robot possono utilizzare mappe predefinite o creare mappe in tempo reale dell'ambiente circostante per facilitare la navigazione.

2.3 Algoritmi di navigazione

Gli algoritmi di navigazione sono fondamentali per consentire ai robot mobili autonomi di pianificare e seguire traiettorie sicure ed efficienti. Di seguito vengono presentati i principali approcci utilizzati.

2.3.1 Pianificazione del percorso (Path Planning)

La pianificazione del percorso consiste nel determinare una traiettoria che colleghi un punto di partenza a una destinazione, evitando gli ostacoli presenti nell'ambiente. I principali algoritmi includono i seguenti.

- **A* (A-star)** [12]: algoritmo di ricerca su grafi che trova il percorso ottimale combinando il costo effettivo del tragitto con una stima euristica della distanza rimanente. È ampiamente utilizzato per la sua efficienza e garanzia di ottimalità.
- **Dijkstra** [7]: algoritmo classico che esplora sistematicamente tutti i nodi di un grafo per trovare il percorso più breve. Garantisce la soluzione ottimale ma può risultare computazionalmente oneroso in ambienti ampi.
- **RRT (Rapidly-exploring Random Trees)** [15]: algoritmo probabilistico che costruisce un albero di esplorazione espandendosi casualmente nello spazio. È particolarmente adatto per spazi ad alta dimensionalità e ambienti complessi.

2.3.2 Evitamento degli ostacoli (Obstacle Avoidance)

Una volta pianificato il percorso, il robot deve essere in grado di reagire a ostacoli imprevisti o dinamici. Le tecniche più comuni sono le seguenti.

- **VFH (Vector Field Histogram)** [1]: crea un istogramma polare delle distanze dagli ostacoli e seleziona la direzione di movimento più sicura e vicina all'obiettivo.

- **DWA (Dynamic Window Approach)** [10]: valuta un insieme di velocità ammissibili considerando i vincoli cinematici del robot e seleziona quella che massimizza il progresso verso l'obiettivo minimizzando il rischio di collisione.
- **Potential Fields** [14]: modella l'obiettivo come un attrattore e gli ostacoli come repulsori, generando un campo di forze che guida il movimento del robot.

2.3.3 SLAM (Simultaneous Localization and Mapping)

Lo SLAM è una tecnica che permette al robot di costruire una mappa dell'ambiente circostante mentre simultaneamente si localizza al suo interno [8]. Questo approccio è fondamentale quando non è disponibile una mappa predefinita. Gli algoritmi SLAM più diffusi includono i seguenti.

- **EKF-SLAM**: basato sul filtro di Kalman esteso, adatto per ambienti di dimensioni limitate.
- **Particle Filter SLAM** [18]: utilizza un insieme di particelle per rappresentare le possibili posizioni del robot, robusto in presenza di incertezze elevate.
- **Graph-based SLAM** [11]: rappresenta il problema come un grafo di vincoli tra pose successive, ottimizzando globalmente la traiettoria stimata.

La scelta dell'algoritmo dipende dalle caratteristiche specifiche dell'applicazione come la complessità dell'ambiente, i requisiti di determinismo e predicibilità d'esecuzione, la sensoristica e le risorse computazionali disponibili.

FAST-LIO 2

Come spiegato nei capitoli successivi, si è scelto di equipaggiare il robot con un sensore LiDAR 3D dotato di IMU (Inertial Measurement Unit). Conseguentemente, si è scelto di adottare un algoritmo di SLAM che permettesse di integrare efficacemente i dati provenienti da questi sensori.

La scelta è ricaduta su **FAST-LIO 2** [20], un algoritmo di odometria LiDAR-inerziale (LIO) ad alte prestazioni sviluppato presso l'Università di Hong Kong. FAST-LIO 2 è progettato per fornire stima dello stato in tempo reale con elevata accuratezza, anche in ambienti complessi e con risorse computazionali limitate.

L'algoritmo si basa su un filtro di Kalman esteso iterato (*iterated Extended Kalman Filter*, iEKF) strettamente accoppiato (*tightly-coupled*), che fonde direttamente le misure grezze del LiDAR con i dati dell'IMU. A differenza degli approcci tradizionali basati sull'estrazione di feature, FAST-LIO 2 utilizza un approccio *direct* che registra direttamente i punti della nuvola LiDAR sulla mappa, eliminando la necessità di estrarre caratteristiche geometriche come spigoli e superfici planari.

Le principali innovazioni di FAST-LIO 2 includono le seguenti.

- **Registrazione diretta dei punti**: l'algoritmo registra ogni punto della scansione LiDAR direttamente sulla mappa globale, senza passare per l'estrazione di feature. Questo approccio migliora la robustezza in ambienti con poche caratteristiche geometriche distintive.

- **ikd-Tree:** una struttura dati incrementale basata su k-d tree che permette l'inserimento, la cancellazione e la ricerca dei punti più vicini in modo efficiente. Questa struttura consente di mantenere e aggiornare la mappa in tempo reale con un overhead computazionale minimo.
- **Compensazione del moto:** i dati dell'IMU vengono utilizzati per compensare la distorsione della nuvola di punti causata dal movimento del sensore durante la scansione, garantendo una ricostruzione accurata dell'ambiente.
- **Calcolo efficiente del guadagno di Kalman:** l'algoritmo sfrutta la struttura sparsa del problema per calcolare il guadagno di Kalman in modo efficiente, riducendo la complessità computazionale.

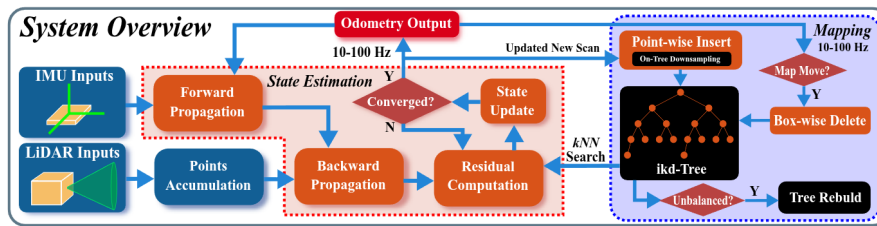


Figura 2.1: Diagramma a blocchi dell'algoritmo FAST-LIO 2 [20].

Grazie a queste caratteristiche, FAST-LIO 2 è in grado di operare a frequenze elevate (superiori a 100 Hz su sistemi embedded) mantenendo un'accuratezza paragonabile o superiore allo stato dell'arte, rendendolo particolarmente adatto per applicazioni robotiche che richiedono localizzazione in tempo reale.

2.4 Protocolli di comunicazione industriali

Nei sistemi robotici mobili, la comunicazione tra i diversi componenti (sensori, attuatori, controllori) riveste un ruolo fondamentale. In ambito industriale e automotive, sono stati sviluppati diversi protocolli di comunicazione che garantiscono affidabilità, determinismo temporale e robustezza in ambienti operativi critici.

2.4.1 CAN Bus

Il **CAN** (Controller Area Network) [9] è un protocollo di comunicazione seriale sviluppato da Bosch negli anni '80, originariamente per applicazioni automotive. Si tratta di un bus multi-master che permette a più nodi di comunicare senza la necessità di un controllore centrale. Le caratteristiche principali sono le seguenti.

- **Arbitraggio non distruttivo:** in caso di trasmissioni simultanee, il messaggio con priorità più alta prevale senza corrompere i dati.
- **Rilevamento e gestione degli errori:** il protocollo include meccanismi robusti di error detection (CRC, bit stuffing, frame check) e gestione automatica degli errori.

- **Velocità:** Fino a 1 Mbit/s per CAN classico, con estensioni come CAN-FD che raggiungono velocità superiori.
- **Affidabilità:** Progettato per ambienti con elevate interferenze elettromagnetiche, tipici del settore automotive e industriale.

2.4.2 CANopen

CANopen [3] è un protocollo di alto livello basato su CAN, standardizzato dalla CiA (CAN in Automation) e ampiamente utilizzato in automazione industriale, robotica e veicoli elettrici. Mentre CAN definisce solo il livello fisico e di collegamento dati, CANopen aggiunge un livello applicativo che standardizza la comunicazione tra dispositivi eterogenei.

Le caratteristiche fondamentali di CANopen includono le seguenti.

- **Object Dictionary (OD):** ogni dispositivo CANopen espone un *dizionario di oggetti* che descrive tutte le sue funzionalità, parametri e dati. Questo approccio permette una configurazione standardizzata e l'interoperabilità tra dispositivi di produttori diversi.
- **PDO (Process Data Objects):** messaggi ad alta priorità per lo scambio di dati di processo in tempo reale, come comandi di velocità o letture di encoder. I PDO sono configurabili e permettono di ottimizzare la larghezza di banda.
- **SDO (Service Data Objects):** messaggi per la configurazione e la lettura/scrittura di parametri nel dizionario oggetti. Utilizzano un protocollo client-server con conferma.
- **NMT (Network Management):** servizi per la gestione dello stato dei nodi (*pre-operational*, *operational*, *stopped*) e il monitoraggio della rete tramite *heartbeat* e *node guarding*.
- **SYNC e EMCY:** messaggi di sincronizzazione per coordinare le operazioni dei nodi e messaggi di emergenza per segnalare condizioni di errore.

Nel contesto del veicolo a guida autonoma oggetto di questa tesi, CANopen è stato scelto come protocollo di comunicazione principale per il controllo degli attuatori (motori di trazione e sterzo). Questa scelta è stata motivata da questi fattori.

- **Standardizzazione dei profili di dispositivo:** CANopen definisce profili standard per diverse classi di dispositivi (CiA 402 per azionamenti, CiA 401 per I/O). Il profilo CiA 402 [2], in particolare, specifica una macchina a stati e modalità operative standard per il controllo di motori elettrici.
- **Determinismo temporale:** la comunicazione basata su PDO sincronizzati garantisce latenze predicibili, essenziali per il controllo in tempo reale del veicolo.
- **Diagnostica integrata:** i meccanismi di heartbeat ed emergency permettono di rilevare rapidamente guasti o disconnessioni dei nodi, aumentando la sicurezza del sistema.
- **Scalabilità:** è possibile aggiungere nuovi dispositivi alla rete senza modificare l'architettura esistente.

2.4.3 Altri protocolli

Oltre a CAN e CANopen, esistono altri protocolli utilizzati in contesti simili, ciascuno con caratteristiche e ambiti di applicazione specifici.

- **EtherCAT** [4]: protocollo Ethernet industriale ad alte prestazioni sviluppato da Beckhoff, con latenze nell'ordine dei microsecondi. A differenza di CANopen, EtherCAT utilizza Ethernet come mezzo fisico e sfrutta un'architettura master-slave con elaborazione "al volo" dei frame. Offre prestazioni superiori in termini di velocità e sincronizzazione ma richiede hardware dedicato e risulta più complesso da implementare. È la scelta preferita per applicazioni dove il controllo deve operare a rate elevati e con minima latenza.
- **PROFINET** [5]: standard Ethernet industriale promosso da Siemens, diffuso nell'automazione industriale. Rispetto a CANopen, PROFINET offre maggiore larghezza di banda e integrazione nativa con i sistemi Siemens ma è meno diffuso nel settore della robotica mobile e dei veicoli elettrici. La versione PROFINET IRT (Isochronous Real-Time) garantisce determinismo paragonabile a EtherCAT.
- **Modbus** [17]: protocollo seriale sviluppato da Modicon nel 1979, uno dei più semplici e diffusi nell'automazione industriale. Utilizza un'architettura master-slave con comunicazione request-response. Rispetto a CANopen, Modbus è più semplice da implementare ma offre funzionalità limitate: non prevede meccanismi nativi di sincronizzazione, gestione degli errori avanzata o profili di dispositivo standardizzati. È adatto per applicazioni con requisiti di determinismo d'esecuzione meno stringenti.
- **Modbus TCP** [16]: estensione di Modbus su rete Ethernet/TCP-IP. Mantiene la semplicità del protocollo originale aggiungendo i vantaggi dell'infrastruttura Ethernet (maggiore velocità, distanze più lunghe, integrazione con reti IT). Tuttavia, l'utilizzo di TCP introduce latenze non deterministiche, rendendolo meno adatto rispetto a CANopen per il controllo in tempo reale di attuatori.

La scelta del protocollo dipende dai requisiti specifici dell'applicazione in termini di latenza, throughput, numero di nodi e compatibilità con i dispositivi disponibili. Nel caso del veicolo oggetto di questa tesi, CANopen rappresenta il miglior compromesso tra prestazioni, garanzie, standardizzazione dei profili per azionamenti elettrici, e disponibilità di componenti compatibili.

Capitolo 3

Requisiti e specifiche

3.1 Selezione del payload

L'idea iniziale del progetto è stata quella di movimentare le sezioni ventilanti (SV) che compongono il sottoassieme superiore dei *chiller* prodotti nello stabilimento. La ragione di questo obiettivo risiede nel fatto che le SV sono componenti di grandi dimensioni, che vengono prodotte all'interno di uno dei capannoni dello stabilimento e successivamente poste negli spazi esterni in piazzole sparse in diversi punti, in attesa di essere assemblate sul prodotto finito.

Nota: Attualmente, una volta che una SV è pronta per passare alla fase di assemblaggio successiva, viene calettata su appositi supporti con ruote *caster* e trasportate tramite dei muletti elettrici. Questo richiede l'intervento di personale specializzato all'uso di carroponti e muletti, con conseguente dispendio di tempo e risorse.

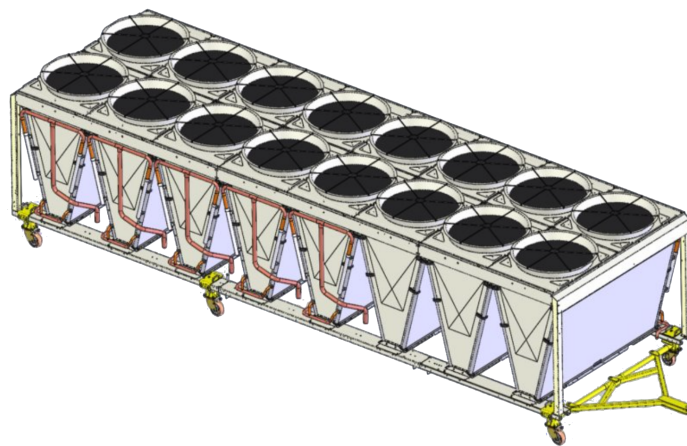


Figura 3.1: Modello 3D di una Sezione Ventilante di un chiller, formata da 8 **moduli a V**. Ogni modulo, ha dimensioni $(L \times W \times H)m = (2 \times 1 \times 1.5)m$. L'intero sottoassieme ha quindi dimensioni complessive di circa $(8 \times 2 \times 1.5)m$. Si possono apprezzare in giallo i supporti con ruote *caster* su cui viene calettata la SV.

Data la difficoltà di dover gestire delle strutture così ingombranti in un ambiente produttivo già di per sé complesso, si è deciso di ridefinire l'obiettivo del progetto, orientandolo la movimentazione dei compressori *a vite*, componenti di taglia standard dal peso contenuto. Questi sono componenti fon-

damentali per il funzionamento dei chiller, in quanto sono responsabili della compressione del refrigerante e del suo trasferimento attraverso il sistema di climatizzazione per attuare il ciclo termico e pertanto è di interesse primario una gestione logistica efficiente di questi componenti all'interno dello stabilimento.

Nota: I compressori a vite vengono prodotti in un capannone dedicato e distaccato rispetto a quello dove vengono prodotti i chiller. Una volta prodotti, questi vengono posti in una piazzola di stoccaggio esterna in attesa di essere prelevati e trasportati al reparto di assemblaggio dei chiller. Il trasporto avviene tramite muletti elettrici che inforcano delle pedane in ferro su cui sono posizionati i compressori.

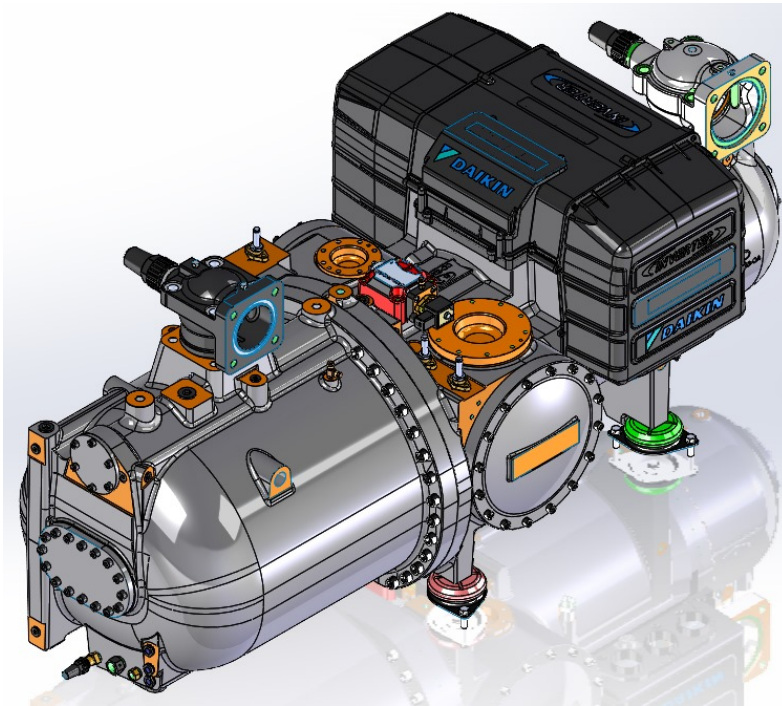


Figura 3.2: Compressore a vite Daikin, modello 35-A-CO1005. Peso: ~ 730 kg. Dimensioni: $(L \times W \times H)m = (1.9 \times 0.9 \times 0.75)m$.

In figura 3.2 possiamo vedere un render del modello di compressori più grande e pesante attualmente in produzione. La scelta di questo payload ha permesso di definire requisiti più gestibili in termini di capacità di carico e dimensioni del veicolo, ottenendo comunque un valido dimostratore tecnologico. Si è comunque mantenuta l'idea di dimensionare i motori e la struttura del veicolo in modo da poter essere scalabile in futuro, qualora si volesse aumentare la capacità di carico per movimentare componenti più grandi come le SV.

3.2 Requisiti del veicolo

In base alla scelta del payload e all'ambiente operativo, sono stati definiti i seguenti requisiti funzionali e prestazionali per il veicolo a guida autonoma.

- **Limitare i costi:** il progetto deve mirare a contenere i costi, usando il minimo numero di componenti necessari per raggiungere gli obiettivi prefissati e privilegiando soluzioni economiche ma affidabili.
- **Capacità di carico:** il veicolo deve essere in grado di trasportare un carico utile minimo di 1000 kg, per garantire la movimentazione sicura del compressore a vite e di eventuali accessori o attrezzature aggiuntive.
- **Velocità massima:** la velocità massima del veicolo deve essere al massimo di 1 m/s, per garantire la sicurezza degli operatori e la stabilità del carico durante il trasporto.
- **Tipologia di motori:** tutti i motori (trazione e sterzo) devono essere elettrici, per ridurre l'impatto ambientale e facilitare la manutenzione.
- **Alimentazione:** il veicolo deve essere alimentato a batteria, per garantire l'autonomia operativa e la flessibilità di movimento all'interno dello stabilimento.
- **Autonomia operativa:** il veicolo deve essere in grado di operare almeno per 4 ore prima di dover essere ricaricato.
- **Navigazione e localizzazione:** il veicolo deve essere in grado di asservire traiettorie preimpostate all'interno dello stabilimento, come parte delle lavorazioni in cui è coinvolto, individuando ostacoli statici e dinamici per garantire la sicurezza degli operatori circostanti.
- **Manovrabilità:** il veicolo deve essere in grado di effettuare manovre quali: traiettorie rettilinee, curve morbide, rotazioni sul posto e traslazioni laterali.
- **Spazio operativo:** il veicolo deve essere in grado di operare in esterni, su asfalto e cemento, e in interni, su pavimentazioni industriali.

3.3 Specifiche tecniche

Le specifiche tecniche che deve avere il veicolo sono state calcolate considerando i suoi requisiti funzionali e prestazionali. Sono stati considerati i seguenti parametri, sovradimensionati per poter avere un adeguato margine di tolleranza.

- **Carico utile:** (robot e payload) 1000 kg
- **Velocità massima:** 1 m/s
- **Autonomia operativa:** 4 h
- **Raggio delle ruote:** 0.1 m

- **Pendenza massima affrontabile:** 10%, presa rispetto a una rampa di 0.3m di altezza per 3m di lunghezza; corrispondente a circa 5.7° .
- **Attrito statico:** coefficiente di attrito statico tra ruota e asfalto $\mu_s = 0.5$.
- **Attrito volvente:** coefficiente di attrito volvente tra ruota e asfalto $\mu_r = 0.03$.
- **Vel. & acc. max:** supponendo di voler raggiungere una velocità massima di 1 m/s in 3 s, si ha un'accelerazione di $\sim 0.333 \text{ m/s}^2$.

Da questi dati possono essere ricavate le specifiche tecniche necessarie affinché avvenga il primo distacco.

- In piano, occorre vincere

$$\begin{aligned} F_r &= \mu_r \cdot m \cdot g = 0.03 \cdot 1000 \cdot 9.81 = 294.3 \text{ N}, \\ F_a &= m \cdot a = 1000 \cdot 0.333 = 333 \text{ N}, \\ F_{tot} &= F_r + F_a = 294.3 + 333 = 627.3 \text{ N}, \end{aligned} \quad (3.1)$$

quindi, la coppia totale richiesta è

$$\tau_{tot} = F_{tot} \cdot r = 627.3 \cdot 0.1 = 62.73 \text{ Nm}, \quad (3.2)$$

che corrisponde a una potenza totale di

$$P_{tot} = F_{tot} \cdot v = 627.3 \cdot 1 = 627.3 \text{ W}. \quad (3.3)$$

- Su una pendenza del 10%, occorre vincere

$$\begin{aligned} F_{\parallel} &= m \cdot g \cdot \sin(\theta) = 1000 \cdot 9.81 \cdot \sin(5.7^\circ) = 973.6 \text{ N}, \\ F_r &= \mu_r \cdot m \cdot g \cdot \cos(\theta) = 0.03 \cdot 1000 \cdot 9.81 \cdot \cos(5.7^\circ) = 293.1 \text{ N}, \\ F_a &= m \cdot a = 1000 \cdot 0.333 = 333 \text{ N}, \\ F_{tot} &= F_{\parallel} + F_r + F_a = 973.6 + 293.1 + 333 = 1599.7 \text{ N}, \end{aligned} \quad (3.4)$$

quindi, la coppia totale richiesta a ciascuna ruota è

$$\tau_{tot} = F_{tot} \cdot r = 1599.7 \cdot 0.1 = 159.97 \text{ Nm}, \quad (3.5)$$

che corrisponde a una potenza totale di

$$P_{tot} = F_{tot} \cdot v = 1599.7 \cdot 1 = 1599.7 \text{ W}. \quad (3.6)$$

Approssimando la potenza totale necessaria a 1 kW, si è deciso di richiedere per la batteria una capacità energetica di almeno 4 kWh per garantire l'autonomia operativa di 4 ore.

Nota: In questa trattazione preliminare, non è stato calcolato l'effetto inerziale delle masse in rotazione né tantomeno le coppie necessarie ad eventuali motori di sterzo per orientare le ruote una volta messe sotto carico e/o in movimento. Questi aspetti più complessi sono stati demandati ai fornitori.

3.4 Sistemi di bordo e sensoristica

Al fine di ottenere un sistema in grado di navigare autonomamente e di poter controllare i motori, si è deciso di dotare il veicolo di un sistema di bordo composto da:

- **Computer di bordo:** un *Single Board Computer* (SBC) con capacità di calcolo sufficiente per eseguire gli algoritmi di navigazione e controllo in tempo reale. Si è optato per una soluzione basata su architettura x86 per garantire compatibilità con una vasta gamma di software e librerie, data anche la necessità di utilizzare Linux e ROS 2 come sistema operativo e middleware rispettivamente.
- **Inverter per motori PMAC:** per il controllo dei motori di trazione e sterzo, si è scelto di utilizzare inverter compatibili con motori brushless a magneti permanenti (PMAC), in grado di fornire la potenza richiesta e di supportare protocolli di comunicazione industriali come CANOpen.
- **Batteria LiFePO4:** per l'alimentazione del veicolo, si è optato per una batteria al litio-ferro-fosfato (LiFePO4) da almeno 4 kWh, in grado di garantire l'autonomia operativa richiesta e di supportare cicli di carica/scarica profondi senza degradazione significativa.
- **Sistema di comunicazione CAN:** per l'integrazione e il controllo dei vari componenti del veicolo, si è scelto di utilizzare un adattatore USB-CAN, che permetta la comunicazione tra il computer di bordo e gli inverter dei motori in CAN bus.

Per la sensoristica, si è cercato di contenere il numero di sensori al minimo per ridurre i costi, selezionando solo quelli strettamente necessari per la navigazione autonoma e il controllo del veicolo.

- **LIDAR 3D:** per la mappatura tridimensionale dell'ambiente circostante e l'individuazione di ostacoli. Il cono di visione del sensore scelto deve coprire almeno 180° in orizzontale e 30° in verticale, con una portata minima di 10 m.
- **Encoder sui motori:** per la misurazione della velocità, della posizione degli sterzi e del numero di giri dei motori di trazione per un calcolo a valle della stima della posizione del veicolo tramite odometria.

Nota: La scelta di non utilizzare telecamere è stata dettata dalla volontà di evitare problemi legati alla privacy degli operatori nelle prossimità del robot nonché alla gestione sicura della trasmissione di immagini acquisite all'interno dell'impianto industriale.

3.5 Sistema di Movimentazione

Per soddisfare i requisiti di manovrabilità, è risultata da subito ovvia la necessità di prevedere meccanismi di sterzo al fine di garantire la riuscita di manovre come rotazioni sul posto e traslazioni laterali. Tuttavia, introdurre meccanismi di sterzo avrebbe comportato un aumento della complessità meccanica andando a incidere sui costi complessivi del progetto. Si è quindi

proceduto ad analizzare diverse configurazioni di ruote motrici e sterzanti per trovare un compromesso tra le richieste esplicitate. Le configurazioni prese in considerazione sono state:

1. **4 ruote motrici e sterzanti:** offre la massima trazione e manovrabilità, ma comporta costi elevati e complessità meccanica.
2. **2 ruote motrici + 2 ruote fisse:** compromesso tra trazione e semplicità, ma con limitata capacità di sterzata tramite guida differenziale.
3. **2 ruote motrici e sterzanti + 2 ruote fisse:** buona manovrabilità e trazione, con costi e complessità moderate. Simile ad una configurazione di veicolo tradizionale.
4. **2 ruote motrici e sterzanti + 2 ruote caster:** migliore manovrabilità se le attuazioni sono posizionate opportunamente, con riduzione dei costi e della complessità meccanica.

Dopo un'attenta valutazione, si è deciso di adottare la configurazione a due ruote motrici e sterzanti più due ruote caster in quanto offre un buon equilibrio tra manovrabilità, trazione, costi e semplicità meccanica. Le ruote motrici e sterzanti sono state posizionate su una diagonale del veicolo, per massimizzare l'efficacia della sterzata e permettere manovre come rotazioni sul posto e traslazioni laterali. Le ruote caster, invece, sono state posizionate sull'altra diagonale per garantire stabilità e supporto al veicolo durante il movimento.

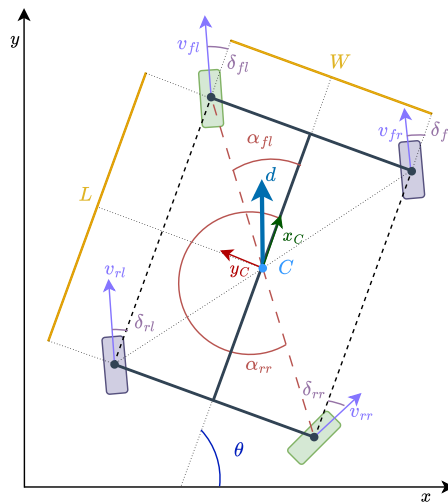


Figura 3.3: Schema della configurazione a 2 ruote motrici e sterzanti (in verde) e 2 ruote caster (in viola).

3.6 Selezione componenti

Sulla base delle specifiche tecniche calcolate e della configurazione di ruote scelta, si è proceduto alla selezione dei componenti principali del veicolo.

3.6.1 Motori

Per gruppo motori, si è scelto di acquistare delle *motoruote*. Una motoruota integra in un unico componente il motore elettrico, la ruota stessa e il sistema di trasmissione, semplificando notevolmente la progettazione meccanica e riducendo i costi di assemblaggio. Infatti, possono essere montate direttamente sul telaio del veicolo, a patto di predisporre opportune staffe di supporto.

Il prodotto acquistato è la MRT05 di CFR, che combina inoltre un motore di sterzo posizionato verticalmente rispetto al piano della ruota, permettendo di ottenere la sterzata mediante un accoppiamento ad ingranaggi. Questo consente di ottenere un sistema di sterzo compatto e integrato, riducendo ulteriormente la complessità meccanica.



Figura 3.4: Render della motoruota MRT05 di CFR.

3.6.2 Verifica dei requisiti di trazione

Verifichiamo ora che i motori selezionati soddisfino i requisiti di coppia e potenza calcolati in precedenza.

Requisiti di trazione:

- in piano: coppia totale richiesta $\tau_{richiesta} = 62.73 \text{ Nm}$, potenza $P_{richiesta} = 627.3 \text{ W}$;
- su rampa 10%: coppia totale richiesta $\tau_{richiesta} = 159.97 \text{ Nm}$, potenza $P_{richiesta} = 1599.7 \text{ W}$.

Caratteristiche dei motori:

- coppia nominale per ruota: 37 Nm;

Tabella 3.1: Specifiche tecniche dei motori di trazione e sterzo.

Parametro	Trazione	Sterzo	Unità
Tipo	BR Brushless	BR Brushless	-
Potenza (S2 - 60 min)	500	300	W
Tensione batteria	48	48	V
Velocità motore	2300	2800	RPM
Frequenza	76	93	Hz
Coppia motore	2	1	Nm
Rapporto di riduzione	1:21	1:45.56	-
Coppia in uscita	37	34	Nm
Coppia max (acc./frenata)	180	-	Nm
Grado di protezione	IP65	IP65	-
Lubrificazione	Grasso	Grasso	-
Codice motore PMAC	BD 090 - 50 - 4	BD 090 - 25 - 4	-

Tabella 3.2: Peso complessivo del gruppo ruota motrice.

Componente	Valore	Unità
Peso totale gruppo ruota	42	kg

- coppia di picco per ruota: 180 Nm;
- potenza nominale per ruota (S2 - 60 min): 500 W.

Verifica in piano: la coppia totale disponibile con 2 motoruote è

$$\tau_{disponibile} = 2 \times \tau_{nominale} = 2 \times 37 = 74 \text{ Nm} \geq \tau_{richiesta} = 62.73 \text{ Nm}. \quad \checkmark \quad (3.7)$$

La forza di trazione massima erogabile in condizioni nominali è

$$F_{trazione} = \frac{\tau_{disponibile}}{r} = \frac{74}{0.099} \approx 747 \text{ N} \geq F_{richiesta} = 627.3 \text{ N}. \quad \checkmark \quad (3.8)$$

Il margine di sicurezza in piano risulta

$$\text{Margine}_{\tau} = \frac{74 - 62.73}{62.73} \times 100 \approx 18\%. \quad (3.9)$$

Verifica su rampa 10%: nel caso peggiore (salita 10% con accelerazione $a = 0.33 \text{ m/s}^2$), la forza richiesta è

$$F_{richiesta} = m \cdot g \cdot \sin(\theta) + \mu_r \cdot m \cdot g \cdot \cos(\theta) + m \cdot a = 976 + 293 + 333 = 1602 \text{ N}, \quad (3.10)$$

la coppia corrispondente è

$$\tau_{richiesta} = F_{richiesta} \cdot r = 1602 \times 0.099 \approx 159 \text{ Nm}. \quad (3.11)$$

Questa condizione supera la coppia nominale ma rientra ampiamente nella coppia di picco disponibile:

$$\tau_{picco} = 2 \times 180 = 360 \text{ Nm} \gg \tau_{richiesta} = 159 \text{ Nm}. \quad \checkmark \quad (3.12)$$

La durata della rampa è

$$t_{rampa} = \frac{L}{v} = \frac{3}{1} = 3 \text{ s} \ll t_{picco, max}. \quad (3.13)$$

In conclusione, i motori selezionati sono adeguati per operazioni in piano con un margine di sicurezza del 18%, e possono gestire rampe fino al 10% utilizzando la coppia di picco per brevi periodi (3 secondi), ben al di sotto del limite temporale consentito.

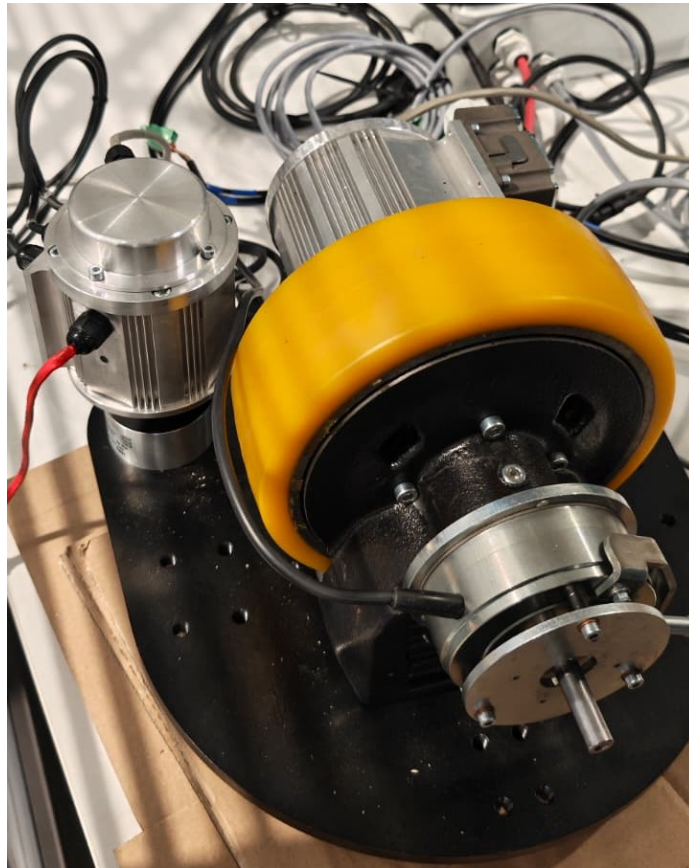


Figura 3.5: Foto reale della motoruota MRT05 di CFR su un banco di prova. Si possono notare i due motori e la piastra di supporto.

3.6.3 Batteria

Per l'alimentazione del veicolo, si è scelto di utilizzare una batteria con tecnologia LiFePO₄ (Litio-Ferro-Fosfato). Questa tecnologia offre numerosi vantaggi, tra cui i seguenti.

- **Sicurezza:** le batterie LiFePO₄ sono più stabili termicamente e chimicamente rispetto ad altre tecnologie al litio, riducendo il rischio di incendi o esplosioni.
- **Durata:** le batterie LiFePO₄ hanno una vita utile più lunga, con un numero maggiore di cicli di carica/scarica rispetto ad altre tecnologie.
- **Prestazioni:** le batterie LiFePO₄ offrono buone prestazioni in termini di capacità di scarica ad alta corrente, rendendole adatte per applicazioni che richiedono picchi di potenza.

Su queste basi, la ricerca del fornitore per tale componente si è concentrata su aziende che operassero nel campo dei veicoli elettrici. Dopo un'attenta valutazione, si è deciso di affidarsi a *FlashBattery*, un'azienda italiana specializzata nel settore, che offre soluzioni dotate di sistemi di gestione della batteria (BMS) avanzati con comunicazione in CANOpen. Sulle base delle specifiche tecniche calcolate in precedenza, il fornitore ha proposto una batteria con le seguenti caratteristiche.

Tabella 3.3: Specifiche tecniche della batteria LiFePO4.

Parametro	Valore	Unità
Tensione nominale	51.2	V
Capacità nominale	105	Ah
Chimica	LiFePO4	-
Tensione massima	59.2	V
Tensione minima	44	V
Corrente di scarica	105 (continua)	A
Corrente di scarica picco ($t \leq 60s$)	210	A
Corrente di carica massima	50	A
Profondità di scarica (DOD)	80	%
Tensione di rigenerazione massima	56	V
Corrente di rigenerazione massima	210	A
Energia totale	5.38	kWh
Energia disponibile (80% DOD)	4.30	kWh
Temperatura operativa	-10 / +45	°C
Auto-scarica (@25°C)	3 / mese	%
Auto-scarica (@50°C)	18 / mese	%

Tabella 3.4: Specifiche meccaniche della batteria.

Parametro	Valore	Unità
Lunghezza	510	mm
Larghezza	320	mm
Altezza	275	mm
Peso stimato	55	kg
Materiale	Acciaio al carbonio	-
Finitura esterna	RAL 3002 (rosso)	-
Grado di protezione	IP65	-

3.6.4 Verifica dei requisiti energetici

Si presenta ora un confronto tra i requisiti energetici calcolati in precedenza e le caratteristiche della batteria selezionata.

Requisito energetico: era stato calcolato un fabbisogno di almeno 4 kWh per garantire 4 ore di autonomia operativa con una potenza media stimata di 1 kW.

Caratteristiche della batteria:

- energia totale: 5.38 kWh;
- energia disponibile (80% DOD): 4.30 kWh.



Figura 3.6: Foto del pacco batteria (rosso) e del caricabatterie (nero) forniti da FlashBattery, alloggiati su un carrello di prova. Si possono notare in rosso/nero e in giallo i cavi di alimentazione e comunicazione.

Verifica della capacità energetica:

$$E_{disponibile} = 4.30 \text{ kWh} \geq E_{richiesta} = 4 \text{ kWh}. \quad \checkmark \quad (3.14)$$

La batteria soddisfa il requisito minimo con un margine di sicurezza del

$$\text{Margine} = \frac{E_{disponibile} - E_{richiesta}}{E_{richiesta}} \times 100 = \frac{4.30 - 4}{4} \times 100 = 7.5\%. \quad (3.15)$$

Verifica della corrente di scarica: la potenza massima richiesta in condizioni critiche (rampa 10%) è $P_{max} = 1.6 \text{ kW}$. La corrente corrispondente alla tensione nominale è

$$I_{richiesta} = \frac{P_{max}}{V_{nominale}} = \frac{1600}{51.2} \approx 31.25 \text{ A}. \quad (3.16)$$

La batteria fornisce

- corrente continua: $105 \text{ A} \gg 31.25 \text{ A}$; $\checkmark$
- corrente di picco ($t \leq 60\text{s}$): 210 A . $\checkmark$

In conclusione, la batteria selezionata è adeguata ai requisiti del veicolo, offrendo capacità energetica sufficiente per l'autonomia richiesta e correnti di scarica ampiamente superiori alle necessità operative, garantendo così un margine di sicurezza per eventuali picchi di potenza o inefficienze del sistema.

3.6.5 Caricabatterie

Insieme al pacco batteria, FlashBattery ha fornito un caricabatterie dedicato progettato specificamente per batterie al litio-ferro-fosfato (LiFePO₄). Il modello fornito è lo Zivan NG3 configurato per batterie 48V con corrente di carica massima di 60A, un caricabatterie industriale di alta qualità prodotto da Zivan, azienda leader nel settore dei sistemi di ricarica per veicoli elettrici e applicazioni industriali.

Il caricabatterie implementa un algoritmo di carica ottimizzato per la chimica LiFePO₄, garantendo cicli di carica completi e sicuri che massimizzano la vita utile della batteria.

Di seguito le sue caratteristiche tecniche principali.

- **Modello:** Zivan NG3 48V 60A.
- **Tensione di ingresso:** alimentazione da rete elettrica monofase 230V AC (50-60 Hz).
- **Tensione di uscita:** configurata per batterie LiFePO₄ a 48V nominali (16 celle in serie).
- **Corrente di carica:** programmabile fino a 60A, configurata a 50A per rispettare le specifiche della batteria.
- **Potenza massima:** circa 3 kW.
- **Profilo di carica:** CC-CV (Constant Current - Constant Voltage) ottimizzato per LiFePO₄, con fase di bilanciamento delle celle.

- **Protezioni integrate:**
 - protezione da sovratensione e sovracorrente;
 - protezione termica con ventilazione forzata;
 - protezione da inversione di polarità;
 - protezione da cortocircuito.
- **Comunicazione:** interfaccia CAN integrata per monitoraggio stato di carica e parametri in tempo reale.
- **Indicatori:** LED di stato per visualizzazione fasi di carica (carica in corso, bilanciamento, carica completata).
- **Efficienza:** superiore al 90% a pieno carico.
- **Grado di protezione:** IP20 (per uso in ambiente interno protetto).

Il caricabatterie è dotato di sistema di gestione intelligente che comunica con il BMS (Battery Management System) della batteria tramite bus CAN per ottimizzare il processo di carica, garantendo il bilanciamento delle celle e prevenendo situazioni di sovraccarica o sottocarica che potrebbero danneggiare il pacco batteria. Il tempo di ricarica tipico da 20% a 100% di capacità è di circa 2-3 ore, compatibile con le esigenze operative del veicolo.

3.6.6 Driver per motori elettrici

Per il controllo di basso livello dei motori PMAC integrati nelle motoruote, si è deciso di utilizzare degli inverter compatibili con le specifiche dei motori stessi, ovvero

- tensione di alimentazione: 48V;
- potenza nominale: almeno 500W per i motori di trazione e 300W per i motori di sterzo;
- supporto per protocollo di comunicazione CANOpen.

Dopo una ricerca tra i possibili fornitori¹, si è scelto di accettare la proposta offerta da **MicroPhase**, un'azienda italiana specializzata in azionamenti per motori elettrici. Il modello da loro selezionato è il *Mini-Traction Power* che soddisfa tutti i requisiti richiesti. Difatti, tali driver **implementano completamente** i protocolli CiA 301 e 402 per la definizione degli oggetti e della macchina a stati finiti per il controllo dei motori.

Inoltre, l'azienda ha offerto supporto tecnico per

- configurazione dei driver per l'uso specifico con i motori CFR e gli encoder *SIN/COS* da loro integrati;
- taratura dei PID per il controllo di basso livello delle fasi di corrente dei motori.

¹Tale ricerca è stata necessaria in quanto CFR **non produce** inverter.



Figura 3.7: Inverter Mini-Traction Power di MicroPhase, utilizzato per il controllo dei motori PMAC delle motoruote.

Tabella 3.5: Specifiche tecniche dell'inverter Mini-Traction Power (Modello 48V).

Parametro	Valore	Unità
Tensione nominale	48	V
Tensione minima	20	V
Tensione massima ammessa	80	V
Corrente di boost (5s)	120 @ 5s	A
Corrente di picco (2 min)	100 @ 2min	A
Corrente nominale	75	A
Potenza nominale di uscita	3.6	kW
Frequenza PWM	20	kHz
Temperatura di esercizio	0 / +40	°C
Temperatura di stoccaggio	-10 / +70	°C
Uscita ausiliaria 5V	5 (max 200 mA)	V
Banda di corrente	2	kHz
Banda di velocità	150	Hz
Induttanza minima motore	200	μ H
Peso	0.5	kg
Grado di contaminazione	2° o superiore	-
Altitudine	0-1000m / 1000-2000m	m
Infiammabilità 94V-0	Conforme	-

3.6.7 Computer di bordo

Per il computer di bordo, si è scelto il *LattePanda Sigma*, un SBC basato su architettura x86 che offre prestazioni eccezionali in un formato compatto. Questo dispositivo è equipaggiato con un processore Intel® Core™ i5-1340P di 13ª generazione (Raptor Lake), caratterizzato da 12 core (4 Performance-core + 8 Efficient-core) e 16 thread, con frequenza turbo fino a 4.6 GHz per i core ad alte prestazioni e 3.4 GHz per quelli efficienti.



Figura 3.8: LattePanda Sigma Single Board Computer con processore Intel Core i5-1340P.

La scelta di questo SBC è stata motivata da diversi fattori chiave.

- **Architettura x86:** garantisce compatibilità nativa con ROS 2 e una vasta gamma di software e librerie di robotica, semplificando lo sviluppo e il debug.
- **Potenza di calcolo:** il processore i5-1340P offre prestazioni elevate per eseguire algoritmi di navigazione, localizzazione e controllo in tempo reale, con una potenza base di 28W (configurabile fino a 45W).
- **Memoria veloce:** dotato di 32GB di RAM LPDDR5-6400 dual-channel, con una banda massima di 102.4 GB/s, permette l'elaborazione simultanea di dati provenienti dai sensori e l'esecuzione di algoritmi complessi.
- **GPU integrata:** Intel® Iris® Xe Graphics con 80 unità di esecuzione, utile per eventuali elaborazioni grafiche o visualizzazioni.
- **Connettività industriale:** include interfacce essenziali per applicazioni robotiche.
 - 2 porte Ethernet 2.5GbE (Intel® i225-V) per comunicazioni di rete affidabili.
 - Porte USB (2x USB2.0, 2x USB3.2 Gen2, 2x Thunderbolt™ 4) per collegare adattatori CAN-USB e altri dispositivi.

- Porta COM con supporto nativo per protocolli industriali RS232/RS485.
- Header GPIO a 34 pin con co-processore ATmega32U4 per interfacciamento I/O.
- **Espandibilità:** slot M.2 per storage NVMe ad alte prestazioni e moduli wireless, permettendo l'installazione di SSD per logging dei dati e moduli WiFi/4G per comunicazioni remote.
- **Supporto multi-OS:** compatibile con Windows, Ubuntu e altre distribuzioni Linux, Proxmox VE, permettendo la scelta del sistema operativo più adatto. Per il progetto è stato scelto Ubuntu 24.04 LTS con ROS 2 Jazzy Jalisco.
- **Formato compatto:** form factor 3.5" (146 × 102 mm), facilmente integrabile nel telaio del robot.
- **Gestione energetica:** alimentazione flessibile tramite DC Jack (12-20V) o USB Type-C PD (20V), con funzioni BIOS avanzate tra cui *Auto Power-On* e *Watchdog Timer*, essenziali per applicazioni autonome.

Tabella 3.6: Specifiche tecniche principali del LattePanda Sigma.

Parametro	Valore	Unità
Processore	Intel Core i5-1340P	-
Core / Thread	12C (4P+8E) / 16T	-
Frequenza massima	4.6 (P-core), 3.4 (E-core)	GHz
Cache L2	12	MB
Potenza base (TDP)	28 (default: 45W)	W
GPU	Intel Iris Xe Graphics (80 EU)	-
Frequenza GPU	1.45	GHz
RAM	32 GB LPDDR5-6400 Dual-Channel	GB
Banda passante memoria	102.4	GB/s
Ethernet	2 × 2.5GbE (Intel i225-V)	-
USB	2×USB2.0, 2×USB3.2, 2×TB4	-
Storage	M.2 NVMe PCIe 4.0 x4	-
Display	HDMI 2.1, 2×USB-C DP, eDP	-
Dimensioni	146 × 102	mm
Temperatura operativa	0 / +45	°C

Questa piattaforma hardware fornisce le risorse computazionali necessarie per eseguire l'intero stack software del robot, includendo la gestione dei sensori, la localizzazione e il controllo dei motori, mantenendo al contempo un profilo energetico compatibile con l'alimentazione a batteria del veicolo.

3.6.8 Adattatore USB-CAN

Per l'integrazione e il controllo dei vari componenti del veicolo tramite bus CAN, si è scelto di utilizzare il convertitore *USB-CAN-A* di Waveshare. Questo dispositivo permette la comunicazione tra il computer di bordo (LattePanda Sigma), gli inverter Mini-Traction Power e la batteria FlashBattery.

Il *USB-CAN-A* è un convertitore USB-Seriale-CAN economico e versatile, basato su una soluzione con chip STM32 che garantisce stabilità e affidabilità della comunicazione. La scelta di questo dispositivo è stata motivata principalmente da:

- **Convenienza economica:** disponibile a basso prezzo rispetto ad alternative industriali più costose, mantenendo comunque funzionalità adeguate per l'applicazione.
- **Facilità di reperibilità:** acquistabile facilmente tramite Amazon e altri marketplace online, riducendo i tempi di approvvigionamento.
- **Compatibilità con i protocolli CAN:** supporta completamente CAN 2.0A (frame standard, ID 11-bit) e CAN 2.0B (frame esteso, ID 29-bit).
- **Velocità adeguata:** range di velocità CAN configurabile da 5 kbps a 1 Mbps, compatibile con le specifiche CANOpen degli altri dispositivi.
- **Supporto software per Linux:** automaticamente riconosciuto come porta seriale virtuale (ttyUSB) tramite driver CH341, con demo in C e Python fornite dal produttore.



Figura 3.9: Convertitore USB-CAN-A di Waveshare con terminale CAN a vite e interruttore per resistenza di terminazione 120Ω.

Seguono le caratteristiche tecniche principali del convertitore.

- **Protocolli CAN:** supporto CAN 2.0A (standard frame) e CAN 2.0B (extended frame).
- **Velocità CAN:** configurabile da 5 kbps a 1 Mbps, con preset per le velocità comuni (1M, 500K, 250K, 125K, ecc.).
- **Baud rate porta seriale:** predefinito a 2 Mbps (compatibile con la maggior parte delle velocità CAN), configurabile da 9600 bps a 2 Mbps.

- **Modalità operative:** normale, loopback (per self-test), silenziosa (solo ricezione), silenziosa-loopback
- **Protezione hardware:** TVS (Transient Voltage Suppressor) integrato per protezione da sovratensioni e picchi transienti sul bus CAN.
- **Resistenza di terminazione:** resistenza 120 Ω commutabile tramite interruttore, essenziale per terminare correttamente il bus CAN.
- **Filtro ID:** possibilità di configurare filtri per ricevere solo messaggi con ID specifici.
- **Buffer interno:** capacità di buffering fino a 20 messaggi per gestire burst di dati.
- **Timestamp:** i messaggi ricevuti includono timestamp per sincronizzazione temporale.

Tabella 3.7: Specifiche tecniche del convertitore USB-CAN-A.

Parametro	Valore	Unità
Modello	USB-CAN-A (Waveshare)	-
Chipset	STM32	-
Protocolli CAN	CAN 2.0A, CAN 2.0B	-
Velocità CAN	5 - 1000	kbps
Baud rate seriale predefinito	2000	kbps
Baud rate seriale configurabile	9.6 - 2000	kbps
Driver USB	CH341	-
Interfaccia CAN	Terminale a vite (H, L, GND)	-
Resistenza terminazione	120 (commutabile)	Ω
Protezione TVS	Integrata	-
Buffer messaggi	20	msg
Grado di protezione	IP20	-
Temperatura operativa	-10 / +60	°C
Alimentazione	5 (via USB)	V
Dimensioni	70 x 35 x 12	mm
Peso	25	g
Sistema operativo supportato	Windows, Linux	-

Il dispositivo è supportato da un tool di configurazione gratuito per Windows (USB-CAN-A TOOL) che permette di

- configurare la velocità del bus CAN e il baud rate della porta seriale;
- selezionare la modalità operativa (normale, loopback, silent);
- impostare filtri sul CAN ID per la ricezione selettiva di messaggi;
- inviare e ricevere messaggi CAN in tempo reale con visualizzazione timestamp;
- salvare log dei messaggi in formato TXT o Excel per analisi offline;
- configurare risposte automatiche a messaggi con ID specifici.

Il convertitore USB-CAN-A implementa una conversione USB → Seriale → CAN, a differenza di soluzioni più avanzate che utilizzano direttamente USB → CAN (come USB-CAN-B dello stesso produttore).

Nota: Questo comporta una latenza maggiore nella trasmissione dei messaggi, ma per l'applicazione in questione dovrebbe essere trascurabile contando che la lunghezza del bus CAN è limitata a pochi metri all'interno del veicolo. Studi futuri verranno condotti per quantificare questa latenza e valutarne l'impatto sulle prestazioni complessive del sistema di controllo.

Il problema di questa conversione si traduce a livello software con l'incompatibilità con librerie CANOpen più diffuse come *SocketCAN* su Linux, che richiedono un'interfaccia CAN nativa o un driver specifico che si occupi di mostrare il bus come una interfaccia di rete virtuale. Difatti, l'adattatore usa un protocollo proprietario per l'invio e la ricezione dei messaggi tramite la porta seriale. Il funzionamento di tale protocollo è stato analizzato tramite reverse engineering delle demo fornite dal produttore, permettendo così di sviluppare una libreria in C++. Nel Capitolo 5 verrà descritto in dettaglio il funzionamento di questa libreria e il suo utilizzo all'interno del software di controllo del veicolo.

3.6.9 Convertitori DC-DC

Per l'alimentazione del computer di bordo e dei sensori è stato necessario integrare convertitori DC-DC che adattino la tensione di batteria (48V nominali) alle tensioni richieste dai vari componenti. La scelta è ricaduta su convertitori switching della serie *SD* di Meanwell, un produttore affermato nel settore dell'elettronica di potenza industriale, noti per affidabilità ed efficienza.

Sono stati selezionati due modelli.

- **SD-25B-12:** convertitore DC-DC da 48V a 12V con potenza nominale di 25W. Questo modulo è utilizzato per l'alimentazione del sensore LiDAR Livox Mid-360, che richiede una tensione di alimentazione tra 9V e 27V. Il convertitore fornisce una tensione stabile di 12V con corrente massima di 2.1A, ampiamente sufficiente per il consumo medio del LiDAR di 6.5W. Il dispositivo è dotato di protezione da sovratensione, sovracorrente e cortocircuito, garantendo affidabilità operativa.
- **SD-100D-12:** convertitore DC-DC da 48V a 12V con potenza nominale di 102W. Questo modulo alimenta il computer di bordo LattePanda Sigma tramite l'ingresso DC Jack. Con una corrente massima di 8.5A a 12V, il convertitore è dimensionato per gestire il consumo del processore i5-1340P che, in configurazione TDP a 28W (estendibile fino a 45W), richiede una potenza significativa durante le fasi di calcolo intensivo. Il margine di potenza disponibile permette di alimentare anche eventuali periferiche USB collegate al computer.

Entrambi i convertitori operano con un'ampia gamma di tensioni di ingresso (18-75V DC) compatibile con le variazioni di tensione della batteria LiFePO4 durante i cicli di carica e scarica (44V-59.2V). Presentano inoltre un'efficienza tipica superiore al 75%, limitando le perdite energetiche e il surriscaldamento.

Il grado di protezione e le certificazioni di sicurezza (IEC/EN 62368-1) li rendono adatti all'impiego in ambiente industriale.

3.7 Sensoristica

Viene ora analizzato l'insieme della sensoristica a disposizione e/o scelta per il veicolo.

3.7.1 LiDAR

Il sensore LiDAR scelto per il progetto è il *Livox Mid-360*, un dispositivo di scansione 3D prodotto da Livox Technology che offre un eccellente compromesso tra prestazioni, costo e facilità di integrazione. Questo sensore utilizza un pattern di scansione non ripetitivo proprietario di Livox, che garantisce una copertura più uniforme dell'ambiente rispetto ai LiDAR tradizionali a scansione meccanica.

La scelta di questo sensore è stata motivata da diverse caratteristiche chiave, quali le seguenti.

- **Campo visivo esteso:** 360° in orizzontale e 59° in verticale (−7° a +52°), permettendo una copertura quasi completa dell'ambiente circostante il veicolo, essenziale per la navigazione autonoma e il rilevamento di ostacoli.
- **Portata adeguata:** fino a 70 m per superfici con riflettività dell'80%, ampiamente sufficiente per l'ambiente industriale interno ed esterno dello stabilimento.
- **Alta frequenza di acquisizione:** 200,000 punti/secondo con frame rate di 10 Hz, fornendo una densità di punti adeguata per algoritmi di SLAM e mappatura in tempo reale.
- **IMU integrata:** dotato di IMU a 6 assi (modello ICM40609) che fornisce dati di accelerazione e velocità angolare, fondamentali per la fusione sensoriale con gli algoritmi di localizzazione come FAST-LIO2.
- **Sincronizzazione precisa:** supporto per IEEE 1588-2008 (PTPv2) e GPS per la sincronizzazione temporale, utile per la fusione multi-sensoriale.
- **Interfaccia Ethernet:** comunicazione tramite Ethernet 100BASE-TX, facilmente integrabile con il computer di bordo e compatibile con ROS 2.
- **Compattezza e leggerezza:** dimensioni ridotte (65×65×60 mm) e peso contenuto (265 g) facilitano l'integrazione meccanica sul veicolo.
- **Robustezza:** grado di protezione IP67, adatto per operazioni in ambiente industriale e all'aperto.
- **Basso consumo:** potenza media di 6.5 W, compatibile con l'alimentazione a batteria del veicolo.



Figura 3.10: Sensore LIDAR Livox Mid-360.

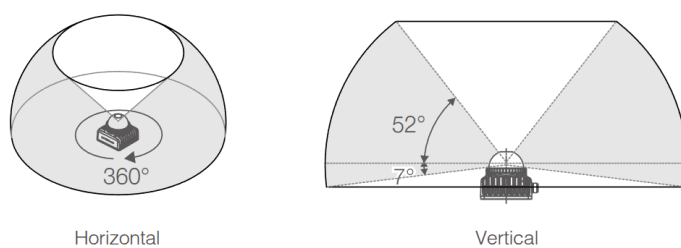


Figura 3.11: Campo visivo del LiDAR Livox Mid-360: 360° orizzontale e 59° verticale (-7° a $+52^\circ$).

Tabella 3.8: Specifiche tecniche del LiDAR Livox Mid-360.

Parametro	Valore	Unità
Modello	Mid-360	-
Lunghezza d'onda laser	905	nm
Classe di sicurezza laser	Class 1 (IEC60825-1:2014)	-
Portata di rilevamento (@ 100 klx)	40 @ 10% rifl., 70 @ 80% rifl.	m
Zona cieca proximale	0.1	m
Campo visivo orizzontale	360	°
Campo visivo verticale	-7 a +52 (59° totale)	°
Precisione della distanza (1σ @ 10m)	≤ 2	cm
Precisione della distanza (1σ @ 0.2m)	≤ 3	cm
Precisione angolare (1σ)	< 0.15	°
Frequenza punti	200,000 (first return)	pts/s
Frame rate	10 (tipico)	Hz
Porta dati	Ethernet 100BASE-TX	-
Sincronizzazione	IEEE 1588-2008, GPS	-
Tasso falsi allarmi (@ 100 klx)	< 0.01	%
IMU integrata	ICM40609 (6 assi)	-
Temperatura operativa	-20 / +55	°C
Grado di protezione	IP67	-
Potenza media	6.5	W
Tensione di alimentazione	9 - 27	V DC
Dimensioni	65 × 65 × 60	mm
Peso	265	g

Supporto software

Il Livox Mid-360 dispone di un ecosistema software completo sviluppato e mantenuto da Livox Technology, che facilita l'integrazione del sensore in applicazioni robotiche.

Livox SDK2: è la libreria di base scritta in C/C++ che implementa il protocollo di comunicazione proprietario di Livox. L'SDK fornisce

- API C-style per il controllo del LiDAR e la ricezione dei dati;
- supporto multi-piattaforma (Linux e Windows);
- gestione di configurazioni multi-sensore e multi-host;
- sincronizzazione temporale tramite IEEE 1588 (PTP) e/o GPS;
- logging dei dati del firmware per diagnostica;
- modalità master/slave per scenari multi-casting.

Livox ROS Driver 2: è il driver ufficiale per l'integrazione con ROS e ROS 2, sviluppato a partire dall'SDK. Supporta

- ROS Melodic e Noetic (ROS 1)
- ROS 2 Foxy e Humble. Jazzy non è ancora ufficialmente supportato, ma il driver per Humble funziona correttamente anche su questa versione.
- Pubblicazione dati in formati multipli:

- pointCloud2 standard (sensor_msgs/PointCloud2);
- formato personalizzato Livox con timestamp per punto;
- formato PCL (pcl::PointXYZI).
- Configurazione tramite file JSON per parametri di rete, modalità di scansione e calibrazione estrinseca.
- Pubblicazione separata di dati IMU integrati.
- Supporto per topic multipli (un topic per LiDAR o topic condiviso).
- Frequenza di pubblicazione configurabile (fino a 100 Hz).

Per il progetto è stata utilizzata la versione ROS 2 Humble del driver, che permette l'integrazione diretta con l'architettura software del robot. Il sensore è inoltre supportato nativamente da FAST-LIO2, l'algoritmo di SLAM scelto per il progetto, garantendo prestazioni ottimali per la localizzazione e la mappatura in tempo reale. La compatibilità nativa con FAST-LIO2 deriva dal fatto che il formato dati personalizzato di Livox, che fornisce un timestamp per ogni singolo punto anziché per frame, è particolarmente adatto agli algoritmi di SLAM basati su odometria LiDAR-inerziale.

Nota: Il cono cieco del sensore implica che ci sono dei limiti nella copertura verticale che il sensore può offrire. Questo implica che è importante il punto di montaggio del sensore sul veicolo per massimizzare l'area coperta e minimizzare le zone non visibili in funzione delle operazioni previste.

Nel progetto, il sensore è stato montato frontalmente e rivolto verso il basso, in modo da coprire efficacemente l'area immediatamente davanti e circostante il veicolo.

3.7.2 Encoder

Gli encoder per il controllo della velocità e della posizione angolare sono componenti integrati di fabbrica nelle motoruote MRT05 fornite da CFR. Si tratta di encoder magnetici angolari di tipo SIN/COS modello **FTS1** prodotti da COBO s.p.a., progettati specificamente per applicazioni "through-shaft" (albero passante).

Questa tecnologia utilizza un anello magnetico permanente polarizzato diametralmente montato sull'albero rotante del motore. Il sensore FTS1 rileva la distribuzione del campo magnetico attraverso una combinazione brevettata di sensori Hall allo stato solido e algoritmi di elaborazione digitale del segnale. Le uscite sinusoidali (SIN) e cosinusoidali (COS) variano in funzione della posizione angolare del magnete, fornendo informazioni precise sulla rotazione.

Le sue caratteristiche tecniche principali sono a seguire.

- **Tecnologia contactless:** assenza di usura meccanica e manutenzione, ideale per ambienti industriali severi
- **Uscite multiple programmabili:**

- segnali analogici SIN/COS con ampiezze configurabili (1.0, 1.1, 2.2 o 4 Vpp);
 - uscita analogica 0.5-4.5 Vdc per posizione assoluta;
 - encoder incrementale ABZ con 1024 conteggi per giro;
 - uscite UVW per motori brushless;
 - interfaccia SPI per dati digitali a 12 bit.
- **Risoluzione:** 10-bit assoluta, 12-bit tramite SPI, interpolabile fino a 1024 PPR in modalità incrementale.
- **Latenza ridotta:** tipicamente 12 μ s (high bandwidth) o 75 μ s (low bandwidth).
- **Non linearità integrale:** $\pm 0.6^\circ$ con magneti ideali e calibrazione.
- **Velocità rotazionale:** fino a 25000 rpm (prestazioni ottimali fino a 10000 rpm).
- **Funzioni avanzate:**
 - auto-calibrazione per compensazione offset e gain;
 - funzione ZERO programmabile;
 - emulazione fino a 16 poli per uscite SIN/COS.
- **Robustezza:** resistenza a vibrazioni (20g, 10-2000 Hz) e shock (50g, 11ms), grado di protezione IP67.
- **Range di temperatura:** -40°C a +125°C (uscite SIN/COS, SPI, ABZ, UVW @ 5V).
- **Alimentazione flessibile:** 5 Vdc o 9-36 Vdc.

Nel sistema del robot, gli encoder FTS1 sono collegati direttamente ai driver Mini-Traction Power, ai quali forniscono feedback di posizione angolare e velocità tramite i segnali SIN/COS analogici. I driver eseguono l'interpolazione dei segnali per implementare il controllo in anello chiuso della posizione e della velocità dei motori, garantendo precisione e reattività nella risposta ai comandi. I dati di encoder sono inoltre accessibili dal computer di bordo tramite il bus CAN per il calcolo dell'odometria delle ruote, contribuendo alla stima della posizione del veicolo insieme ai dati del LiDAR e dell'IMU.

3.7.3 Sonda di temperatura

Le sonde di temperatura integrate nelle motoruote MRT05 sono sensori resistivi (RTD) modello **STS1** prodotti da Microtherm GmbH, forniti di serie da CFR per il monitoraggio termico dei motori. Si tratta di sensori con caratteristiche simili al classico KTY84 ma con prestazioni migliorate.

Il principio di funzionamento si basa sulla variazione della resistenza elettrica di un elemento sensibile in funzione della temperatura. Il sensore fornisce una curva resistenza-temperatura ben definita e ripetibile, permettendo una misurazione precisa attraverso la lettura della resistenza con un circuito di condizionamento appropriato.

Di seguito le sue caratteristiche tecniche principali.

Tabella 3.9: Specifiche tecniche dell'encoder magnetico FTS1.

Parametro	Valore	Unità
Modello	FTS1 (COBO Spa)	-
Tipo	Magnetico angolare through-shaft	-
Tecnologia sensori	Hall allo stato solido	-
Risoluzione assoluta	10 bit	-
Risoluzione SPI	12 bit	-
Risoluzione incrementale	1024	PPR
Uscite SIN/COS	1.0, 1.1, 2.2, 4 (prog.)	Vpp
Uscita analogica	0.5 - 4.5	Vdc
Non linearità integrale	± 0.6 (calibrato)	°
Latenza (high bandwidth)	12	μs
Latenza (low bandwidth)	75	μs
Velocità massima	25000	rpm
Velocità ottimale	< 10000	rpm
Tensione alimentazione	5 o 9-36	Vdc
Temperatura operativa	-40 / +125	°C
Resistenza vibrazioni	20g (10-2000 Hz)	-
Resistenza shock	50g (11 ms)	-
Grado di protezione	IP67	-
Materiale involucro	PBT	-

- **Range di temperatura esteso:** da -40°C a +170°C (fino a +190°C con possibile perdita di linearità).
- **Resistenza nominale:** 1000 Ω a 100°C ($\pm 3\%$).
- **Eccellente stabilità termica a lungo termine:** garantisce misurazioni affidabili nel tempo.
- **Alta precisione:** curve resistenza-temperatura con tolleranze ben definite.
- **Nessuna polarità:** non richiede attenzione alla connessione (+/-).
- **Isolamento elevato:** resistenza di isolamento minima di 100 M Ω a 100 Vdc.
- **Basso assorbimento:** corrente nominale 5 mA, massima 8 mA.
- **Potenza dissipata ridotta:** massimo 50 mW.
- **Robustezza meccanica:** disponibile con involucro in PPS o guaina termorestringente in Kynar®.
- **Compatibilità RoHS:** conformità alle normative ambientali.

Nel sistema del robot, le sonde STS1 sono collegate ai driver Mini-Traction Power, che monitorano costantemente la temperatura dei motori per prevenire danni da surriscaldamento. I driver implementano protezioni termiche automatiche, riducendo la potenza erogata o arrestando il motore qualora la temperatura superi soglie critiche. I dati di temperatura sono accessibili anche dal computer di bordo tramite il bus CAN, permettendo la supervisione termica in tempo reale dell'intero sistema e l'implementazione di strategie di

Tabella 3.10: Specifiche tecniche della sonda di temperatura STS1.

Parametro	Valore	Unità
Modello	STS1 (Microtherm)	-
Tipo	RTD (resistivo)	-
Resistenza tipica @ 100°C	1000	Ω
Tolleranza resistenza	± 3	%
Range temperatura operativa	-40 / +170	°C
Range esteso	-40 / +190	°C
Resistenza @ -40°C (tip.)	359	Ω
Resistenza @ 0°C (tip.)	498	Ω
Resistenza @ +25°C (tip.)	626	Ω
Resistenza @ +100°C (tip.)	1000	Ω
Resistenza @ +170°C (tip.)	1482	Ω
Resistenza isolamento (min.)	100 @ 100 Vdc	MΩ
Corrente nominale	5	mA
Corrente massima	8	mA
Potenza dissipata massima	50	mW
Materiale involucro	PPS / Kynar®	-
Tipo cavi	ETFE, AWG24	-
Conformità	RoHS	-

gestione termica a livello di veicolo (ad esempio, limitazione della velocità massima in condizioni di alta temperatura ambiente).

3.8 Telaio e struttura meccanica

Il telaio del veicolo è stato progettato tramite software CAD (Computer-Aided Design) SolidWorks, tenendo in considerazione i requisiti di robustezza, leggerezza e facilità di assemblaggio. Dopo diverse iterazioni di design, si è deciso di adottare una struttura composta da profili in acciaio (30 × 30 × 2.6 mm) saldati tra loro, con travi di rinforzo (20 × 20 × 2 mm) per garantire rigidità e resistenza alle sollecitazioni meccaniche causate dall'alto carico da trasportare.

La dimensione finale del telaio è risultata essere quindi di 1.8 × 1m, con delle piastre in acciaio da 5mm agli angoli per il montaggio delle ruote. Ulteriori parti in lamiera sono state studiate per l'alloggiamento dei quadri elettrici contenenti l'elettronica di controllo e l'alimentazione.

Telaio e lamiere sono stati infine realizzati tramite un fornitore qualificato che si è occupato del taglio dei profilati, della saldatura e di applicare un trattamento di zincatura per prevenire la corrosione.

Nelle figure a seguire sono riportati alcuni disegni tecnici estratti dal progetto CAD, completo di lamierati di supporto, ruote e compressore. In tabella 3.12 sono riportate invece le quote principali del veicolo.

3.8.1 Analisi agli elementi finiti

Per garantire che il telaio del veicolo fosse in grado di sopportare i carichi previsti senza deformazioni eccessive o cedimenti strutturali, è stata condotta

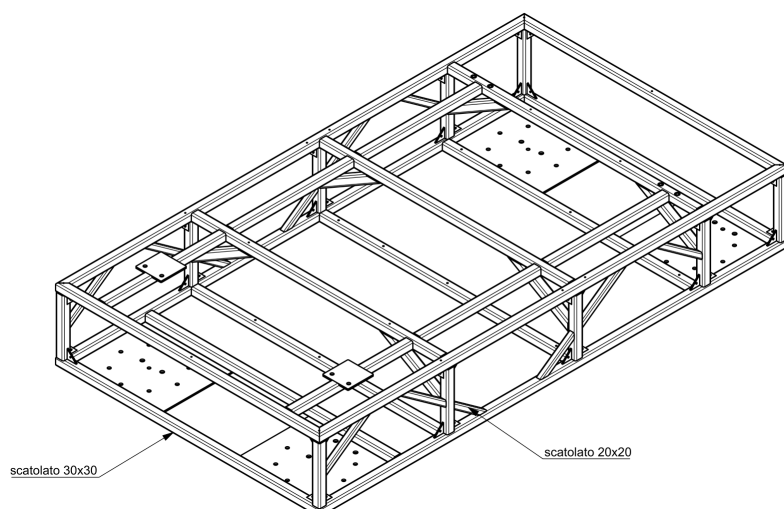


Figura 3.12: Progetto CAD del telaio del veicolo – disegno in vista isometrica.

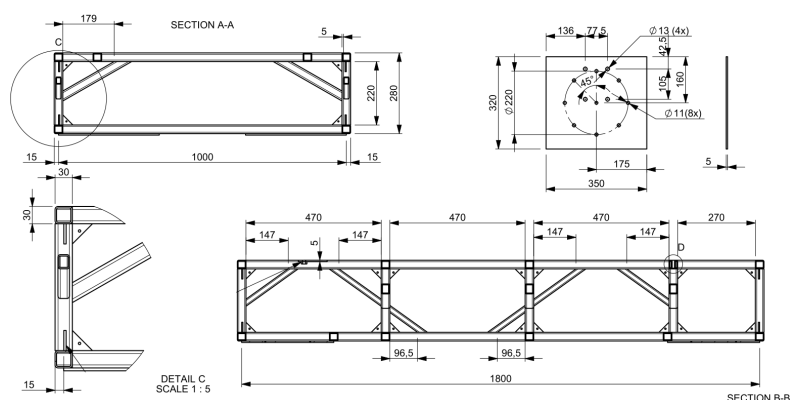


Figura 3.13: Progetto CAD del telaio del veicolo – quote significative.

Tabella 3.11: Distinta componenti principali del telaio per figura 3.15.

Pos.	Descrizione
1	Struttura AGV
2	Pannello posteriore
3	Pannello laterale
4	Pannello centrale batteria
5	Lamiera supporto batteria
6	Pannello anteriore laterale
7	Staffa fissaggio batteria
8	Planale lamiera posteriore
9	Planale interno
10	Planale lamiera anteriore
11	Pannello anteriore
12	Planale interno anteriore
13	Lamiera fissaggio lidar
14	Lamiera componentistica

Tabella 3.12: Dimensioni principali del telaio da figura 3.13.

Parametro	Valore	Unità
Lunghezza totale	1800	mm
Larghezza totale	1000	mm
Altezza totale	280	mm
Altezza interna	220	mm
Sezione profilato	30 x 30	mm
Spessore profilato	2.6	mm
Sezione rinforzi	20 x 20	mm
Spessore rinforzi	2	mm
Interasse sezioni	470	mm
Diametro fori montaggio	11	mm
Diametro fori M13	13	mm
Spessore lamiera	5	mm
Passo fori laterali	147	mm

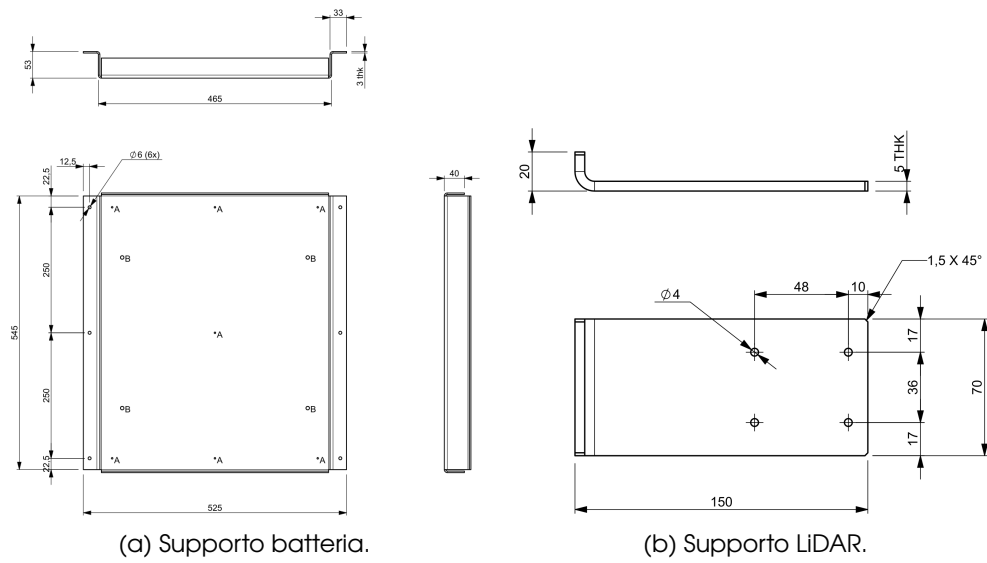


Figura 3.14: Dettagli di piastre di supporto.

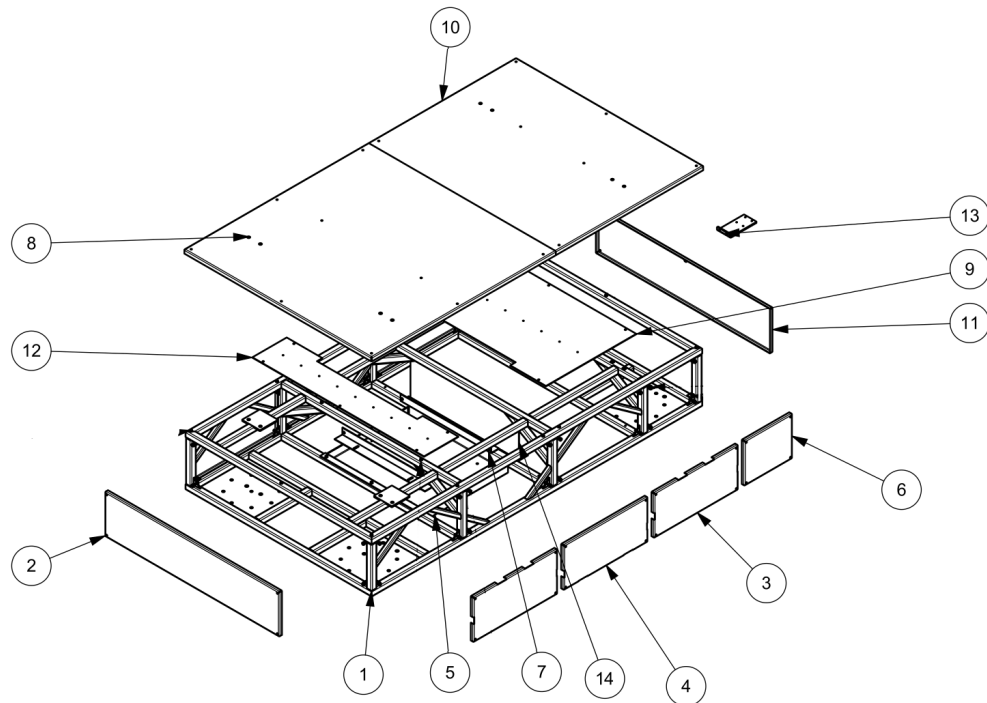


Figura 3.15: Vista esplosa del telaio del veicolo.

un'analisi agli elementi finiti utilizzando FASTRAN, un software specializzato in simulazioni strutturali.

L'analisi è stata condotta sulla penultima versione del telaio, che prevedeva una forma trapezoidale per le travi tubolari frontali per permettere il montaggio del LiDAR, sostituito poi nella versione finale con una trave orizzontale più semplice e una lamiera di supporto. Una volta importato il modello 3D nel software, sono stati definiti tutti i parametri necessari alla caratterizzazione del materiale e le condizioni al contorno (es.: vincoli, distribuzione dei carichi, ecc.), così come sono state configurate le mesh di calcolo per suddividere il modello in elementi finiti.

In figura 3.16 è possibile vedere una schermata del programma di analisi dove è stato caricato il modello ed è stata definita la mesh di calcolo. Si può vedere con delle linee bianche la *struttura a ragno* generata dal software per la distribuzione del carico sui punti di appoggio programmati.

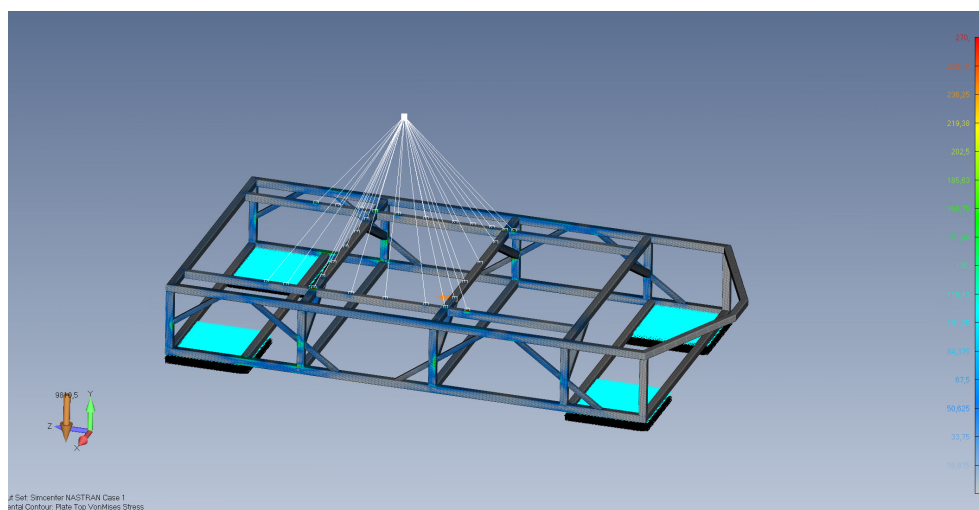


Figura 3.16: Schermata di FASTRAN con il modello FEM del telaio del veicolo con la struttura a ragno.

Sempre da figura 3.16 si può osservare la **heatmap** dei risultati dell'analisi, che mostra la distribuzione delle deformazioni (in mm) del telaio sotto carico. Le aree in rosso indicano le zone di massima deformazione, mentre le aree in blu rappresentano le zone con deformazioni minime. Nelle figura 3.17a e figura 3.17b possiamo vedere dei dettagli più ravvicinati dove si notano zone a tensione maggiore, che però rimangono comunque entro limiti di sicurezza. Si possono notare alcune zone colorate di rosso (massima tensione), localizzate in alcuni punti di giunzione tra travi e piastre di supporto, ma che probabilmente sono dovute a come il software genera le mesh piuttosto che a reali criticità strutturali.

3.9 Analisi dei costi

Di seguito viene riportata una tabella riepilogativa dei costi sostenuti per l'acquisto dei componenti principali del veicolo. Si annoverano i seguenti

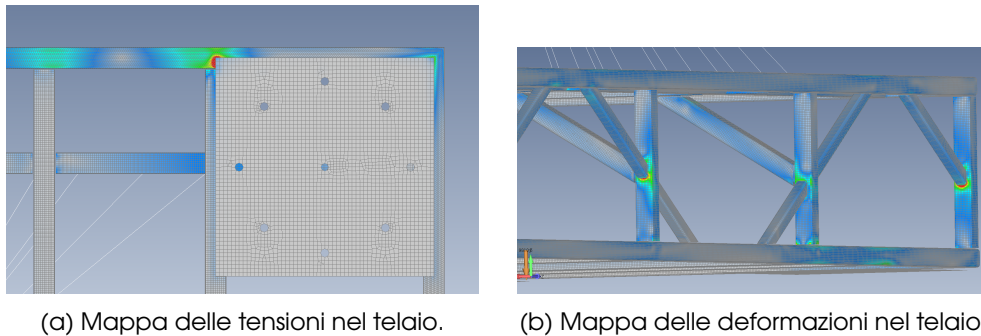


Figura 3.17: Risultati dell'analisi FEM del telaio.

extra, non inclusi nella trattazione dei singoli componenti.

- Costi per accessori e cavi connettori. Es: cover di protezione per SBC, modulo Wi-Fi, antenne, SSD (Solid-State Drive), cavo di collegamento LiDAR e interfaccia USB per configurazione driver.
- Costi di cablaggio e messa in opera del veicolo, inclusi materiali di consumo (connettori, canaline, fascette, ecc.) e manodopera specializzata.
- Costi di ricerca e acquisto delle ruote caster per la stabilizzazione del veicolo, inclusi materiali di montaggio.

I prezzi indicati sono quelli forniti nelle offerte ricevute dai fornitori al momento dell'acquisto, senza IVA. Il costo totale del veicolo ammonta a poco più di

Tabella 3.13: Analisi dei costi dei componenti principali del veicolo.

Componente	Quantità	Costo Unitario (€)	Costo Totale (€)
Motoruote CFR MRT05	2	2.740	5.480
Driver Mini-Traction Power	4	410	1.640
Batteria FlashBattery	1	3.950	3.950
Caricabatterie Zivan NG3	1	355	355
LattePanda Sigma	1	563	563
USB-CAN-A Adapter	1	25	25
DC-DC SD-25C-12	1	25	25
DC-DC SD-100C-12	1	65	65
Livox Mid-360	1	659	659
Telaio, lamierati e caster	-	-	2.356
Cablaggi e messa in opera	-	-	1.800
Extra accessori	-	-	314
Totale			17.232

€17.000, includendo tutti i componenti principali, l'elettronica di controllo, la sensoristica, il telaio e le spese di cablaggio e messa in opera. Questo budget, seppur non trascurabile, risulta contenuto rispetto a soluzioni commerciali di almeno un ordine di grandezza rispetto ad offerte ricevute all'azienda per veicoli con capacità simili.

In figura 3.18 è possibile vedere un render 3D del veicolo completo di tutti i componenti principali, dove sopra vi è posizionato un compressore. In figura 3.19 si può vedere invece una disposizione ideale dei componenti.

Nota: Lo studio di questo prototipo non tiene conto di costi di sviluppo software, test e validazione, né di ridondanze hardware o certificazioni di sicurezza necessarie per un prodotto che, anche se ad uso interno, deve operare in ambienti nei quali sono presenti operatori umani e altre macchine. Questi aspetti verranno discussi nel capitolo conclusivo, dove verranno proposte possibili evoluzioni del progetto per la realizzazione di un prototipo più avanzato e vicino ad un prodotto commerciale.

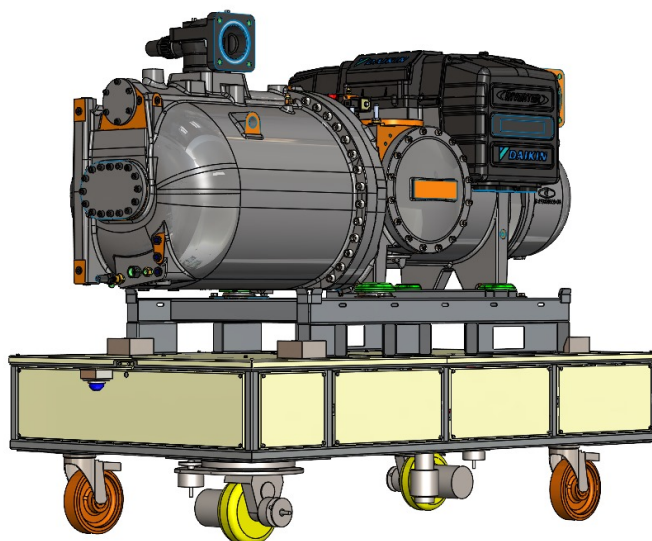


Figura 3.18: Render 3D del veicolo completo di componenti principali.

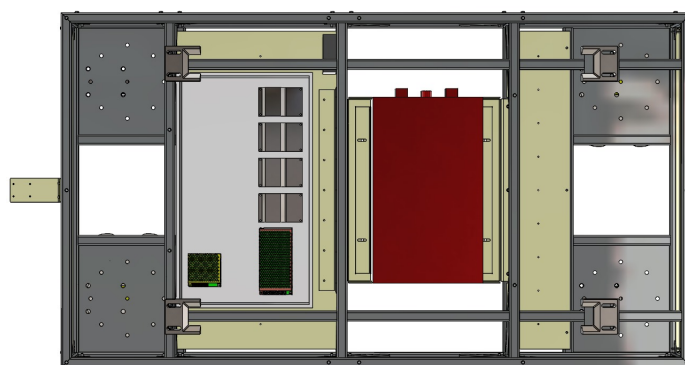


Figura 3.19: Disposizione ideale dei componenti interni del veicolo.

Capitolo 4

Modello cinematico del robot

Come accennato nella sezione 3.5, il veicolo è stato progettato per essere in grado di eseguire sia manovre comuni come avanzamenti in linea retta e curve con orientamento tangenziale all'arco di curva, sia manovre più complesse come rotazioni sul posto e traslazioni laterali. Per raggiungere questo obiettivo, il sistema di movimentazione del robot è stato progettato con due ruote motrici indipendenti e due ruote caster libere di girare attorno al proprio perno verticale, disposte diagonalmente rispetto al centro geometrico del veicolo.

In figura 4.1 è mostrata una rappresentazione schematica del robot con le sole ruote attive, gli angoli di sterzo e le velocità lineari, dove il suo centro geometrico coincide con l'origine degli assi. In figura 4.2 si riporta anche il caso generale in cui il veicolo è ad una certa distanza x_c e y_c dall'origine del sistema di riferimento globale.

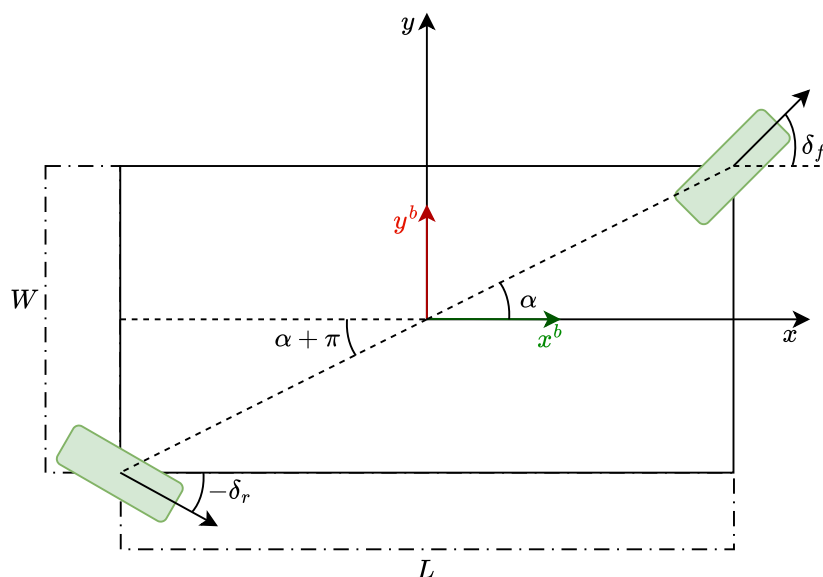


Figura 4.1: Rappresentazione schematica del robot.

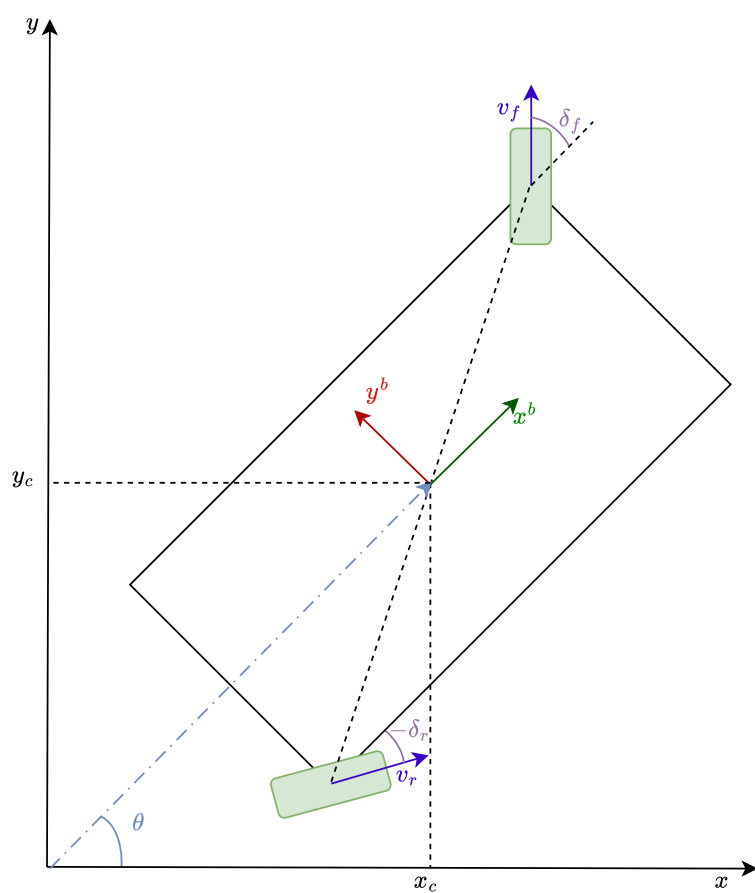


Figura 4.2: Rappresentazione schematica del robot in posizione generica.

4.1 Cinematica diretta

L'obiettivo di questa sezione è quello di derivare le equazioni cinematiche che descrivono il moto del veicolo, inteso come vettore di velocità generalizzate $\dot{q} = [\dot{x}, \dot{y}, \dot{\theta}]^T$, in funzione degli input di controllo, ovvero gli angoli di sterzo δ_f, δ_r e le velocità angolari delle ruote motrici ω_f, ω_r .

Per derivare le equazioni cinematiche del veicolo, facciamo le seguenti ipotesi semplificative.

- Le ruote motrici sono in grado di ruotare attorno al proprio asse verticale per variare l'angolo di sterzo $\delta_i, i \in f, r$.
- Le ruote motrici sono in grado di variare la propria velocità angolare $\omega_i, i \in f, r$ in modo indipendente.
- Le ruote caster sono libere di ruotare attorno al proprio perno verticale senza attrito.
- Le ruote sono perfettamente aderenti al terreno senza slittamenti laterali.
- Il veicolo si muove su un piano orizzontale senza pendenze.
- Le dimensioni del veicolo sono note e costanti pari ad L e W , con le ruote motrici montate nei punti $(L/2, W/2)$ e $(-L/2, -W/2)$ rispetto al centro geometrico. Conseguentemente, lo sfasamento angolare delle ruote rispetto l'asse longitudinale è dato da

$$\alpha = \text{atan2}(W/2, L/2). \quad (4.1)$$

- Il centro di massa del veicolo coincide con il centro geometrico.
- Le forze esterne agenti sul veicolo (es. vento, urti, ecc.) sono trascurabili.
- Le accelerazioni del veicolo sono moderate, permettendo di trascurare effetti dinamici complessi come l'inerzia delle ruote e le forze di Coriolis.
- Il veicolo è simmetrico rispetto agli assi longitudinale e trasversale.
- Le ruote motrici sono identiche tra loro, con lo stesso raggio r e caratteristiche di attrito.
- Il sistema di riferimento *globale* è inerziale e fisso, mentre il sistema di riferimento *locale* al robot si muove con esso.
- Si trascurano gli effetti di deformazione delle ruote e del terreno.
- Si trascurano differenze legate alle geometrie delle ruote motrici che ne vincolano il montaggio nella realtà e per questo si assume che il verso di rotazione positivo delle ruote motrici sia sempre quello che produce una velocità positiva lungo l'asse longitudinale del veicolo.

Tali ipotesi sono comuni nella letteratura sui robot mobili e permettono di semplificare notevolmente l'analisi del modello cinematico. In particolare, per un robot dotato di quattro ruote indipendenti, tipicamente si considerano le ruote sullo stesso asse (quindi le due anteriori e le due posteriori) come un'unica ruota virtuale per asse, ognuna con un proprio angolo di sterzo e

velocità [19]. Questo approccio riduce il numero di variabili da considerare e semplifica le equazioni cinematiche del veicolo.

Tuttavia, quando questa semplificazione viene applicata, è importante tenere conto delle possibili differenze di comportamento tra le ruote sullo stesso asse, specialmente durante manovre di traslazione laterale (anche dette *crab steering*) o delle rotazioni sul posto. In questi casi, i modelli spesso diventano singolari e richiedono di essere adattati per gestire correttamente tali situazioni.

Per questo motivo, sebbene lo studio iniziale del problema sia stato condotto sul modello descritto in [19], si è deciso di verificare la validità di un modello più generico che invece è ottenuto tramite la trattazione classica delle equazioni di vincolo per i robot mobili [6].

4.1.1 Modello a biciclo sterzante

Al modello a biciclo descritto in [19] può essere aggiunta la seguente ipotesi.

- Le ruote motrici sono approssimabili come se fossero montate lungo l'asse longitudinale del veicolo, ovvero, la motoruota anteriore è montata in $(L/2, 0)$ e la motoruota posteriore in $(-L/2, 0)$.

Le equazioni cinematiche del veicolo sono date da:

$$\begin{cases} \dot{x} = v \cos(\theta + \beta), \\ \dot{y} = v \sin(\theta + \beta), \\ \dot{\theta} = \frac{v \cos(\beta)}{l_f + l_r} (\tan(\delta_f) - \tan(\delta_r)), \end{cases} \quad (4.2)$$

dove

$$v = \frac{v_f \cos(\delta_f) + v_r \cos(\delta_r)}{2 \cos(\beta)}, \quad (4.3)$$

$$\beta = \tan^{-1} \left(\frac{l_f \tan(\delta_r) + l_r \tan(\delta_f)}{l_f + l_r} \right), \quad (4.4)$$

rappresentano rispettivamente velocità del centro del robot e angolo di *side-slip*, ovvero l'angolo tra la direzione di avanzamento del veicolo descritta dal vettore velocità e l'asse longitudinale del robot.

In queste equazioni, l_f e l_r rappresentano rispettivamente la distanza tra il centro del veicolo e l'asse anteriore e posteriore delle ruote motrici, mentre v_f e v_r sono le velocità lineari delle ruote motrici anteriori e posteriori, per cui chiaramente valgono

$$v_f = r\omega_f, \quad v_r = r\omega_r \quad (4.5)$$

dove r è il raggio delle ruote motrici e ω_f , ω_r sono le velocità angolari delle ruote motrici anteriori e posteriori.

Si può vedere poi che sostituendo le equazioni (4.3) and (4.4) nella equazione (4.2), si può ottenere un sistema matriciale del tipo $\dot{q} = J(q, u)u$, per mettere in relazione le velocità generalizzate del robot ($\dot{q}$) con le velocità angolari delle ruote motrici e gli angoli di sterzo (u). In particolare, scrivendo $J(q, u)$ come

$$J(q, u) = R(\theta)\hat{J}(u) \quad (4.6)$$

dove $R(\theta)$ è la matrice di rotazione tra il sistema di riferimento globale e quello locale al robot descritta da

$$R(\theta) = \begin{bmatrix} \cos(\theta) & -\sin(\theta) & 0 \\ \sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} \quad (4.7)$$

e $\hat{J}(u)$ è la matrice jacobiana del sistema nel sistema di riferimento locale al robot, data da

$$\hat{J}(u) = \frac{r}{2} \begin{bmatrix} \cos(\delta_f) & \cos(\delta_r) \\ \cos(\delta_f)\tan(\beta) & \cos(\delta_r)\tan(\beta) \\ \frac{\cos(\delta_f)}{l}(\Delta_{tan}) & \frac{\cos(\delta_r)}{l}(\Delta_{tan}) \end{bmatrix}, \quad (4.8)$$

dove

$$l = l_f + l_r, \quad (4.9)$$

$$\Delta_{tan} = \tan(\delta_f) - \tan(\delta_r), \quad (4.10)$$

si ottiene una rappresentazione compatta del modello cinematico, che può essere utile per l'implementazione del controllo.

$$\begin{bmatrix} \dot{x} \\ \dot{y} \\ \dot{\theta} \end{bmatrix} = \frac{r}{2} \begin{bmatrix} \bar{J}_1 \cos(\delta_f) & \bar{J}_1 \cos(\delta_r) \\ \bar{J}_2 \cos(\delta_f) & \bar{J}_2 \cos(\delta_r) \\ \frac{1}{l} \Delta_{tan} \cos(\delta_f) & \frac{1}{l} \Delta_{tan} \cos(\delta_r) \end{bmatrix} \begin{bmatrix} \omega_f \\ \omega_r \end{bmatrix}, \quad (4.11)$$

con

$$\bar{J}_1 = \cos(\theta) - \sin(\theta)(\Sigma_{tan}), \quad (4.12)$$

$$\bar{J}_2 = \sin(\theta) + \cos(\theta)(\Sigma_{tan}), \quad (4.13)$$

$$\Sigma_{tan} = \frac{l_r \tan(\delta_f) + l_f \tan(\delta_r)}{l}. \quad (4.14)$$

Osservando l'equazione (4.4), si nota che l'angolo di side-slip β si comporta come una media pesata delle tangenti dell'angolo di sterzo. Questo implica che, se uno dei due angoli di sterzo è pari a $\pi/2$, l'angolo di side-slip è infinito e quindi tutto il modello perde di significato.

Analogamente, si può osservare che, per ottenere una rotazione sul posto, sarebbe necessario imporre che le velocità lungo gli assi x e y siano nulle, ma se ciò fosse vero per angoli θ e β tali per cui la loro somma non è un multiplo di $\pi/2$, allora la velocità v dovrebbe essere nulla, il che implicherebbe che entrambe le velocità delle ruote motrici siano nulle, rendendo impossibile la rotazione.

Nota: Questo comportamento evidenzia una limitazione del modello a bicicletta, che non è in grado di gestire correttamente parte delle manovre richieste, come le traslazioni laterali e le rotazioni sul posto.

4.1.2 Modello a doppia ruota sterzante

Il modello proposto di seguito, derivato tramite l'analisi dei vincoli non olonimi del veicolo, è in grado di gestire correttamente tutte le manovre desiderate senza incorrere in singolarità, a patto di risolvere ai minimi quadrati un sistema di equazioni sovradeterminato.

Si consideri quindi lo schema di sistemi di riferimento descritto da figura 4.1, dove per semplicità non sono riportate le ruote caster e viene considerato il robot orientato in modo concorde con l'asse x del sistema di riferimento globale.

Nota: Si può osservare che i due sistemi di riferimento possono essere collegati tramite la matrice di rotazione $R(\theta)$ definita in equazione (4.7).

Oltre alle variabili già definite nel modello a biciclo sterzante, si definiscono

- x_c, y_c le coordinate del centro del veicolo nel sistema di riferimento globale;
- θ l'angolo di orientamento del veicolo rispetto l'asse x del sistema di riferimento globale;
- v_c la velocità lineare del centro del veicolo nel sistema di riferimento globale;
- $\dot{\theta}$ la velocità angolare del veicolo attorno all'asse verticale che passa per il centro del veicolo;
- p_i^b la posizione della ruota motrice i nel sistema di riferimento locale al veicolo, con $i \in \{f, r\}$;
- d la distanza tra il centro del veicolo e il punto di montaggio delle ruote motrici

$$d = \sqrt{\left(\frac{L}{2}\right)^2 + \left(\frac{W}{2}\right)^2}; \quad (4.15)$$

- α l'angolo tra l'asse longitudinale del veicolo e la linea che congiunge il centro del veicolo con il punto di montaggio delle ruote motrici, definito in equazione (4.1).

Le posizioni delle ruote motrici nel sistema di riferimento locale al robot sono quindi date da

$$p_f^b = d \begin{bmatrix} \cos(\alpha) \\ \sin(\alpha) \end{bmatrix} = \begin{bmatrix} \frac{L}{2} \\ \frac{W}{2} \end{bmatrix}, \quad (4.16)$$

$$p_r^b = -d \begin{bmatrix} \cos(\alpha) \\ \sin(\alpha) \end{bmatrix} = \begin{bmatrix} -\frac{L}{2} \\ -\frac{W}{2} \end{bmatrix}. \quad (4.17)$$

Considerando che le ruote motrici non si possono muovere rispetto al loro punto di montaggio, possiamo dire che la velocità di ogni ruota motrice nel sistema locale sarà data da

$$v_i^b = v_c^b + \omega \times p_i^b, \quad i \in \{f, r\}, \quad (4.18)$$

dove $v_c^b = [\dot{x}_c^b, \dot{y}_c^b, 0]^T$ è la velocità lineare del centro del veicolo nel sistema di riferimento locale, e $\omega = [0, 0, \dot{\theta}]^T$ è la velocità angolare del veicolo.

Esplicitando il prodotto vettoriale e considerando che la velocità angolare $\dot{\theta}$ è diretta lungo l'asse verticale

$$\omega \times p_i^b = \dot{\theta} \begin{bmatrix} 0 \\ 0 \\ 1 \end{bmatrix} \times \begin{bmatrix} p_{ix} \\ p_{iy} \\ 0 \end{bmatrix} = \dot{\theta} \begin{bmatrix} -p_{iy} \\ p_{ix} \\ 0 \end{bmatrix}. \quad (4.19)$$

Sostituendo le posizioni delle ruote motrici dalle equazioni (4.16) and (4.17), si ottengono le velocità delle ruote nel sistema di riferimento locale:

$$v_f^b = \begin{bmatrix} \dot{x}_c^b - \dot{\theta} \frac{W}{2} \\ \dot{y}_c^b + \dot{\theta} \frac{L}{2} \\ 0 \end{bmatrix}, \quad (4.20)$$

$$v_r^b = \begin{bmatrix} \dot{x}_c^b + \dot{\theta} \frac{W}{2} \\ \dot{y}_c^b - \dot{\theta} \frac{L}{2} \\ 0 \end{bmatrix}. \quad (4.21)$$

Da questo punto in poi, poiché il moto si svolge nel piano orizzontale, si considerano solo le componenti x e y delle velocità delle ruote motrici.

Vincoli di non slittamento

Il primo vincolo fondamentale per un robot mobile è quello di evitare lo slittamento laterale delle ruote. Questo significa che la componente della velocità della ruota perpendicolare alla direzione di avanzamento deve essere nulla. In altri termini, la velocità della ruota deve essere sempre tangente al piano della ruota stessa.

Per esprimere matematicamente questo vincolo, è necessario individuare il versore perpendicolare all'asse di rotazione della ruota. Dato che l'angolo di sterzo δ_i è definito come l'angolo tra l'asse della ruota e l'asse longitudinale del robot (positivo in senso antiorario), il versore perpendicolare all'asse della ruota nel sistema di riferimento locale è dato da

$$n_i = \begin{bmatrix} -\sin(\delta_i) \\ \cos(\delta_i) \end{bmatrix}, \quad i \in \{f, r\}. \quad (4.22)$$

Il vincolo di non slittamento può quindi essere espresso come

$$n_i^T v_i^b = 0, \quad i \in \{f, r\}. \quad (4.23)$$

Sostituendo le espressioni delle velocità delle ruote dalle equazioni (4.20) and (4.21) e del versore normale dall'equazione (4.22), si ottiene

$$-\sin(\delta_f) \left(\dot{x}_c^b - \dot{\theta} \frac{W}{2} \right) + \cos(\delta_f) \left(\dot{y}_c^b + \dot{\theta} \frac{L}{2} \right) = 0, \quad (4.24)$$

$$-\sin(\delta_r) \left(\dot{x}_c^b + \dot{\theta} \frac{W}{2} \right) + \cos(\delta_r) \left(\dot{y}_c^b - \dot{\theta} \frac{L}{2} \right) = 0. \quad (4.25)$$

È ora possibile riscrivere questi vincoli in forma matriciale come $A^T(u) \dot{q} = 0$, dove $\dot{q} = [\dot{x}_c, \dot{y}_c, \dot{\theta}]^T$ è il vettore delle velocità generalizzate nel sistema di riferimento globale. La matrice dei vincoli è data da

$$A^T = \begin{bmatrix} -\sin(\delta_f) & \cos(\delta_f) & \frac{L}{2} \cos(\delta_f) + \frac{W}{2} \sin(\delta_f) \\ -\sin(\delta_r) & \cos(\delta_r) & -\frac{L}{2} \cos(\delta_r) - \frac{W}{2} \sin(\delta_r) \end{bmatrix}. \quad (4.26)$$

Il rango di questa matrice è sempre 2, ma la forma è 2×3 , il che significa che il sistema ha un grado di libertà. Una soluzione per questo sistema può essere trovata cercando il nucleo di equazione (4.26) il quale rappresenta l'insieme di tutte le velocità **istantanee** ammissibili che soddisfano i vincoli di non slittamento:

$$\ker(A^T) = \begin{bmatrix} \frac{L \cos(\delta_f) \cos(\delta_r)}{\sin(\delta_f - \delta_r)} + \frac{W \sin(\delta_f + \delta_r)}{2 \sin(\delta_f - \delta_r)} \\ \frac{L \sin(\delta_f + \delta_r)}{2 \sin(\delta_f - \delta_r)} + \frac{W \sin(\delta_f) \sin(\delta_r)}{\sin(\delta_f - \delta_r)} \\ 1 \end{bmatrix}. \quad (4.27)$$

A questo punto si può ottenere un modello cinematico che descrive le velocità del veicolo moltiplicando le prime due righe di equazione (4.27) per una velocità scalare v_1 e la terza riga per una velocità scalare v_2 , che però non ha un significato fisico diretto.

Vincoli di puro rotolamento

Oltre al vincolo di non slittamento, è possibile aggiungere ulteriori vincoli richiedendo che le ruote rotolino senza strisciare. Questo significa che la velocità lineare nel punto di contatto di ciascuna ruota deve essere uguale alla velocità tangenziale dovuta alla rotazione della ruota stessa:

$$v_i = r\omega_i, \quad i \in \{f, r\}. \quad (4.28)$$

Questa velocità deve essere uguale alla proiezione della velocità della ruota lungo la direzione della ruota stessa. Si definisce quindi il versore tangente alla ruota come

$$u_i = \begin{bmatrix} \cos(\delta_i) \\ \sin(\delta_i) \end{bmatrix}, \quad i \in \{f, r\}. \quad (4.29)$$

Il vincolo di puro rotolamento diventa quindi

$$v_i^b \cdot u_i = r\omega_i, \quad i \in \{f, r\}. \quad (4.30)$$

Esplicitando per entrambe le ruote, si ha

$$\left(\dot{y}_c^b + \dot{\theta} \frac{L}{2} \right) \sin(\delta_f) + \left(\dot{x}_c^b - \dot{\theta} \frac{W}{2} \right) \cos(\delta_f) = r\omega_f, \quad (4.31)$$

$$\left(\dot{y}_c^b - \dot{\theta} \frac{L}{2} \right) \sin(\delta_r) + \left(\dot{x}_c^b + \dot{\theta} \frac{W}{2} \right) \cos(\delta_r) = r\omega_r. \quad (4.32)$$

Matrice estesa dei vincoli

Considerando sia i vincoli di non slittamento che quelli di puro rotolamento, è possibile costruire una matrice estesa dei vincoli A_{ext}^T tale che

$$A_{ext}^T \dot{q} = \begin{bmatrix} 0 \\ 0 \\ r\omega_f \\ r\omega_r \end{bmatrix}, \quad (4.33)$$

dove la matrice estesa è

$$A_{\text{ext}}^T = \begin{bmatrix} -\sin(\delta_f) & \cos(\delta_f) & \frac{L}{2} \cos(\delta_f) + \frac{W}{2} \sin(\delta_f) \\ -\sin(\delta_r) & \cos(\delta_r) & -\frac{L}{2} \cos(\delta_r) - \frac{W}{2} \sin(\delta_r) \\ \cos(\delta_f) & \sin(\delta_f) & \frac{L}{2} \sin(\delta_f) - \frac{W}{2} \cos(\delta_f) \\ \cos(\delta_r) & \sin(\delta_r) & -\frac{L}{2} \sin(\delta_r) + \frac{W}{2} \cos(\delta_r) \end{bmatrix}. \quad (4.34)$$

Questo è un sistema di equazioni sovradeterminato (4 equazioni in 3 incognite), che in generale non ammette una soluzione esatta. Tuttavia, se ne può trovare una soluzione ai minimi quadrati utilizzando la pseudoinversa di Moore-Penrose. La pseudoinversa è definita come

$$A_{\text{ext}}^\dagger = (A_{\text{ext}}^T A_{\text{ext}})^{-1} A_{\text{ext}}^T. \quad (4.35)$$

Calcolando esplicitamente questa pseudoinversa si ha

$$A_{\text{ext}}^\dagger = \begin{bmatrix} -\frac{\sin(\delta_f)}{\cos^2(\delta_f)} & -\frac{\sin(\delta_r)}{\cos^2(\delta_r)} & \frac{\cos(\delta_f)}{\sin^2(\delta_f)} & \frac{\cos(\delta_r)}{\sin^2(\delta_r)} \\ \frac{\cos^2(\delta_f)}{a_{13}^2} & \frac{\cos^2(\delta_r)}{a_{23}^2} & \frac{\sin^2(\delta_f)}{a_{33}^2} & \frac{\sin^2(\delta_r)}{a_{43}^2} \\ \frac{a_{13}^2}{L^2+W^2} & \frac{a_{23}^2}{L^2+W^2} & \frac{a_{33}^2}{L^2+W^2} & \frac{a_{43}^2}{L^2+W^2} \end{bmatrix}, \quad (4.36)$$

dove per compattezza

$$a_{13} = L \cos(\delta_f) + W \sin(\delta_f), \quad (4.37)$$

$$a_{23} = -L \cos(\delta_r) - W \sin(\delta_r), \quad (4.38)$$

$$a_{33} = L \sin(\delta_f) - W \cos(\delta_f), \quad (4.39)$$

$$a_{43} = -L \sin(\delta_r) + W \cos(\delta_r). \quad (4.40)$$

Moltiplicando la pseudoinversa per il termine noto dell'equazione (4.33) e fattorizzando in termini delle velocità angolari delle ruote, si ottiene il modello cinematico diretto nel sistema di riferimento locale:

$$\begin{bmatrix} \dot{x}_c^b \\ \dot{y}_c^b \\ \dot{\theta} \end{bmatrix} = \frac{r}{2} \begin{bmatrix} \cos(\delta_f) & \cos(\delta_r) \\ \sin(\delta_f) & \sin(\delta_r) \\ \frac{2(L \sin(\delta_f) - W \cos(\delta_f))}{L^2+W^2} & \frac{2(W \cos(\delta_r) - L \sin(\delta_r))}{L^2+W^2} \end{bmatrix} \begin{bmatrix} \omega_f \\ \omega_r \end{bmatrix}. \quad (4.41)$$

Questa rappresentazione evidenzia diverse proprietà interessanti.

- Le velocità lineari $\dot{x}_c^b$ e $\dot{y}_c^b$ sono date da una media pesata dei contributi delle due ruote, dove i pesi dipendono dagli angoli di sterzo.
- La velocità angolare $\dot{\theta}$ dipende dal momento angolare generato dalle ruote, normalizzato da un'approssimazione del momento d'inerzia del robot ($L^2 + W^2$).
- Il modello è non singolare per nessuna configurazione degli angoli di sterzo, permettendo di gestire correttamente tutte le manovre desiderate, incluse le traslazioni laterali e le rotazioni sul posto.
- Non impone in modo rigido una relazione tra le velocità delle ruote motrici e gli angoli di sterzo, permettendo quindi lo slittamento ma minimizzandolo.

Nota: Sulla non singolarità, si può vedere che questa è anche data dal fatto che la matrice $A_{ext}^T A_{ext}$ è sempre invertibile per ogni configurazione degli angoli di sterzo, il che garantisce l'esistenza della pseudoinversa.

Per quanto riguarda la mobilità che questo modello consente, si può osservare che se si impone $\delta_f = \delta_r = \gamma$ con $\omega_f = \omega_r = \dot{\omega}$ si ottiene che

$$\begin{bmatrix} \dot{x}_c^b \\ \dot{y}_c^b \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} \frac{r \cos(\gamma)}{r \sin^2(\gamma)} & \frac{r \cos(\gamma)}{r \sin^2(\gamma)} \\ \frac{r(L \sin(\gamma) - W \cos(\gamma))}{L^2 + W^2} & \frac{r(-L \sin(\gamma) + W \cos(\gamma))}{L^2 + W^2} \end{bmatrix} \begin{bmatrix} \dot{\omega} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} r \dot{\omega} \cos(\gamma) \\ r \dot{\omega} \sin(\gamma) \\ 0 \end{bmatrix}, \quad (4.42)$$

che corrisponde ad una traslazione pura nella direzione definita dall'angolo di sterzo γ . Questo implica che, per $\gamma = \pm \pi/2$, si può effettuare una manovra di *parcheggio parallelo* senza problemi di singolarità, mentre per $\gamma = 0$ o π si ottiene una traslazione avanti o indietro.

Invece, per ottenere una rotazione sul posto, si può imporre $\delta_f = \alpha + \pi/2$ e $\delta_r = \delta_f + \pi$, con $\omega_f = \omega_r = \dot{\omega}$, ottenendo

$$\begin{bmatrix} \dot{x}_c^b \\ \dot{y}_c^b \\ \dot{\theta} \end{bmatrix} = \begin{bmatrix} -\frac{r \sin(\alpha)}{r \cos^2(\alpha)} & \frac{r \sin(\alpha)}{r \cos^2(\alpha)} \\ \frac{r(L \cos(\alpha) + W \sin(\alpha))}{L^2 + W^2} & \frac{r(L \cos(\alpha) - W \sin(\alpha))}{L^2 + W^2} \end{bmatrix} \begin{bmatrix} \dot{\omega} \\ \dot{\omega} \end{bmatrix} = \begin{bmatrix} 0 \\ 0 \\ \frac{2r(L \cos(\alpha) + W \sin(\alpha))}{L^2 + W^2} \dot{\omega} \end{bmatrix}, \quad (4.43)$$

che corrisponde ad una rotazione pura attorno al centro del veicolo.

4.2 Cinematica inversa

L'obiettivo è ora quello di derivare le espressioni per calcolare gli ingressi di controllo necessari per ottenere una data velocità generalizzata del veicolo, ovvero, dato un vettore di velocità generalizzate desiderate $\dot{q}_d = [\dot{x}_d, \dot{y}_d, \dot{\theta}_d]^T$, si vogliono calcolare i valori degli angoli di sterzo δ_f, δ_r e delle velocità angolari delle ruote motrici ω_f, ω_r che permettono di ottenere tali velocità.

Si osservi che è possibile derivare delle soluzioni analitiche per entrambi i modelli cinematici descritti, sfruttando delle considerazioni di tipo geometrico basate sul concetto di centro di rotazione istantaneo (ICR), ovvero, il punto attorno al quale il veicolo sta ruotando in un dato istante di tempo.

L'ICR può essere determinato considerando l'intersezione degli assi di rotazione delle ruote motrici, che sono perpendicolari alla direzione di avanzamento di ciascuna ruota. Per la ruota frontale, l'asse di rotazione è dato dalla retta passante per il punto di montaggio della ruota e con angolo $\delta_f + \pi/2$ rispetto all'asse longitudinale del veicolo. Analogamente, per la ruota posteriore, l'asse di rotazione è dato dalla retta passante per il punto di montaggio della ruota e con angolo $\delta_r + \pi/2$.

Per semplicità, ci si ponga ora nel sistema di riferimento locale al robot. È possibile scrivere che

$$\begin{bmatrix} x_{ICR} \\ y_{ICR} \end{bmatrix} = p_f^b + s_f \begin{bmatrix} \cos(\delta_f + \pi/2) \\ \sin(\delta_f + \pi/2) \end{bmatrix} = p_r^b + s_r \begin{bmatrix} \cos(\delta_r + \pi/2) \\ \sin(\delta_r + \pi/2) \end{bmatrix}, \quad (4.44)$$

dove s_f e s_r sono delle variabili scalari che rappresentano la distanza lungo l'asse di rotazione di ciascuna ruota fino all'ICR. Ora, ricordando che $\cos(k + \pi/2) = -\sin(k)$ e $\sin(k + \pi/2) = \cos(k)$, si possono riscrivere le equazioni come

$$\begin{bmatrix} x_{ICR} \\ y_{ICR} \end{bmatrix} = p_f^b + s_f \begin{bmatrix} -\sin(\delta_f) \\ \cos(\delta_f) \end{bmatrix} = p_r^b + s_r \begin{bmatrix} -\sin(\delta_r) \\ \cos(\delta_r) \end{bmatrix}. \quad (4.45)$$

Risolvendo il sistema di equazioni rispetto a s_f e s_r , si hanno

$$s_f = \frac{(p_{fx} - p_{rx}) \cos(\delta_r) + (p_{fy} - p_{ry}) \sin(\delta_r)}{\sin(\delta_f - \delta_r)}, \quad (4.46)$$

$$s_r = \frac{(p_{fx} - p_{rx}) \cos(\delta_f) + (p_{fy} - p_{ry}) \sin(\delta_f)}{\sin(\delta_f - \delta_r)}. \quad (4.47)$$

Si può osservare che, poiché in un moto generico il veicolo si muove ruotando attorno all'ICR, il punto di contatto di ciascuna ruota con il suolo deve appartenere ad una circonferenza centrata nell'ICR con raggio pari alla distanza tra tale punto di contatto e l'ICR stesso. Questo implica che, denotando con R_f ed R_r i raggi delle circonferenze per la ruota anteriore e posteriore, si ha che

$$R_f = \sqrt{(x_{ICR} - p_{fx})^2 + (y_{ICR} - p_{fy})^2} = |s_f|, \quad (4.48)$$

$$R_r = \sqrt{(x_{ICR} - p_{rx})^2 + (y_{ICR} - p_{ry})^2} = |s_r|. \quad (4.49)$$

Nei casi considerati, si hanno le seguenti configurazioni:

- **Modello a biciclo sterzante:**

$$R_f = \sqrt{\frac{(l_f + l_r)^2 \cos^2(\delta_r)}{\sin^2(\delta_f - \delta_r)}}, \quad (4.50)$$

$$R_r = \sqrt{\frac{(l_f + l_r)^2 \cos^2(\delta_f)}{\sin^2(\delta_f - \delta_r)}}, \quad (4.51)$$

$$x_{icr} = -\frac{l_f \sin(\delta_r) \cos(\delta_f) + l_r \sin(\delta_f) \cos(\delta_r)}{\sin(\delta_f - \delta_r)}, \quad (4.52)$$

$$y_{icr} = \frac{(l_f + l_r) \cos(\delta_f) \cos(\delta_r)}{\sin(\delta_f - \delta_r)}. \quad (4.53)$$

- **Modello a doppia ruota sterzante:**

$$R_f = \sqrt{\frac{(L \cos(\delta_r) + W \sin(\delta_r))^2}{\sin^2(\delta_f - \delta_r)}}, \quad (4.54)$$

$$R_r = \sqrt{\frac{(L \cos(\delta_f) + W \sin(\delta_f))^2}{\sin^2(\delta_f - \delta_r)}}, \quad (4.55)$$

$$x_{icr} = -\frac{L \sin(\delta_f + \delta_r) + 2W \sin(\delta_f) \sin(\delta_r)}{2 \sin(\delta_f - \delta_r)}, \quad (4.56)$$

$$y_{icr} = \frac{2L \cos(\delta_f) \cos(\delta_r) + W \sin(\delta_f + \delta_r)}{2 \sin(\delta_f - \delta_r)}. \quad (4.57)$$

Ora, osservando che la velocità angolare delle ruote è legata alla velocità angolare del robot tramite la relazione

$$\dot{\theta} R_i = r \omega_i \quad , \quad i \in \{f, r\}, \quad (4.58)$$

si può dedurre la *condizione di compatibilità* tra le velocità desiderate del robot affinché il moto sia fisicamente realizzabile senza slittamento, ovvero, senza che le ruote si allontanino dalla stessa traiettoria circolare definita dall'ICR. Questa condizione è data da

$$R_r \omega_f = R_f \omega_r. \quad (4.59)$$

Nota: È importante notare che, per costruzione, quando gli angoli di sterzo sono tali per cui le direzioni di avanzamento delle ruote sono parallele, le distanze R_f e R_r tendono all'infinito (portando anche l'ICR all'infinito), rendendo impossibile soddisfare la condizione di compatibilità definita dalla equazione (4.59). Pertanto, in tali situazioni, l'equazione (4.59) diventa

$$\omega_f = \omega_r \quad (4.60)$$

4.2.1 Modello a Biciclo Sterzante

Sia $\dot{q}_d = [\dot{x}_d, \dot{y}_d, \dot{\theta}_d]^T$ il vettore delle velocità desiderate nel sistema di riferimento globale. Se consideriamo le relazioni per $\dot{x}$ e $\dot{y}$ date da equazione (4.2), si può osservare che:

$$\begin{aligned} \dot{x}^2 + \dot{y}^2 &= v^2 \cos^2(\theta + \beta) + v^2 \sin^2(\theta + \beta) \\ &= v^2 (\cos^2(\theta + \beta) + \sin^2(\theta + \beta)) \\ &= v^2 \end{aligned}$$

Quindi, la velocità del veicolo desiderata data dal vettore $\dot{q}_d$ è:

$$v_d = \sqrt{\dot{x}_d^2 + \dot{y}_d^2} \quad (4.61)$$

Inoltre, se consideriamo il rapporto tra le velocità si vede che:

$$\frac{\dot{y}}{\dot{x}} = \frac{v \sin(\theta + \beta)}{v \cos(\theta + \beta)} = \tan(\theta + \beta)$$

Da cui, per le velocità desiderate, si ottiene:

$$\beta_d = \text{atan2}(\dot{y}_d, \dot{x}_d) - \theta \quad (4.62)$$

Dove θ è l'orientamento attuale del veicolo.

Supponiamo di cercare una soluzione tale per cui $\dot{\theta} \neq 0$, in modo da poter definire un ICR finito. Per l'equazione (4.4), β_d deve soddisfare la relazione:

$$\tan(\beta_d) = \frac{l_r \tan(\delta_f) + l_f \tan(\delta_r)}{l_f + l_r} \quad (4.63)$$

Lo stesso è vero per la velocità angolare desiderata, che deve soddisfare la relazione data da equazione (4.2):

$$\dot{\theta}_d = \frac{v_d \cos(\beta_d)(\tan(\delta_f) - \tan(\delta_r))}{l_f + l_r} \quad (4.64)$$

Definendo per semplicità:

$$S = \tan(\beta_d) \quad (4.65)$$

$$D = \frac{l\dot{\theta}_d}{v_d \cos(\beta_d)} \quad (4.66)$$

Possiamo comporre il sistema di equazioni:

$$\begin{cases} l_r \tan(\delta_f) + l_f \tan(\delta_r) = lS \\ \tan(\delta_f) - \tan(\delta_r) = D \end{cases} \quad (4.67)$$

Risolvendo questo sistema rispetto a $\tan(\delta_f)$ e $\tan(\delta_r)$, otteniamo:

$$\delta_f = \arctan\left(S + \frac{l_f}{l}D\right) \quad (4.68)$$

$$\delta_r = \arctan\left(S - \frac{l_r}{l}D\right) \quad (4.69)$$

Noti gli angoli di sterzo, possiamo calcolare i raggi R_f e R_r usando le equazioni (4.50) and (4.51), e quindi le velocità angolari delle ruote motrici usando la relazione di compatibilità data da equazioni (4.58) and (4.59). Supponendo di usare la ruota frontale come ruota dominante, possiamo scrivere:

$$\omega_r = \frac{R_r}{R_f} \omega_f \quad (4.70)$$

E infine ottenere ω_f riscrivendo l'equazione (4.3) rispetto a tale variabile:

$$\omega_f = \frac{2R_f v_d \cos(\beta_d)}{r(R_f \cos(\delta_f) + R_r \cos(\delta_r))} \quad (4.71)$$

Considerando ora il caso in cui $\dot{\theta}_d = 0$, si ha che l'ICR tende all'infinito e quindi le direzioni di avanzamento delle ruote diventano parallele. In questo caso, possiamo imporre $\delta_f = \delta_r = \delta$ e notare da equazione (4.63) che:

$$\beta_d = \arctan\left(\frac{l_f + l_r}{l_f + l_r} \tan(\delta)\right) = \delta \quad (4.72)$$

Per cui vale che:

$$\delta = \text{atan2}(\dot{y}_d, \dot{x}_d) - \theta \quad (4.73)$$

Mentre la equazione (4.60), possiamo semplicemente imporre che:

$$\omega_f = \omega_r = \frac{v_d}{r} = \frac{\sqrt{\dot{x}_d^2 + \dot{y}_d^2}}{r} \quad (4.74)$$

Nota: Ancora una volta si può notare la singolarità del modello biciclo nel caso in cui si cerchi di ottenere una traslazione rigida con sterzi a $\pm\pi/2$, in quanto $\dot{x}_d = 0$ la equazione (4.73) non è definita, richiedendo.

4.2.2 Modello a Doppia Ruota Sterzante

Per il modello a doppia ruota sterzante, possiamo seguire un procedimento simile al precedente, ma sfruttando direttamente le relazioni date dai vincoli di non slittamento. Infatti, considerando le equazioni (4.24) and (4.25) e risolvendo rispetto agli angoli di sterzo δ_f e δ_r , otteniamo:

$$\delta_f = \text{atan2}\left(\dot{y}_d + \dot{\theta}_d \frac{L}{2}, \dot{x}_d - \dot{\theta}_d \frac{W}{2}\right) \quad (4.75)$$

$$\delta_r = \text{atan2}\left(\dot{y}_d - \dot{\theta}_d \frac{L}{2}, \dot{x}_d + \dot{\theta}_d \frac{W}{2}\right) \quad (4.76)$$

A questo punto, per $\dot{\theta} \neq 0$ dalle equazioni di puro rotolamento date da equazioni (4.31) and (4.32), possiamo calcolare le velocità angolari delle ruote motrici come:

$$\omega_f = \frac{(\dot{y}_d + \dot{\theta}_d \frac{L}{2}) \sin(\delta_f) + (\dot{x}_d - \dot{\theta}_d \frac{W}{2}) \cos(\delta_f)}{r} \quad (4.77)$$

$$\omega_r = \frac{(\dot{y}_d - \dot{\theta}_d \frac{L}{2}) \sin(\delta_r) + (\dot{x}_d + \dot{\theta}_d \frac{W}{2}) \cos(\delta_r)}{r} \quad (4.78)$$

Nel caso in cui si desideri una traslazione pura, ovvero $\dot{\theta}_d = 0$, si ha che le direzioni di avanzamento delle ruote diventano parallele. In questo caso, possiamo imporre $\delta_f = \delta_r = \delta$ e notare che dalle equazioni (4.75) and (4.76) si ottiene:

$$\delta = \text{atan2}(\dot{y}_d, \dot{x}_d) \quad (4.79)$$

Mentre le velocità angolari delle ruote motrici diventano:

$$\omega_f = \omega_r = \frac{\sqrt{\dot{x}_d^2 + \dot{y}_d^2}}{r} \quad (4.80)$$

4.3 Analisi dei modelli e Simulazioni

In questa sezione confronteremo i due modelli cinematici sviluppati, analizzandone le caratteristiche principali e le differenze, mediante un sistema di controllo in catena aperta su traiettorie preprogrammate. Lo scopo è quello di evidenziare i punti di forza e le limitazioni di ciascun modello, al fine di guidare la scelta del modello più adatto per applicazioni specifiche.

4.3.1 Setup e Considerazioni Preliminari

I due modelli cinematici presentano caratteristiche distintive che li rendono più o meno adatti a diverse situazioni operative. Tuttavia, entrambi i mo-

delli peccano di alcune limitazioni intrinseche legate alla loro natura semplificata di modelli cinematici e inoltre assumono di poter variare istantaneamente gli angoli di sterzo per ottenere tutte le manovre desiderate, il che non è realistico visto che nella realtà gli attuatori di sterzo e trazione hanno limiti di velocità e accelerazione. Per questo, nelle simulazioni che seguiranno, si è scelto di includere almeno dei limiti sulle velocità angolari e agli angoli di sterzo, approssimando inoltre la dinamica degli attuatori con un semplice modello del primo ordine con funzione di trasferimento:

$$P(s) = \frac{K}{\tau s + 1} \quad (4.81)$$

Dove K è il guadagno statico e τ la costante di tempo dell'attuatore. Nella tabella 4.1 sono riportati i parametri utilizzati per le simulazioni. La dinamica

Parametro	Simbolo	Valore	Unità
Lunghezza veicolo	L	2	m
Larghezza veicolo	W	1	m
Diametro ruote	r	0.198	m
Angolo di montaggio ruote	α	25.57	deg
Guadagno attuatori	K	1	-
Cost. tempo trazione	τ_ω	1	s
Cost. tempo sterzo	τ_δ	2	s
Massima velocità angolare ruote	ω_{max}	10	rad/s
Massimo angolo di sterzo	δ_{max}	$\pm \frac{2\pi}{3}$	rad

Tabella 4.1: Parametri utilizzati per le simulazioni.

degli attuatori inoltre rende praticamente impossibile per entrambi i modelli soddisfare le condizioni di compatibilità tra le velocità delle ruote e gli angoli di sterzo, specialmente durante traiettorie curvilinee. Quest porta inevitabilmente a fenomeni di slittamento delle ruote, che i modelli cinematici non sono in grado di rappresentare accuratamente. Per valutare le prestazioni dei modelli, si è scelto di utilizzare le seguenti traiettorie di riferimento:

- **Traiettoria Rettilinea:** Una semplice traiettoria lineare diretta verso Est, partendo dall'origine.

$$\begin{cases} x(t) = v_{ref}t \\ y(t) = 0 \\ \theta(t) = 0 \\ \dot{x}(t) = v_{ref} \\ \dot{y}(t) = 0 \\ \dot{\theta}(t) = 0 \end{cases} \quad \begin{cases} v_{ref} = 0.5 \text{ m/s} \\ T_{sim} = 5 \text{ s} \\ q(0) = [0, 0, 0]^T \\ q(T) = [2.5, 0, 0]^T \end{cases} \quad (4.82)$$

- **Traiettoria Circolare:** Una traiettoria circolare con raggio di 3 m, parten-

do dall'origine e ruotando in senso antiorario.

$$\begin{cases} x(t) = R \sin(\omega_{ref} t) \\ y(t) = R(1 - \cos(\omega_{ref} t)) \\ \theta(t) = \omega_{ref} t \\ \dot{x}(t) = v_{ref} \cos(\omega_{ref} t) \\ \dot{y}(t) = v_{ref} \sin(\omega_{ref} t) \\ \dot{\theta}(t) = \omega_{ref} \end{cases} \quad \begin{aligned} R &= 3 \text{ m} \\ v_{ref} &= 0.6 \text{ m/s} \\ \omega_{ref} &= 0.2 \text{ rad/s} \\ T_{sim} &= 8 \text{ s} \\ q(0) &= [0, 0, 0]^T \\ q(T) &= [3, 3, 2\pi]^T \end{aligned} \quad (4.83)$$

- **Traiettoria Traslatoria a Nord:** Una traslazione rigida verso Nord, partendo dall'origine mantenendo orientamento costante verso Est.

$$\begin{cases} x(t) = 0 \\ y(t) = v_{ref} t \\ \theta(t) = 0 \\ \dot{x}(t) = 0 \\ \dot{y}(t) = v_{ref} \\ \dot{\theta}(t) = 0 \end{cases} \quad \begin{aligned} v_{ref} &= 0.4 \text{ m/s} \\ T_{sim} &= 5 \text{ s} \\ q(0) &= [0, 0, 0]^T \\ q(T) &= [0, 2, 0]^T \end{aligned} \quad (4.84)$$

- **Rotazione sul Posto:** Una rotazione sul posto di circa 180 gradi in senso antiorario, partendo dall'origine.

$$\begin{cases} x(t) = 0 \\ y(t) = 0 \\ \theta(t) = \omega_{ref} t \\ \dot{x}(t) = 0 \\ \dot{y}(t) = 0 \\ \dot{\theta}(t) = \omega_{ref} \end{cases} \quad \begin{aligned} \omega_{ref} &= \pi/6 \text{ rad/s} \\ T_{sim} &= 6 \text{ s} \\ q(0) &= [0, 0, 0]^T \\ q(T) &= [0, 0, \pi]^T \end{aligned} \quad (4.85)$$

Nel seguito, chiameremo per semplicità queste traiettorie rispettivamente *Straight*, *Arc*, *Crab* e *Spin*. Per ogni traiettoria, si calcolano le velocità desiderate $\dot{q}_d$ e si utilizzano le formule di cinematica inversa per ottenere gli input di controllo per entrambi i modelli. Successivamente, si simula l'evoluzione del sistema nel tempo, tenendo conto della dinamica degli attuatori e dei limiti imposti. Infine, si confrontano le traiettorie effettive seguite dai modelli con quelle desiderate, analizzando gli errori di tracciamento e le caratteristiche del moto risultante.

Nota: Poiché in un contesto reale le traiettorie *Crab* e *Spin* non sono facilmente realizzabili in movimento, si è scelto di simulare solo il caso in cui il veicolo parte da fermo con gli sterzi già orientati correttamente per la manovra desiderata. Questo permette di isolare l'effetto del modello cinematico senza introdurre ulteriori complessità legate alla fase di transizione iniziale.

Lo schema di controllo utilizzato per le simulazioni in catena aperta illustrato in figura 4.3 segue la logica descritta precedentemente, dove per semplicità di rappresentazione abbiamo indicato con:

- IK le operazioni di cinematica inversa per calcolare gli input di controllo dagli stati desiderati.
- FK le operazioni di cinematica diretta per calcolare gli stati effettivi del veicolo dagli input di controllo.
- $\frac{1}{\tau_j s + 1}, j = 1, 2$ le funzioni di trasferimento della dinamica degli attuatori, che sono state implementate in forma discreta utilizzando il metodo di Eulero in avanti con passo di integrazione pari a quello della simulazione.

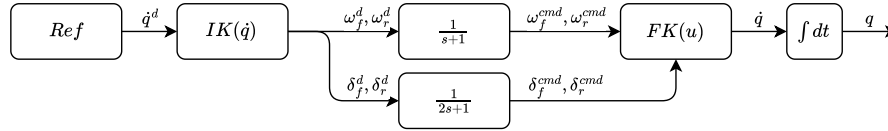


Figura 4.3: Schema di controllo in catena aperta utilizzato per le simulazioni.

Nota: È importante notare che per lo schema di controllo in catena aperta del biciclo, si è utilizzato come angolo θ dell'equazione (4.62) quello proveniente dalla traiettoria desiderata e non quello effettivo del veicolo.

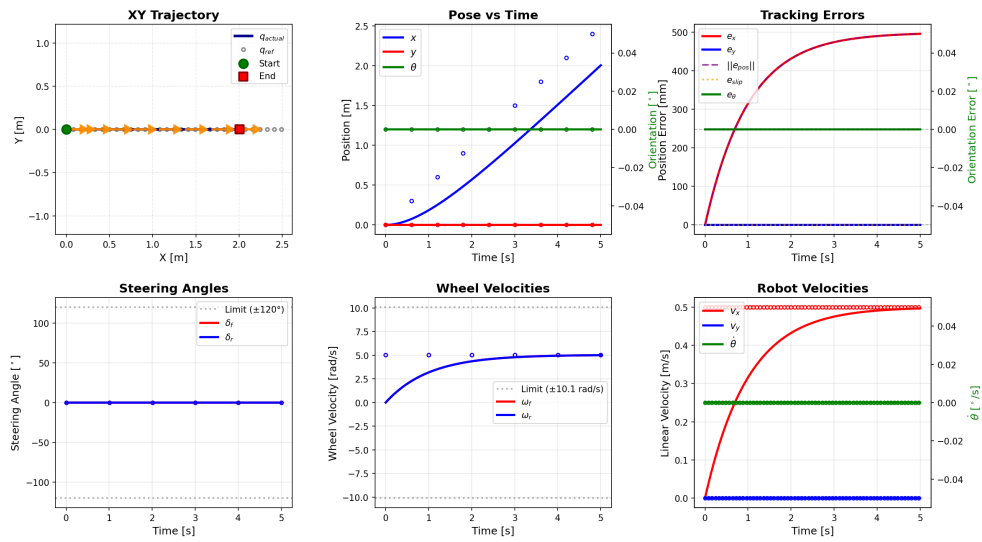
Nota: Le implementazioni di cinematica diretta e inversa includono saturazioni sugli angoli di sterzo e sulle velocità angolari delle ruote motrici, in modo da rispettare i limiti imposti dagli attuatori.

4.3.2 Simulazioni in Catena Aperta

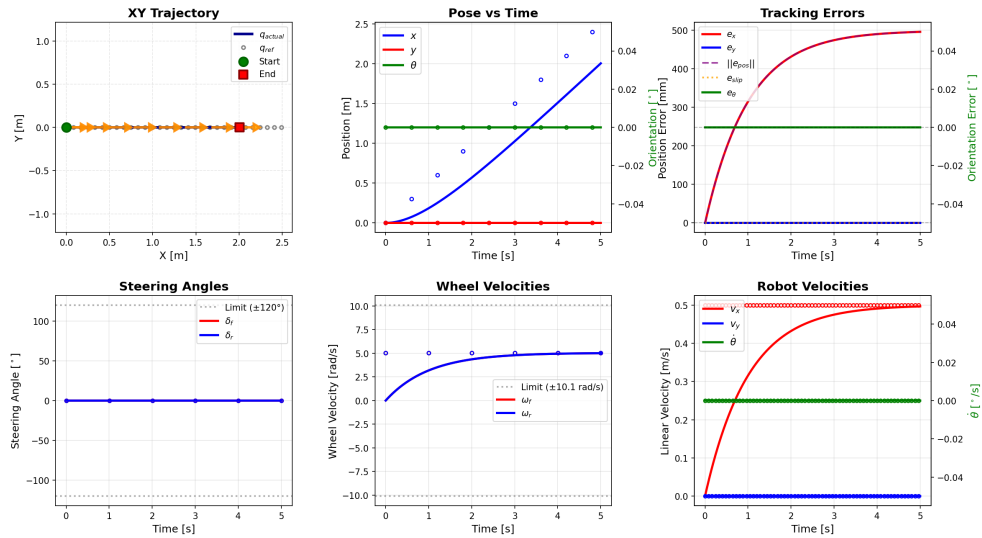
Tutte le simulazioni sono state eseguite in Python, derivando le equazioni simbolicamente con la libreria SymPy e implementando la simulazione numerica con NumPy e SciPy. I grafici sono stati generati utilizzando Matplotlib. Le simulazioni sono state eseguite con un passo di integrazione di 0.002s utilizzando il metodo di integrazione di Runge-Kutta del quarto ordine (RK4). In tabella 4.2 sono riportati i risultati delle simulazioni per ciascuna traiettoria, valutati in termini di Root Mean Square Error (RMSE) sia per la posizione che per l'orientamento del veicolo rispetto alla traiettoria desiderata.

Traiettoria	Modello	Supporto	RMSE Pos. [mm]	RMSE Orient. [°]
Straight	Bicycle	x	418.05	0.00
	Bisteer	x	418.05	0.00
Arc	Bicycle	x	749.92	20.80
	Bisteer	x	694.70	18.85
Crab	Bicycle	-	1154.82	0.00
	Bisteer	x	334.44	0.00
Spin	Bicycle	-	0.00	103.93
	Bisteer	x	0.00	25.93

Tabella 4.2: Confronto prestazioni modelli Bicycle e Bisteer con controllore **FeedForward**

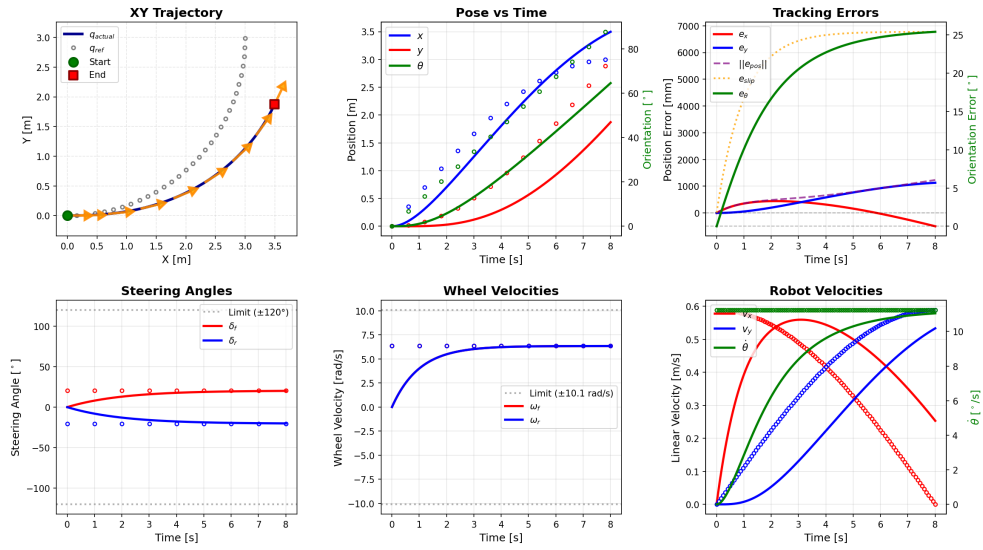


(a) Simulazione con modello biciclo sterzante per traiettoria Straight.

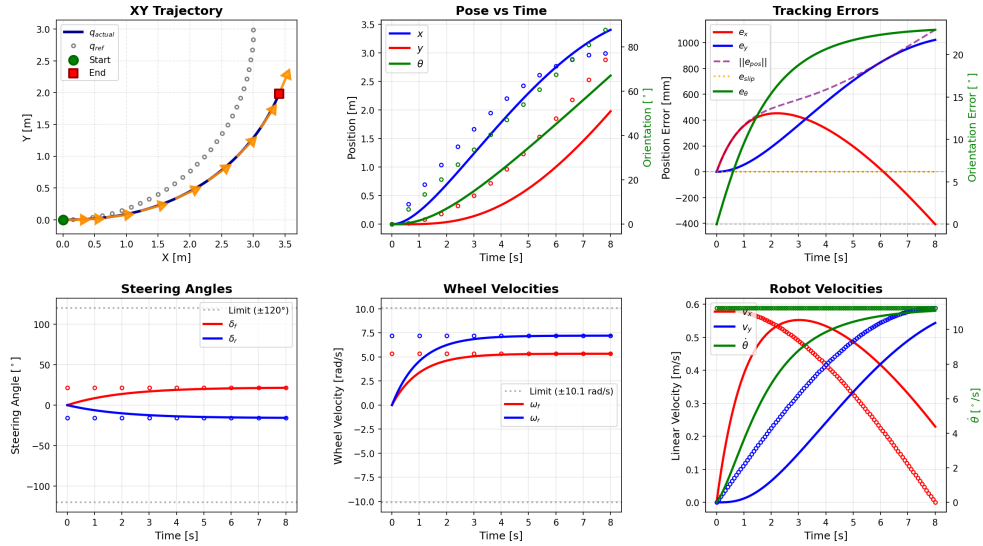


(b) Simulazione con modello a doppia ruota sterzante per traiettoria Straight.

Figura 4.4: Risultati delle simulazioni in 62° a tenuta aperta per la traiettoria rettilinea verso Est.

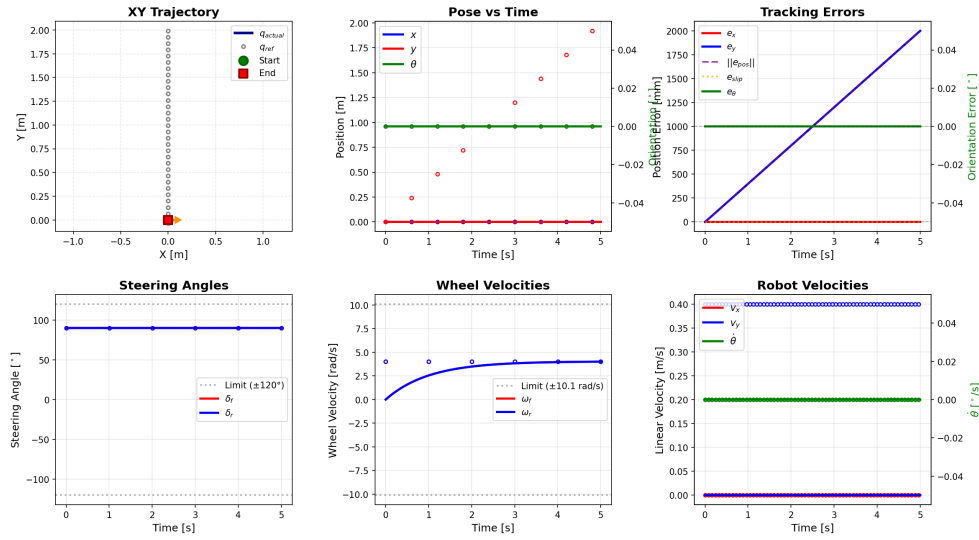


(a) Simulazione con modello biciclo sterzante per traiettoria Arc.

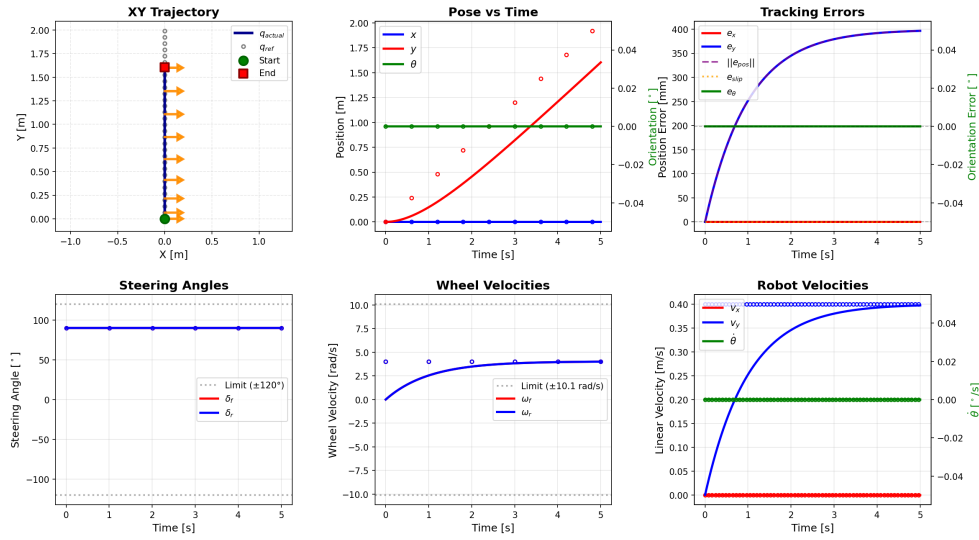


(b) Simulazione con modello a doppia ruota sterzante per traiettoria Arc.

Figura 4.5: Risultati delle simulazioni in catena aperta per la traiettoria circolare.

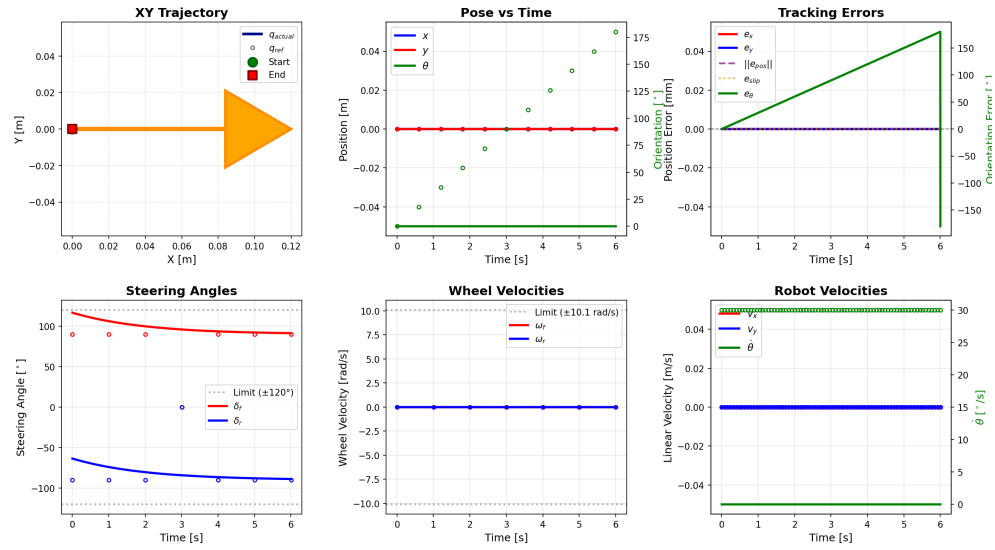


(a) Simulazione con modello biciclo sterzante per traiettoria Crab.

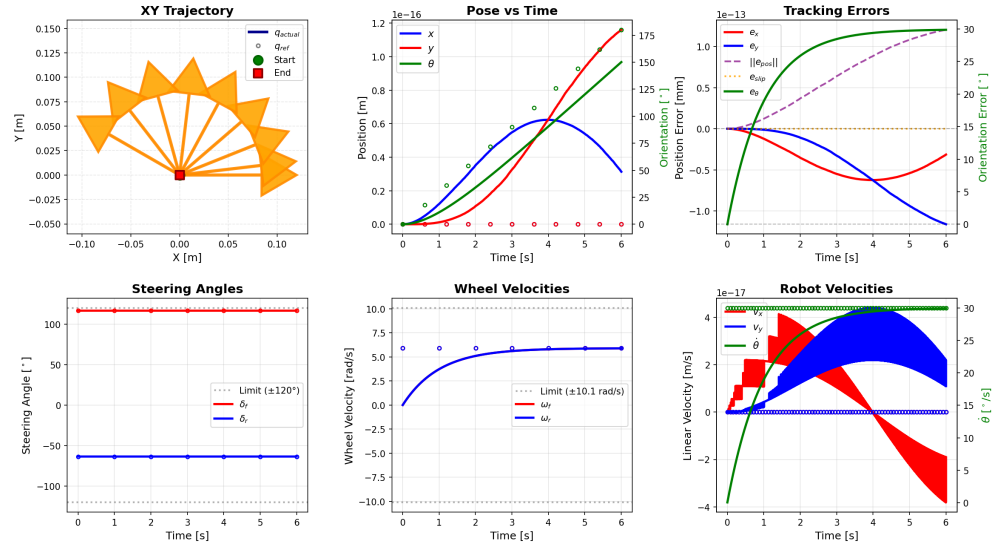


(b) Simulazione con modello a doppia ruota sterzante per traiettoria Crab.

Figura 4.6: Risultati delle simulazioni in catena aperta per la traiettoria traslatoria verso Nord.



(a) Simulazione con modello biciclo sterzante per traiettoria Spin.



(b) Simulazione con modello a doppia ruota sterzante per traiettoria Spin.

Figura 4.7: Risultati delle simulazioni in catena aperta per la traiettoria di rotazione sul posto.

Discussione dei Risultati

Dai risultati delle simulazioni in catena aperta, emergono alcune considerazioni chiave: come ci si aspettava, il modello biciclo non è in grado di eseguire correttamente le manovre di traslazione laterale e rotazione sul posto. Difatti, a livello implementativo, questo modello tende a generare angoli di sterzo estremi che si risolvono in *NaN* durante la fase di calcolo della cinematica inversa, portando a comportamenti non definiti nella simulazione che NumPy tipicamente gestisce come zeri.

Risulta interessante però vedere che il modello a doppia ruota sterzante si comporta in modo analogo nel portare il centro del robot da una posa a un'altra, tenendo a zero l'errore di slip e mostrando una possibile validità teorica del modello proposto. Inoltre, questo mostra una maggiore versatilità, riuscendo a seguire le traiettorie Crab e Spin anche se con un certo errore.

In conclusione, possiamo aggiungere che la dinamica degli attuatori gioca un ruolo cruciale nel determinare le prestazioni di entrambi i modelli. Infatti, la presenza di ritardi e limiti nelle variazioni degli angoli di sterzo e delle velocità delle ruote motrici impedisce ai modelli di soddisfare le condizioni di compatibilità richieste per quando si è al di fuori delle traiettorie rettilinee e inoltre non permette di portare a zero gli errori di tracciamento a causa della mancanza di un feedback di controllo.

4.4 Controllo in Feedback

Per migliorare le prestazioni di tracking, si è implementato un semplice controllore in feedback dallo stato di tipo PID, che agisce sugli errori di posizione ed orientamento del veicolo rispetto alla traiettoria desiderata. Lo schema di controllo utilizzato è illustrato in figura 4.8, dove per semplicità espositiva abbiamo usato le stesse notazioni della sezione precedente, aggiungendo però un blocco solo di controllo PID che però tiene in considerazione tutte e tre le componenti di stato del veicolo.

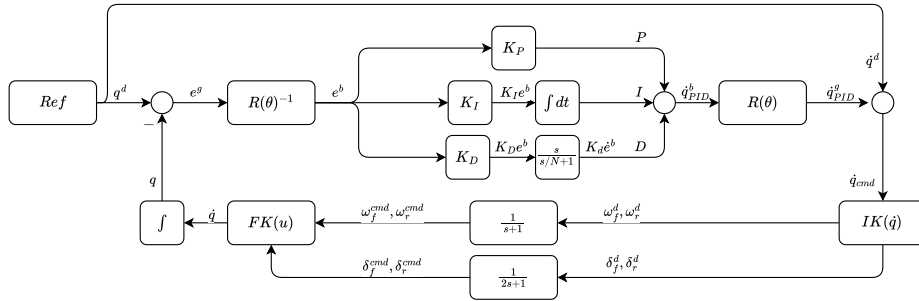


Figura 4.8: Schema di controllo in feedback utilizzato per le simulazioni.

Nota: Tale schema è stato selezionato per provare un primo metodo di controllo semplice ed efficace, che potesse inoltre essere una prima approssimazione dello schema di controllo reale implementato sul veicolo fisico, i cui attuatori sono già dotati di PID opportunamente configurati.

Nel dettaglio, l'algoritmo di controllo implementato calcola l'errore di posizione ed orientamento nel sistema di riferimento globale come:

$$e^g = \begin{bmatrix} x_d - x \\ y_d - y \\ \theta_d - \theta \end{bmatrix} \quad (4.86)$$

riportandolo nel sistema di riferimento del veicolo usando il valore misurato di θ :

$$e^b = \begin{bmatrix} \cos(\theta) & \sin(\theta) & 0 \\ -\sin(\theta) & \cos(\theta) & 0 \\ 0 & 0 & 1 \end{bmatrix} e^g \quad (4.87)$$

Nota: Come descritto in [13], l'errore di tracciamento viene trasformato nel sistema di riferimento locale del robot (equazione (4.87)) per permettere al controllore di operare in modo più intuitivo e robusto.

Successivamente, si calcolano le azioni di controllo sulla singola componente dello stato come:

$$u_i(t) = K_{p,i}e_i^b(t) + K_{i,i} \int_0^t e_i^b(\tau) d\tau + K_{d,i} \frac{de_i^b(t)}{dt}, \quad i \in \{x, y, \theta\} \quad (4.88)$$

Dove $K_{p,i}$, $K_{i,i}$ e $K_{d,i}$ sono i guadagni proporzionale, integrale e derivativo del controllore PID per la componente i dello stato. Le azioni di controllo calcolate vengono poi riportate nel sistema di riferimento globale e sommate alle velocità desiderate provenienti dalla traiettoria di riferimento per ottenere le velocità di input corrette:

$$\dot{q}_{cmd} = \dot{q}^d + u \quad (4.89)$$

Dove $u = [u_x, u_y, u_\theta]^T$ è il vettore delle azioni di controllo calcolate. Infine, si utilizzano le formule di cinematica inversa per ottenere gli input di controllo per entrambi i modelli, e si procede con la simulazione dell'evoluzione del sistema nel tempo, tenendo conto della dinamica degli attuatori e dei limiti imposti.

Nota: Si noti che, a differenza dello schema di controllo in catena aperta, in questo caso si utilizza il valore misurato di θ per calcolare l'errore di tracciamento, permettendo al controllore di correggere gli errori accumulati durante la simulazione.

A seguito di alcune prove sperimentali, si è deciso di implementare il controllore derivativo con un filtro passa basso del primo ordine per evitare di amplificare il rumore di misura e le variazioni rapide dell'errore (importanti soprattutto per la componente angolare). Pertanto, la parte derivativa del controllore PID è stata implementata come:

$$D_i(s) = \frac{K_{d,i}s}{\frac{s}{N} + 1} \quad (4.90)$$

Dove $1/N$ è la costante di tempo del filtro passa basso. Inoltre, per evitare fenomeni di wind-up nell'azione integrale del controllore, si è implementato un meccanismo di anti-windup basato sulla saturazione dell'integrale. In pratica, l'integrale dell'errore viene limitato a un intervallo predefinito per evitare che cresca indefinitamente in presenza di errori persistenti.

Per quanto riguarda invece le saturazioni applicate per rispettare i limiti fisici degli attuatori, si è applicato uno *scaling proporzionale* alle azioni di controllo calcolate, in modo da mantenere le proporzioni tra le diverse componenti di

controllo, ma riducendo la loro magnitudine complessiva per rientrare nei limiti imposti. Lo scaling è calcolato nel modo seguente: Definita la magnitudine massima tra le azioni di controllo calcolate come:

$$v_{mag} = \sqrt{u_x^2 + u_y^2} \quad (4.91)$$

Se $v_{mag} > v_{max}$, con v_{max} la massima velocità lineare ottenibile dagli attuatori, si calcola il fattore di scaling come:

$$k = \frac{v_{max}}{v_{mag}} \quad (4.92)$$

E si scala ogni componente dell'azione di controllo come:

$$u_i = k u_i \quad , \quad i \in \{x, y\} \quad (4.93)$$

Mentre per la componente angolare si applica una saturazione diretta:

$$u_\theta = \begin{cases} u_\theta & \text{se } |u_\theta| \leq \omega_{max} \\ \text{sgn}(u_\theta) \cdot \omega_{max} & \text{altrimenti} \end{cases} \quad (4.94)$$

Dove ω_{max} è la massima velocità angolare ottenibile dagli attuatori. I guadagni del controllore PID sono stati scelti empiricamente attraverso una serie di test e ottimizzazioni manuali, al fine di bilanciare la reattività del sistema con la stabilità e la robustezza del controllo. I valori finali utilizzati per le simulazioni sono riportati in tabella 4.3.

Componente	K_p	K_i	K_d	I_{max}	N
x	0.3	0.05	0.1	2	20
y	0.3	0.05	0.1	2	20
θ	0.2	0.02	0.05	1	20

Tabella 4.3: Guadagni e costanti dei controllori PID utilizzati per le simulazioni.

4.4.1 Simulazioni in Feedback

Le simulazioni in feedback sono state eseguite utilizzando lo stesso setup e le stesse traiettorie descritte nella sezione precedente, con l'aggiunta del controllore PID. I risultati delle simulazioni sono riportati in tabella 4.4, dove sono confrontati i valori di RMSE per la posizione e l'orientamento del veicolo rispetto alla traiettoria desiderata.

Traiettoria	Modello	Supporto	RMSE Pos. [mm]	RMSE Orient. [°]
Straight	Bicycle	x	233.96	0.00
	Bisteer	x	233.96	0.00
Arc	Bicycle	x	231.27	12.53
	Bisteer	x	229.57	11.66
Crab	Bicycle	-	1154.82	0.00
	Bisteer	x	187.17	0.00
Spin	Bicycle	-	0.00	103.93
	Bisteer	x	0.00	16.14

Tabella 4.4: Confronto prestazioni modelli Bicycle e Bisteer con **PID+FF**.

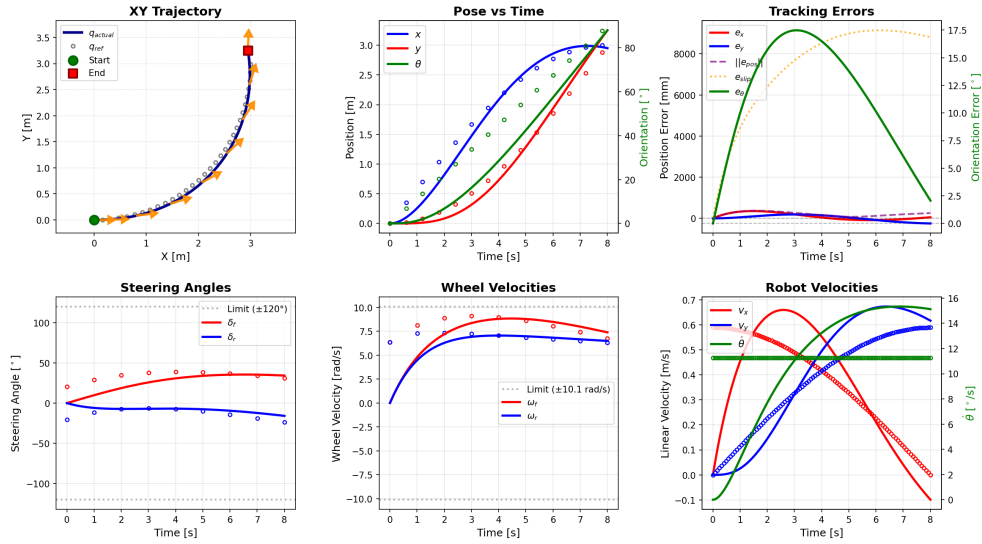
Discussione dei Risultati

L'introduzione del controllore PID in feedback ha portato a miglioramenti significativi nelle prestazioni di tracking per entrambi i modelli cinematici. Purtroppo, in entrambi i casi non si mantiene praticamente mai l'errore di slip a zero, a causa dei ritardi introdotti dagli attuatori e dalle limitazioni fisiche del sistema. Tuttavia, le performance complessive sono migliorate e il modello a doppia ruota sterzante si conferma anche quello con il miglior comportamento di tracking.

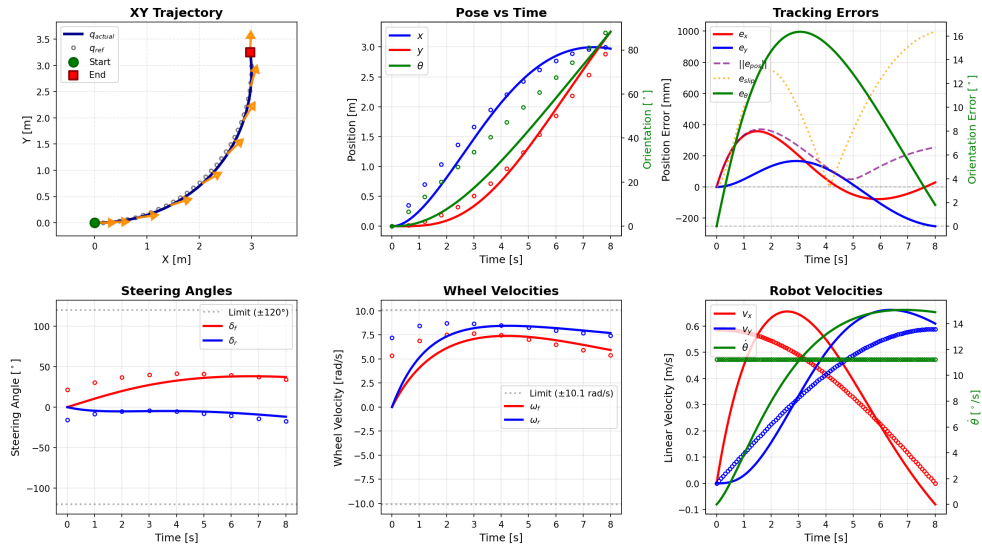
Nota: Si può notare in figura 4.11 come ci sia un rumore molto evidente nelle velocità del robot, probabilmente dovuti a fenomeni di errore numerico, visto che la loro scale è di 10^{-17} . Questo non influenza però le prestazioni di tracking, che risultano comunque ottimali.

In conclusione, l'analisi delle simulazioni in feedback conferma la validità del modello a doppia ruota sterzante sopra a quello del biciclo sterzante per l'applicazione considerata, specialmente a fronte della maggior versatilità.

Questo implica che, sebbene il modello cinematico descritto da [19] sia valido per rappresentare il comportamento di veicoli a quattro ruote sterzanti, il modello proposto in questo lavoro offre un'alternativa più semplice e diretta per la modellazione e il controllo di tali veicoli, pur rappresentando una grande approssimazione della realtà.

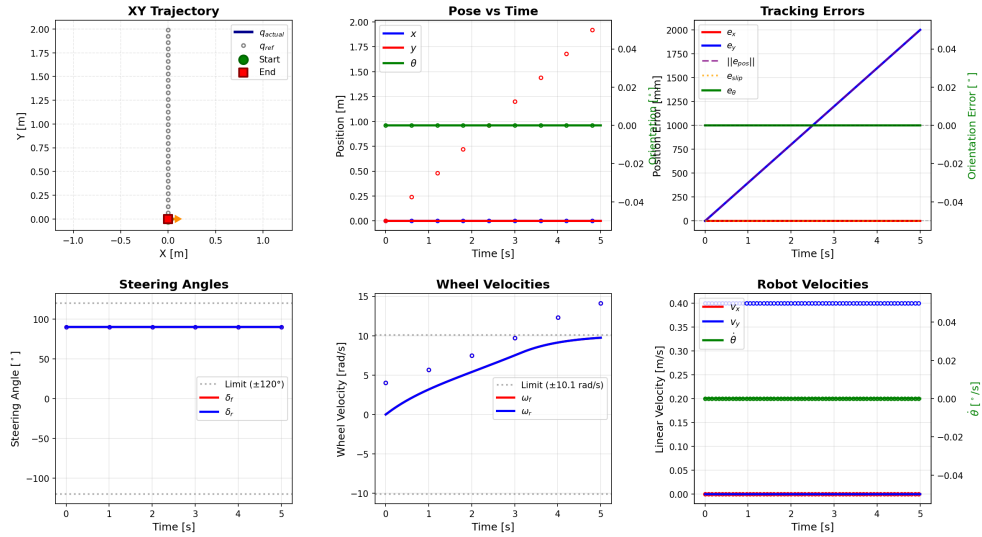


(a) Simulazione con modello biciclo sterzante per traiettoria Arc.

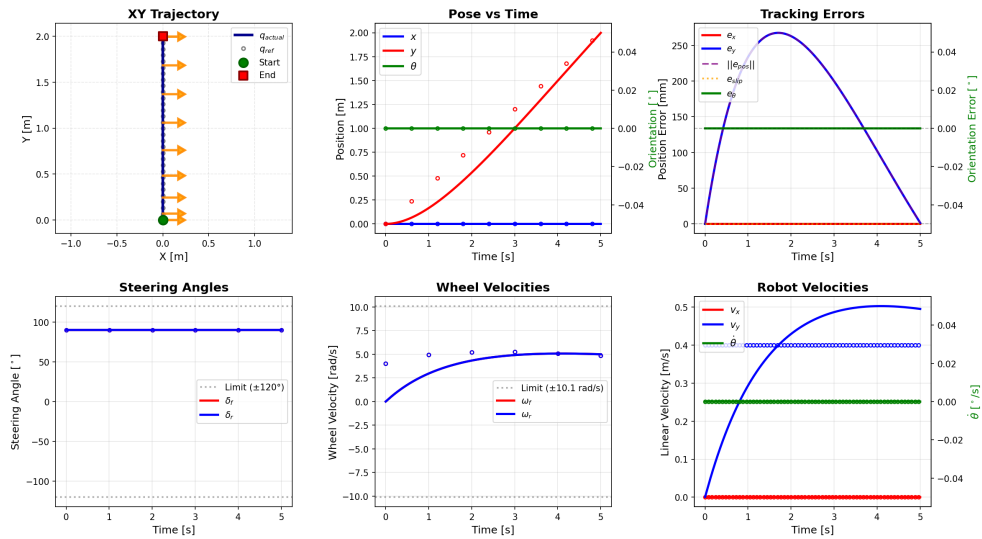


(b) Simulazione con modello a doppia ruota sterzante per traiettoria Arc.

Figura 4.9: Risultati delle simulazioni in feedback per la traiettoria circolare.

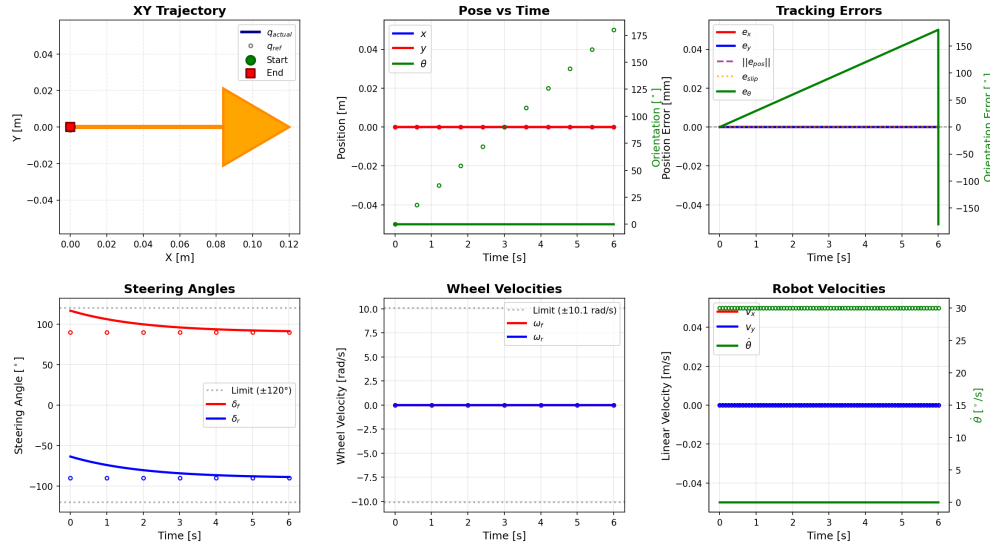


(a) Simulazione con modello biciclo sterzante per traiettoria Crab.

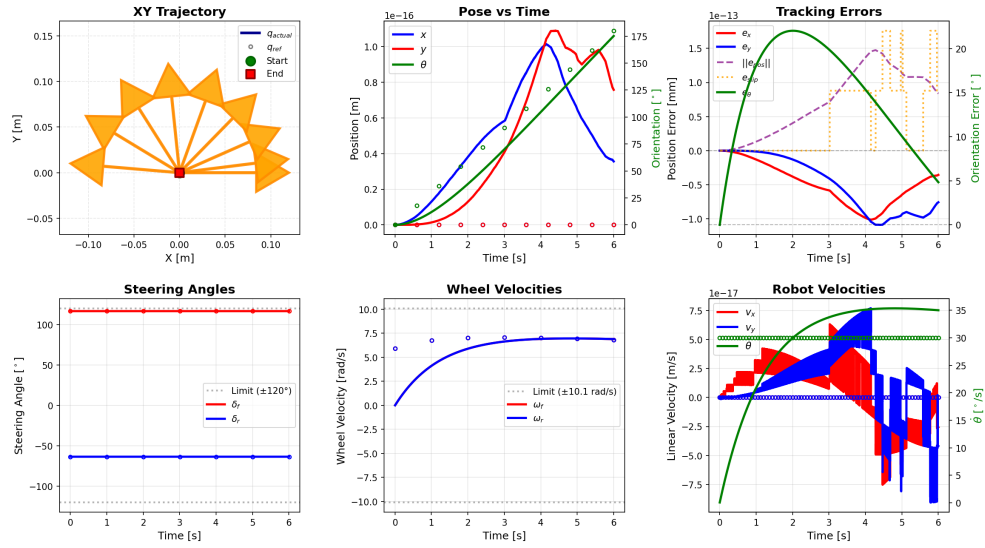


(b) Simulazione con modello a doppia ruota sterzante per traiettoria Crab.

Figura 4.10: Risultati delle simulazioni in feedback per la traiettoria traslatoria verso Nord.



(a) Simulazione con modello biciclo sterzante per traiettoria Spin.



(b) Simulazione con modello a doppia ruota sterzante per traiettoria Spin.

Figura 4.11: Risultati delle simulazioni in feedback per la traiettoria di rotazione sul posto.

Capitolo 5

Architettura software e somunizione

In questo capitolo viene descritta l'architettura software sviluppata per integrare tutto l'hardware caratterizzante il veicolo all'interno dell'ecosistema ROS 2, risolvendo il problema di compatibilità tra il protocollo proprietario dell'adattatore USB-CAN utilizzato e gli standard di comunicazione richiesti: comunicazione con protocollo CANOpen. Dopo aver descritto il problema di integrazione hardware-software, viene presentata la soluzione architetturale adottata, basata su un bridge di traduzione tra protocolli, e ne vengono dettagliate le componenti principali: la libreria di gestione del protocollo Waveshare e il bridge USB-SocketCAN.

5.1 Problema di integrazione hardware-software

Il sistema di controllo del veicolo si basa su una rete CANopen per la gestione degli azionamenti elettrici (motori di trazione e sterzo). Come indicato nella sottosezione 3.6.8, al fine di connettere il computer di bordo con i dispositivi CAN è stato scelto un dispositivo economico e ampiamente disponibile che permette di collegare il computer al bus CAN tramite porta USB.

Tuttavia, a differenza della maggior parte degli adattatori USB-CAN professionali (come Peak PCAN-USB o Kvaser), il dispositivo Waveshare non è compatibile con **SocketCAN**, l'interfaccia CAN nativa del kernel Linux.

Nota: *SocketCAN è il sottosistema del kernel Linux che implementa i protocolli CAN (Controller Area Network) come interfacce di rete standard. Permette di utilizzare le stesse API socket utilizzate per le comunicazioni di rete (TCP/IP) anche per i dispositivi CAN, garantendo un'interfaccia uniforme e semplificando lo sviluppo di applicazioni.*

L'adattatore Waveshare USB-CAN-A utilizza invece un **protocollo seriale proprietario**, documentato dal produttore, che incapsula i messaggi CAN in frame seriali con una struttura specifica. Questa scelta, pur mantenendo bassi i costi del dispositivo, introduce una problematica: i messaggi accettati dai driver e dal BMS di batteria sono messaggi CAN il cui contenuto è conforme allo standard applicativo di più alto livello e accettano messaggi conformi a tale standard. Inoltre, in prospettiva di utilizzo di pacchetti ROS 2 esistenti per la gestione di dispositivi CANopen (come `ros2_canopen`), o più in generale per l'integrazione di ulteriori dispositivi CAN, è preferibile predisporre un'interfaccia basata su uno standard comune.

I requisiti software per il sistema di controllo del veicolo sono i seguenti.

- **Integrazione con ROS 2:** lo stack software del robot si basa su ROS 2 per la gestione dei nodi di controllo, navigazione e localizzazione. Le interfacce usate devono essere compatibili con gli standard del framework per future integrazioni di pacchetti esistenti (come `Nav2`, `ros2_control`).
- **Accesso SocketCAN:** Lo scambio di messaggi tra lo strato applicativo e l'hardware CAN deve avvenire tramite interfaccia SocketCAN, in modo da poter utilizzare driver e librerie esistenti senza modifiche.
- **Comunicazione real-time:** il controllo degli azionamenti richiede latenze predicibili e basse (dell'ordine del millisecondo) per garantire prestazioni adeguate.
- **Affidabilità:** il sistema deve gestire correttamente errori di comunicazione, disconnessioni e stati anomali dei dispositivi.

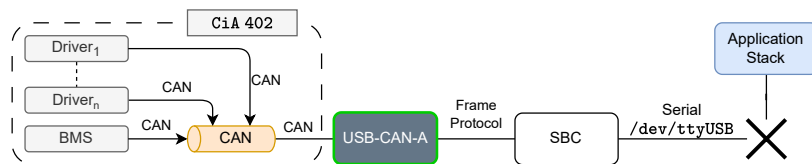


Figura 5.1: Rappresentazione del problema di integrazione: l'hardware (motori CAN e adattatore USB) utilizza un protocollo proprietario, mentre lo stack software ROS 2 richiede interfacce SocketCAN standard. È necessario un layer intermedio per colmare questa incompatibilità.

La soluzione a questo problema di incompatibilità consiste nell'implementazione di un **bridge software** che traduca in modo bidirezionale i messaggi tra il protocollo proprietario Waveshare (su porta seriale USB) e il protocollo SocketCAN standard. Questo approccio permette di mantenere l'intero stack software esistente inalterato, esponendo virtualmente l'adattatore Waveshare come se fosse un dispositivo SocketCAN nativo.

5.2 Architettura a livelli

L'architettura software sviluppata si articola su tre livelli principali, ciascuno con responsabilità ben definite:

1. **Livello Protocollo:** libreria C++ `waveshare_cpp` che implementa il protocollo seriale proprietario dell'adattatore Waveshare USB-CAN-A, permettendo di serializzare e deserializzare i frame CAN incapsulati all'interno dei frame Waveshare.
2. **Livello Bridge:** applicazione `usb_can_bridge` che traduce bidirezionalmente tra protocollo Waveshare (porta seriale USB) e SocketCAN (interfaccia virtuale).
3. **Livello Applicazione:** stack ROS 2 con nodi per le operazioni teleoperate, per l'elaborazione dei comandi di controllo, per l'esecuzione dell'algoritmo di SLAM e qualsiasi altra funzionalità di alto livello come un visualizzatore (RViz) o un'interfaccia utente.

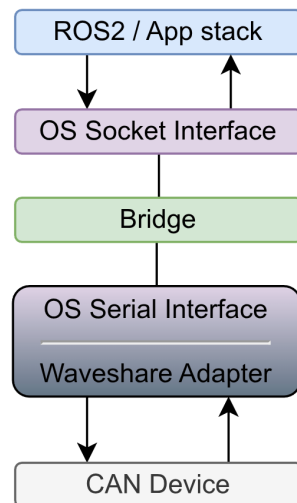


Figura 5.2: Architettura software a livelli. Il bridge USB-SocketCAN funge da layer di traduzione tra il protocollo proprietario e lo stack applicativo ROS 2, esponendo un'interfaccia SocketCAN virtuale.

Questa architettura modulare presenta diversi vantaggi.

- **Separazione delle responsabilità:** ogni livello ha un compito specifico e ben definito.
- **Riusabilità:** la libreria `waveshare_cpp` può essere utilizzata in altri progetti che necessitano di interfacciarsi con lo stesso hardware.
- **Testabilità:** ogni componente può essere testato indipendentemente tramite dependency injection e mock objects.
- **Manutenibilità:** modifiche a un livello non impattano gli altri, purché le interfacce rimangano compatibili.
- **Trasparenza:** lo stack applicativo ROS 2 non è consapevole dell'esistenza del protocollo proprietario e utilizza interfacce standard.

Nei paragrafi successivi vengono descritti in dettaglio il protocollo Waveshare, l'implementazione della libreria e del bridge, e infine l'integrazione con lo stack completo di controllo e comunicazione.

5.3 Protocollo Waveshare USB-CAN-A

Il protocollo utilizzato dall'adattatore Waveshare USB-CAN-A è un protocollo seriale proprietario documentato dal produttore. Il protocollo incapsula i messaggi CAN in frame seriali con una struttura definita, permettendo la configurazione del dispositivo e lo scambio di dati CAN su una connessione seriale USB.

5.3.1 Caratteristiche del protocollo

Il protocollo supporta le seguenti funzionalità.

- **CAN 2.0A e CAN 2.0B:** supporto sia per identificatori standard a 11 bit (CAN 2.0A) che per identificatori estesi a 29 bit (CAN 2.0B).
- **Data e Remote frame:** possibilità di inviare sia frame dati che frame remoti (utilizzati per richiedere trasmissione di dati da altri nodi).
- **Payload fino a 8 byte:** come da specifica CAN, ogni frame può contenere fino a 8 byte di dati.
- **Configurazione adattatore:** impostazione di baud rate CAN (da 5 kbps a 1 Mbps), modalità operativa (normale, loopback, silent), filtri e maschere per la ricezione selettiva.
- **Baud rate seriale:** configurazione del baud rate della porta seriale (da 9600 bps a 2 Mbps), indipendente dal baud rate CAN.

Il protocollo definisce tre tipologie di frame, ciascuna con una struttura specifica.

5.3.2 Tipologie di frame

Fixed Frame

I **Fixed Frame** hanno dimensione fissa di 20 byte e sono utilizzati sia per la configurazione dell'adattatore che per lo scambio di messaggi CAN. La struttura di un Fixed Frame per dati CAN è la seguente:

Tabella 5.1: Struttura Fixed Frame (20 byte totali).

Byte	Campo	Valore	Descrizione
0	START	0xAA	Start of transmission
1	HEADER	0x55	Header del frame
2	TYPE	0x01	Tipo frame (data)
3	CAN_VERSION	0x01/0x02	Standard (11-bit) / Extended (29-bit)
4	FORMAT	0x01/0x02	Data frame / Remote frame
5–8	ID	4 byte	Identificatore CAN (little-endian)
9	DLC	0–8	Data Length Code
10–17	DATA	8 byte	Payload (con padding se DLC < 8)
18	RESERVED	0x00	Riservato
19	CHECKSUM	1 byte	Checksum (somma byte 2–18, LSB)

Il campo **DLC** (Data Length Code) indica il numero di byte validi nel campo DATA. Se DLC è inferiore a 8, i byte rimanenti devono essere riempiti con zeri (*padding*). Il **checksum** è calcolato come la somma di tutti i byte da TYPE (byte 2) a RESERVED (byte 18), considerando solo il byte meno significativo del risultato.

I Fixed Frame sono utilizzati anche per la configurazione del dispositivo, con una struttura leggermente diversa che include campi per il baud rate CAN, i filtri e le maschere di ricezione, e la modalità operativa.

Variable Frame

I **Variable Frame** hanno dimensione variabile (da 5 a 15 byte) e sono utilizzati esclusivamente per lo scambio di messaggi CAN. A differenza dei Fixed Frame, non utilizzano padding e la dimensione del frame si adatta al numero effettivo di byte di dati. La struttura è la seguente:

Tabella 5.2: Struttura Variable Frame (5–15 byte totali).

Byte	Campo	Valore	Descrizione
0	START	0xAA	Start of transmission
1	TYPE	Variabile	Tipo codificato (vedi sotto)
2–3 o 2–5	ID	2 o 4 byte	ID CAN (11 o 29 bit, little-endian)
n o $n + 7$	DATA	0–8 byte	Payload (senza padding)
ultimo	END	0x55	End of transmission

Il campo **TYPE** nei Variable Frame ha una struttura a bit particolare:

$$\text{TYPE} = 0xC0 \mid (\text{IsExtended} \ll 5) \mid (\text{Format} \ll 4) \mid (\text{DLC} \& 0x0F) \quad (5.1)$$

dove:

- I due bit più significativi sono sempre 11 (costante 0xC0).
- **IsExtended** (bit 5): indica se l'ID è esteso (29 bit) o standard (11 bit).
- **Format** (bit 4): indica se è un data frame o remote frame.
- **DLC** (bit 3–0): numero di byte di dati (0–8).

I Variable Frame sono preferiti per la comunicazione in banda, poiché riducono l'overhead evitando il padding. Non utilizzano checksum, affidandosi ai meccanismi di rilevamento errori del protocollo CAN stesso.

Config Frame

I **Config Frame** hanno dimensione fissa di 20 byte e sono utilizzati esclusivamente per configurare i parametri operativi dell'adattatore USB-CAN. Devono essere inviati prima di iniziare qualsiasi trasmissione di dati CAN. La struttura include campi per:

- **Baud rate CAN**: valore da 0x01 a 0x0C, corrispondente a velocità da 5 kbps a 1 Mbps.
- **Filtro e maschera ID**: permettono la ricezione selettiva di messaggi CAN basata sull'identificatore.
- **Modalità operativa**: normale, loopback, silent (solo ascolto), o silent-loopback.
- **Ritrasmissione automatica**: abilita o disabilita la ritrasmissione automatica in caso di mancato acknowledge.

La configurazione tipica per il veicolo utilizza modalità normale, baud rate CAN a 1 Mbps, baud rate seriale a 2 Mbps, e ritrasmissione automatica abilitata.

5.4 Libreria `waveshare_cpp`

La libreria `waveshare_cpp` è una libreria C++17 che implementa la gestione completa del protocollo Waveshare descritto nella sezione precedente. La libreria fornisce classi per la costruzione, serializzazione, deserializzazione e validazione dei frame, oltre a un'interfaccia thread-safe per la comunicazione con l'adattatore USB.

5.4.1 Architettura della libreria

La libreria è organizzata secondo un design modulare e orientato agli oggetti, con le seguenti componenti principali.

- **Classi Frame:** `FixedFrame`, `VariableFrame` e `ConfigFrame` rappresentano i tre tipi di frame del protocollo. Ogni classe implementa metodi per la serializzazione (`to_bytes()`) e la deserializzazione (`from_bytes()`).
- **FrameBuilder:** interfaccia fluent per la costruzione type-safe dei frame, con validazione a compile-time dei parametri richiesti.
- **USBAdapter:** classe che gestisce la comunicazione con la porta seriale, fornendo un'API ad alto livello per l'invio e la ricezione di frame.
- **Helper classes:** forniscono funzioni di utilità per il calcolo dei checksum, la codifica/decodifica del campo TYPE, e la conversione tra frame Waveshare e strutture `can_frame` di SocketCAN.
- **Interfacce astratte:** `ISerialPort` e `ICANSocket` permettono dependency injection per il testing senza hardware reale.
- **Gestione degli errori:** gerarchia di eccezioni personalizzate per una gestione degli errori robusta e specifica.

5.4.2 Costruzione e validazione dei frame

La costruzione dei frame utilizza il pattern *builder* per garantire che tutti i campi obbligatori siano specificati prima della creazione dell'oggetto. Questo approccio, tipico della programmazione orientata agli oggetti moderna, permette una costruzione type-safe con validazione a compile-time, riducendo la possibilità di errori.

La deserializzazione dei frame ricevuti dalla porta seriale avviene attraverso metodi che validano la struttura (start byte, checksum se presente, end byte) e sollevano eccezioni specifiche in caso di errori di protocollo o corruzione dei dati. Questo meccanismo permette una gestione robusta degli errori a tutti i livelli dello stack software.

5.4.3 Gestione della porta seriale

La classe `USBAdapter` gestisce la comunicazione con l'adattatore USB attraverso un'implementazione thread-safe basata su tre mutex indipendenti:

- `state_mutex_`: protegge lo stato di configurazione del dispositivo (shared lock per letture concorrenti).

- `write_mutex_`: serializza le operazioni di scrittura sulla porta seriale.
- `read_mutex_`: serializza le operazioni di lettura dalla porta seriale.

Questo design permette operazioni di lettura e scrittura concorrenti, mentre previene deadlock grazie a un ordine di acquisizione gerarchico dei lock. L'interfaccia di `USBAdapter` fornisce metodi ad alto livello per:

- **Configurazione**: impostazione dei parametri operativi dell'adattatore (baud rate CAN e seriale, modalità operativa, filtri).
- **Invio frame**: trasmissione di messaggi CAN tramite Fixed o Variable Frame.
- **Ricezione frame**: ricezione bloccante con timeout configurabile per gestire correttamente i casi in cui non arrivano messaggi.

L'utilizzo tipico prevede l'apertura della porta seriale, l'invio di un Config Frame per configurare l'adattatore, e successivamente l'invio/ricezione continua di frame dati. La configurazione utilizzata per il veicolo prevede baud rate seriale a 2 Mbps, baud rate CAN a 1 Mbps, modalità normale e ritrasmissione automatica abilitata.

5.4.4 Conversione tra protocolli

Una classe helper dedicata fornisce metodi per la conversione bidirezionale tra frame Waveshare e la struttura `can_frame` utilizzata da SocketCAN. La conversione gestisce correttamente tutti i flag e le maschere necessarie:

- Flag di identificatore esteso per distinguere tra ID standard (11 bit) e estesi (29 bit).
- Flag di remote frame per frame che richiedono trasmissione dati da altri nodi.
- Maschere di ID e copia del payload con gestione corretta del DLC.

Questo componente è fondamentale per il bridge, in quanto permette la traduzione trasparente tra i due formati mantenendo intatta tutta l'informazione contenuta nei messaggi CAN.

5.4.5 Testing e validazione

La libreria è progettata secondo principi di testabilità: tutte le dipendenze da hardware (porta seriale, socket CAN) sono iniettate attraverso interfacce astratte. Questo pattern di design, noto come *dependency injection*, permette di sostituire le implementazioni reali con oggetti mock durante i test, consentendo di verificare la logica software senza necessità di hardware fisico.

La suite di test della libreria include 132 unit test che coprono:

- Serializzazione e deserializzazione di tutti i tipi di frame.
- Validazione dei checksum e gestione degli errori.
- Costruzione dei frame con casi validi e casi limite.

- Conversione tra formati Waveshare e SocketCAN.
- Gestione delle eccezioni e dei timeout.

Tutti i test sono eseguibili senza hardware, garantendo un ciclo di sviluppo rapido e robusto. Questo approccio ha permesso di identificare e correggere numerosi edge case prima del testing sull'hardware reale.

5.5 Bridge USB-SocketCAN

Il bridge USB-SocketCAN è un'applicazione standalone che traduce bidirezionalmente i messaggi tra l'adattatore Waveshare (tramite la libreria `waveshare_` `cpp`) e un'interfaccia SocketCAN virtuale. Il bridge permette di esporre l'adattatore USB come se fosse un dispositivo CAN nativo del kernel Linux, garantendo compatibilità con tutte le applicazioni che utilizzano l'API SocketCAN standard.

5.5.1 Architettura del bridge

Il bridge è implementato dalla classe `SocketCANBridge` e utilizza un'architettura multi-thread con due thread indipendenti per il forwarding bidirezionale:

- **Thread USB → SocketCAN:** legge frame dall'adattatore USB (tramite `USBAdapter`), li converte in strutture `can_frame`, e li invia all'interfaccia SocketCAN.
- **Thread SocketCAN → USB:** legge frame dall'interfaccia SocketCAN, li converte in `VariableFrame`, e li invia all'adattatore USB.

I due thread operano in modo completamente indipendente, senza condivisione di risorse e quindi senza necessità di sincronizzazione. Questa architettura *lock-free* riduce la complessità della gestione della concorrenza e facilita la manutenzione del codice.

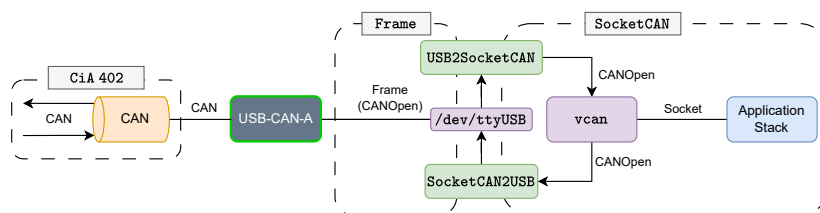


Figura 5.3: Architettura del bridge USB-SocketCAN. I due thread di forwarding operano indipendentemente, traducendo frame in modo bidirezionale tra protocollo Waveshare e SocketCAN.

5.5.2 Configurazione del bridge

La configurazione del bridge avviene attraverso la classe `BridgeConfig`, che supporta tre modalità di caricamento:

1. **File JSON:** configurazione da file, utile per configurazioni persistenti.

2. **Variabili d'ambiente:** permette configurazione runtime senza modificare file.
3. **Valori predefiniti:** configurazione di base per testing rapido.

I parametri configurabili includono:

- Percorso del device seriale (tipicamente `/dev/ttyUSB0` in Linux).
- Baud rate della porta seriale (default: 2 Mbps) e del bus CAN (default: 1 Mbps).
- Nome dell'interfaccia SocketCAN virtuale (tipicamente `vcan0` per testing o `can0` per produzione).
- Modalità operativa del controller CAN (normale, loopback per test, silent per monitoraggio).
- Abilitazione della ritrasmissione automatica dei frame CAN in caso di errore.
- Timeout per le operazioni di lettura (tipicamente 100 ms).

La configurazione può essere specificata tramite file JSON per persistenza o tramite variabili d'ambiente per flessibilità in fase di deployment.

5.5.3 Setup interfaccia virtuale SocketCAN

Prima di avviare il bridge, è necessario creare e attivare un'interfaccia SocketCAN virtuale nel kernel Linux. Questo processo richiede il caricamento del modulo kernel `vcan`, la creazione dell'interfaccia virtuale con un nome specifico (tipicamente `vcan0`), e la sua attivazione.

L'interfaccia virtuale si comporta come un'interfaccia CAN reale ma opera interamente in software, permettendo test e sviluppo senza hardware CAN fisico. Per l'utilizzo in produzione, il bridge si connette a questa interfaccia virtuale e inoltra i messaggi all'adattatore USB fisico, realizzando la traduzione tra i due protocolli in modo trasparente per le applicazioni.

5.5.4 Funzionamento del bridge

Una volta avviato, il bridge esegue le seguenti operazioni:

1. Inizializzazione:

- Apertura della porta seriale USB e configurazione tramite `ConfigFrame`.
- Apertura del socket SocketCAN e binding all'interfaccia virtuale.
- Avvio dei due thread di forwarding.

2. Thread USB → SocketCAN:

- Ciclo continuo di lettura da USB con timeout configurabile.
- Conversione da `VariableFrame` a `struct can_frame`.
- Scrittura sulla socket SocketCAN.
- Aggiornamento statistiche (frame inoltrati, errori).

3. Thread SocketCAN → USB:

- Ciclo continuo di lettura dalla socket SocketCAN con timeout.
- Conversione da `struct can_frame` a `VariableFrame`.
- Scrittura sulla porta USB tramite `USBAdapter`.
- Aggiornamento statistiche.

4. Gestione degli errori:

- Timeout: vengono ignorati e il ciclo riprende (permettono controllo periodico dello stato).
- Errori di I/O: vengono loggati ma non interrompono il bridge.
- Errori critici (device disconnesso): il bridge termina con codice di errore.

Il bridge fornisce anche statistiche in tempo reale accessibili tramite callback opzionali:

- Numero di frame inoltrati in ciascuna direzione.
- Numero di errori e timeout.
- Rate di trasmissione (frame/secondo).

5.6 Integrazione con lo stack ROS 2

L'architettura completa del sistema di controllo integra il bridge USB-SocketCAN con lo stack ROS 2 per il controllo del veicolo e l'interfaccia con gli algoritmi di navigazione.

5.6.1 Stack completo di comunicazione

Lo stack software si articola come segue:

1. **Hardware layer:** motori brushless con inverter CANopen, sensori (LIDAR, IMU), adattatore USB-CAN.
2. **Protocol layer:** libreria `waveshare_cpp` per gestione protocollo proprietario.
3. **Bridge layer:** applicazione `usb_can_bridge` per traduzione USB ↔ SocketCAN.
4. **Communication layer:** interfaccia SocketCAN per l'invio e la ricezione di messaggi CAN dal bus.
5. **Control layer:** nodi ROS 2 per controllo cinematico (PID, feedforward), generazione riferimenti, telemetria.
6. **Navigation layer:** pacchetti per SLAM (`fast_lio`), path planning (`nav2`), mapping.

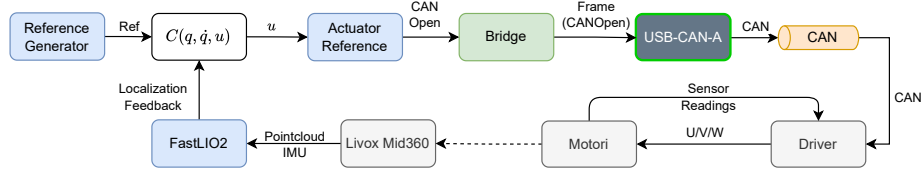


Figura 5.4: Stack completo di comunicazione e controllo. Il bridge USB-SocketCAN permette l'integrazione trasparente dell'hardware con lo stack ROS 2 standard.

5.6.2 Vantaggi dell'architettura

L'architettura sviluppata presenta i seguenti vantaggi:

- **Trasparenza:** lo stack ROS 2 non è consapevole del protocollo proprietario e utilizza interfacce standard.
- **Riusabilità:** il bridge può essere utilizzato con altri dispositivi Waveshare o adattato per protocolli simili.
- **Flessibilità:** è possibile sostituire l'adattatore Waveshare con dispositivi SocketCAN nativi senza modificare il software applicativo.
- **Testabilità:** ogni componente può essere testato indipendentemente usando mock e simulatori.
- **Manutenibilità:** la separazione in layer rende semplici le modifiche e gli aggiornamenti.

In sintesi, l'architettura software sviluppata ha permesso di risolvere efficacemente il problema di integrazione hardware-software, mantenendo elevati standard di qualità del codice e testabilità.

Capitolo 6

Conclusioni e sviluppi futuri

Questo lavoro di tesi ha affrontato la progettazione e lo sviluppo di un veicolo autonomo a guida automatica per la movimentazione di materiali all'interno dell'ambiente industriale dello stabilimento Daikin Applied Europe di Ariccia. L'obiettivo principale è stato quello di creare un dimostratore tecnologico economico in grado di validare l'applicabilità di tecnologie robotiche avanzate in un contesto produttivo caratterizzato da layout variabili, ostacoli dinamici e necessità logistiche complesse tipiche della produzione make-to-order.

Il progetto ha coperto tutte le fasi principali dello sviluppo di un sistema robotico mobile: dall'analisi dei requisiti e la selezione dei componenti hardware, alla modellazione cinematica e alla sintesi di leggi di controllo, fino all'implementazione dell'architettura software per la comunicazione e l'integrazione con l'ecosistema ROS 2.

6.1 Risultati raggiunti

I principali risultati conseguiti durante il lavoro di tesi possono essere sintetizzati come segue.

6.1.1 Analisi e progettazione del sistema

È stata condotta un'analisi approfondita dei requisiti per il veicolo autonomo, considerando le specificità dell'ambiente industriale target. Questo ha portato alla definizione di specifiche tecniche dettagliate in termini di capacità di carico (1000 kg), velocità massima (1 m/s), autonomia operativa (4 ore), e manovrabilità (traslazioni rettilinee, curve, rotazioni sul posto, traslazioni laterali).

La selezione dei componenti hardware è stata effettuata con un criterio di ottimizzazione del rapporto costo-prestazioni, privilegiando soluzioni economiche e ampiamente disponibili che mantenessero comunque caratteristiche adeguate per un dimostratore tecnologico. Questo approccio ha permesso di contenere i costi complessivi del progetto pur garantendo funzionalità complete.

6.1.2 Modellazione cinematica e controllo

È stato sviluppato e validato in simulazione un modello cinematico per il veicolo a doppia ruota sterzante indipendente. Il modello è stato confrontato con l'approccio semplificato a bicicletta, evidenziando vantaggi e limitazioni

di ciascuna soluzione. In particolare, il modello bisteer ha mostrato maggiore accuratezza nella rappresentazione del comportamento reale del veicolo durante manovre complesse come rotazioni sul posto e traslazioni laterali.

Sono state progettate e testate leggi di controllo in retroazione basate su controllori PID con azione feedforward per il tracking di traiettorie. Le simulazioni hanno dimostrato l'efficacia dell'approccio proposto nel garantire inseguimento accurato dei riferimenti di velocità e orientamento.

6.1.3 Architettura software e comunicazione

È stata progettata e implementata un'architettura software modulare per l'integrazione del veicolo con lo stack ROS 2. Il contributo principale in questo ambito riguarda lo sviluppo di una soluzione al problema di incompatibilità tra il protocollo proprietario dell'adattatore USB-CAN utilizzato (Waveshare USB-CAN-A) e l'interfaccia SocketCAN standard richiesta dai pacchetti ROS 2.

La soluzione adottata si basa su un'architettura a tre livelli:

- Una libreria C++17 (`waveshare_cpp`) che implementa la gestione completa del protocollo proprietario, con particolare attenzione alla testabilità tramite dependency injection e alla thread-safety per applicazioni real-time.
- Un bridge software (`usb_can_bridge`) che traduce bidirezionalmente i messaggi tra protocollo Waveshare e SocketCAN, esponendo l'adattatore come se fosse un dispositivo CAN nativo del kernel Linux.
- L'integrazione con lo stack ROS 2, che può utilizzare interfacce standard senza consapevolezza del protocollo sottostante.

Questa architettura garantisce trasparenza, riusabilità e manutenibilità, permettendo inoltre di sostituire facilmente l'hardware di comunicazione senza modifiche al software applicativo.

6.1.4 Stato attuale del progetto

Al termine del lavoro di tesi, il progetto si trova nelle seguenti condizioni:

- **Software:** la libreria di comunicazione con l'adattatore USB-CAN-A è stata completamente sviluppata e testata con una suite di 132 unit test. L'architettura software è definita e il bridge USB-SocketCAN è implementato e funzionante. Mancano i nodi ROS 2 per la generazione di traiettorie e l'invio di comandi ai motori, che rappresentano lo sviluppo naturale del lavoro svolto.
- **Hardware:** tutto l'hardware necessario (motori brushless, inverter, sensori LiDAR e IMU, batteria, telaio, adattatore USB-CAN) è stato acquistato. Sono stati effettuati test di comunicazione con gli inverter su un banco di prova dedicato per validare il protocollo di comunicazione e verificare il corretto funzionamento dei componenti. Il veicolo è in attesa di assemblaggio finale e messa in opera.

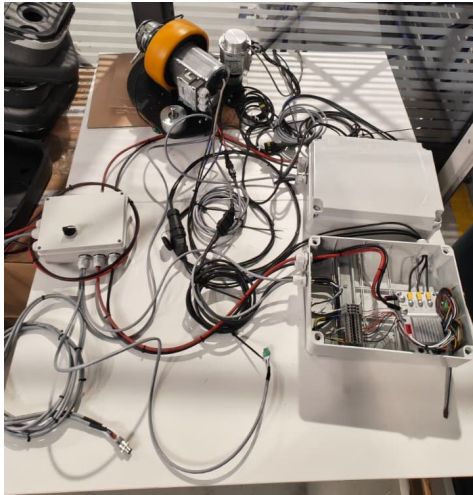


Figura 6.1: Banco di prova per test di comunicazione con una motoruota



Figura 6.2: Telaio del veicolo e componenti strutturali acquistati.

6.2 Sviluppi futuri

Il completamento del progetto e la sua evoluzione verso un sistema operativo richiedono ulteriori attività che possono essere raggruppate in tre categorie principali.

6.2.1 Implementazione e integrazione

Il prossimo passo immediato consiste nell'assemblaggio completo del veicolo e nel cablaggio di tutti i componenti hardware. Questo include il montaggio meccanico delle motoruote sul telaio, l'installazione del sistema di alimentazione, il posizionamento dei sensori (LiDAR, IMU) e la realizzazione del cablaggio per la comunicazione CAN e l'alimentazione.

Dal punto di vista software, sarà necessario completare lo stack ROS 2 con i nodi mancanti per la generazione di traiettorie, l'interfacciamento con i driver dei motori tramite protocollo CAN, e l'integrazione degli algoritmi di controllo sviluppati e validati in simulazione. Questo permetterà di chiudere il ciclo di controllo e abilitare la navigazione autonoma del veicolo.

6.2.2 Validazione sperimentale

Una volta completato l'assemblaggio e l'integrazione software, sarà fondamentale condurre una campagna di test su campo presso lo stabilimento Daikin di Ariccia. Questa fase permetterà di:

- Validare il comportamento del veicolo in condizioni reali, confrontando le prestazioni misurate con quelle previste dalle simulazioni.
- Misurare parametri chiave come precisione nel tracking delle traiettorie, autonomia operativa effettiva, affidabilità della comunicazione e robustezza del sistema di localizzazione.

- Ottimizzare i parametri dei controllori sulla base dei dati sperimentali, tenendo conto di effetti non modellati come attriti variabili, deformazioni del telaio, e caratteristiche reali degli attuatori.
- Valutare l'interazione del veicolo con l'ambiente produttivo reale, inclusa la capacità di evitare ostacoli dinamici (operatori, carrelli) e di operare in spazi ristretti tipici dell'ambiente industriale.

6.2.3 Estensioni e miglioramenti

Una volta validato il dimostratore tecnologico, esistono diverse direzioni per l'evoluzione del sistema verso una soluzione industriale completa:

- **Fleet management:** sviluppo di un sistema di gestione di flotte multi-robot per il coordinamento di più veicoli autonomi operanti contemporaneamente nello stabilimento. Questo richiede algoritmi per la pianificazione delle missioni, la risoluzione di conflitti e deadlock, e l'ottimizzazione delle risorse.
- **Integrazione con sistemi aziendali:** collegamento del sistema di fleet management con il Manufacturing Execution System (MES) e l'Enterprise Resource Planning (ERP) aziendali per automatizzare completamente il processo di movimentazione materiali, dalla richiesta alla consegna, con tracciabilità completa.
- **Sensoristica avanzata:** integrazione di sensori aggiuntivi per migliorare sicurezza e robustezza. Questo include telecamere RGB-D per la percezione tridimensionale dell'ambiente, safety laser scanner certificati per applicazioni industriali, e sensori di prossimità per la rilevazione di ostacoli a bassa quota.
- **Algoritmi di controllo avanzati:** sperimentazione con modelli cinematici alternativi, inclusione di effetti dinamici (inerzia, forze di contatto), e implementazione di architetture di controllo più sofisticate come Model Predictive Control (MPC) per migliorare le prestazioni in termini di precisione e velocità di risposta.

6.3 Considerazioni finali

Il lavoro svolto ha dimostrato la fattibilità tecnica ed economica di sviluppare un veicolo autonomo per applicazioni industriali utilizzando componenti commerciali e soluzioni software open-source. L'approccio metodologico seguito, basato su una chiara separazione tra modellazione, simulazione, implementazione software e validazione hardware, ha permesso di ottenere risultati robusti e facilmente estendibili.

Il progetto rappresenta un primo passo concreto verso l'automazione della logistica interna presso lo stabilimento Daikin, con potenziali benefici in termini di riduzione dei costi operativi, aumento dell'efficienza produttiva, e miglioramento delle condizioni di lavoro degli operatori. Gli sviluppi futuri individuati tracciano una roadmap chiara per la trasformazione del dimostratore in un sistema operativo pronto per l'impiego industriale.

Bibliografia

- [1] Johann Borenstein and Yoram Koren. The vector field histogram—fast obstacle avoidance for mobile robots. *IEEE Transactions on Robotics and Automation*, 7(3):278–288, 1991.
- [2] CAN in Automation (CiA). *CANopen Device Profile for Drives and Motion Control*, 2002.
- [3] CAN in Automation (CiA). *CANopen Application Layer and Communication Profile*, 2011. Version 4.2.0.
- [4] International Electrotechnical Commission. Industrial communication networks — fieldbus specifications — part 3-12: Data-link layer service definition — type 12 elements (ethercat). Standard IEC 61158-3-12, International Electrotechnical Commission, 2019.
- [5] International Electrotechnical Commission. Industrial communication networks — fieldbus specifications — part 6-10: Application layer protocol specification — type 10 elements (profinet). Standard IEC 61158-6-10, International Electrotechnical Commission, 2019.
- [6] A. De Luca, G. Oriolo, and C. Samson. Feedback control of a nonholonomic car-like robot. In Jean-Paul Laumond, editor, *Robot Motion Planning and Control*, number 229 in Lecture Notes in Control and Information Sciences. Springer, 1998.
- [7] Edsger W. Dijkstra. A note on two problems in connexion with graphs. *Numerische Mathematik*, 1(1):269–271, 1959.
- [8] Hugh Durrant-Whyte and Tim Bailey. Simultaneous localization and mapping: Part i. *IEEE Robotics & Automation Magazine*, 13(2):99–110, 2006.
- [9] International Organization for Standardization. Road vehicles — controller area network (can) — part 1: Data link layer and physical signalling. Standard ISO 11898-1:2015, International Organization for Standardization, 2015.
- [10] Dieter Fox, Wolfram Burgard, and Sebastian Thrun. The dynamic window approach to collision avoidance. *IEEE Robotics & Automation Magazine*, 4(1):23–33, 1997.
- [11] Giorgio Grisetti, Rainer Kümmerle, Cyrill Stachniss, and Wolfram Burgard. A tutorial on graph-based slam. *IEEE Intelligent Transportation Systems Magazine*, 2(4):31–43, 2010.
- [12] Peter E. Hart, Nils J. Nilsson, and Bertram Raphael. A formal basis for the heuristic determination of minimum cost paths. *IEEE Transactions on Systems Science and Cybernetics*, 4(2):100–107, 1968.

- [13] Yoichi Kanayama, Yoshihiko Kimura, Fumio Miyazaki, and Tetsuo Noguchi. A stable tracking control method for an autonomous mobile robot. In *IEEE International Conference on Robotics and Automation*, pages 384–389, 1990.
- [14] Oussama Khatib. Real-time obstacle avoidance for manipulators and mobile robots. *The International Journal of Robotics Research*, 5(1):90–98, 1986.
- [15] Steven M. LaValle. Rapidly-exploring random trees: A new tool for path planning. In *Technical Report, Computer Science Department, Iowa State University*, 1998.
- [16] Modbus Organization. *Modbus Messaging on TCP/IP Implementation Guide V1.0b*, 2006.
- [17] Modbus Organization. *Modbus Application Protocol Specification V1.1b3*, 2012.
- [18] Sebastian Thrun, Dieter Fox, Wolfram Burgard, and Frank Dellaert. Robust monte carlo localization for mobile robots. *Artificial Intelligence*, 128(1-2):99–141, 2001.
- [19] H. Tourajizadeh, M. Sarvari, and A. S. Ordoo. Modeling and optimal control of 4 wheel steering vehicle using lqr and its comparison with 2 wheel steering vehicle. *International Journal of Robotics*, 9(1):20–32, 2020.
- [20] Wei Xu, Yixi Cai, Dongjiao He, Jiarong Lin, and Fu Zhang. Fast-llo2: Fast direct lidar-inertial odometry. *IEEE Transactions on Robotics*, 38(4):2053–2073, 2022.